

UNIVERSIDAD ESAN

FACULTAD DE INGENIERÍA

INGENIERÍA DE TECNOLOGÍA DE INFORMACIÓN Y SISTEMAS

Desarrollo de un modelo de clasificación mediante inteligencia artificial de vectores de arbovirosis del Dengue, Zika y Chikungunya

Trabajo de Tesis 2

Yorgie Peter Herrera Yactayo

Asesora: Palomares Pecho Jessica Margarita

Lima, 3 de julio del 2023

Desarrollo de un modelo de clasificación mediante inteligencia artificial de vectores de arbovirosis del Dengue, Zika y Chikungunya

DEDICATORIA

Dedico primero esta tesis a Dios, Jesús y la Virgen María, quienes fomentaron una gran inspiración en mí para el desarrollo de esta investigación. A mis padres y hermanos que siempre me motivaron a seguir adelante. A mi asesora de tesis y profesores de la facultad de ingeniería que me brindaron su apoyo y su confianza en mí. En general, a todos los mencionados les agradezco de todo corazón y les hago presente esta dedicatoria.

Índice

CAPÍTULO I: PLANTEAMIENTO DEL PROBLEMA 14
1.1. Descripción de la realidad problemática 14
1.2. Formulación del problema 19
1.2.1. Problema General 19
1.2.2. Problemas Específicos 19
1.3. Objetivos de la Investigación 20
1.3.1. Objetivos General 20
1.3.2. Objetivos Específicos 20
1.4. Justificación de la Investigación 20
1.4.1. Teórico 20
1.4.2. Práctico 22
1.4.3. Metodológico 22
1.5. Delimitación del Estudio 24
1.5.1. Espacial 24
1.5.2. Temporal 24
1.5.3. Conceptual 24
CAPÍTULO II: MARCO TEÓRICO 26
2.1. Antecedentes de la Investigación 26
2.2. Bases Teóricas 46
2.3. Marco Conceptual o Contexto de Investigación 71
2.4. Hipótesis 75
CAPÍTULO III: METODOLOGÍA 77
3.1. Diseño de la Investigación 77
3.1.1. Diseño 77
3.1.2. Alcance 77
3.1.3. Enfoque 78
3.1.4. Selección de la muestra 78
3.1.5. Operacionalización de Variables 80
3.1.6. Técnicas de recolección de datos 81
3.2. Metodología de la implementación de la solución 81
3.2.1. Obtención del Dataset 84

3.2.2. Pre procesamiento 89
3.2.3. Aplicación y entrenamiento de los modelos de IA 98
3.3. Metodología para la medición de resultados de la implementación 109
3.3.1. Accuracy 109
3.3.2. Recall 110
3.3.3. Precisión 110
3.3.4. F1-Score 110
3.3.5. Especificidad 111
3.3.6. AUC 111
3.4. Cronograma de actividades y presupuesto 112
3.4.1. Cronograma 112
3.4.2. Presupuesto 113
CAPÍTULO IV: DESARROLLO DEL EXPERIMENTO 114
4.1. Obtención del Dataset 114
4.1.1. Obtención del primer dataset 114
4.1.2. Obtención del segundo dataset 115
4.1.3. Obtención del tercer dataset 116
4.1.4. Obtención del cuarto dataset 117
4.1.5. Obtención del quinto dataset 118
4.2. Pre procesamiento 129
4.2.1. Redimensionamiento 129
4.2.2. Balanceo de datos 131
4.2.3. Aumento de datos 134
4.2.4. Normalización de datos 135
4.3. Metodología 136
4.3.1. Construcción del modelo VGG16 136
4.3.2. Construcción del modelo VGG19 143
4.3.3. Construcción del modelo ResNet50 151
4.3.4. Construcción del modelo Vision Transformers 159
4.4. Análisis de los resultados de los modelos implementados 167
4.4.1. Resultados del modelo VGG16 167
4.4.2. Resultado del modelo VGG19 167
4.4.3. Resultado del modelo ResNet50 168

4.4.4. Resultado del modelo Vision Transformers 168
CAPÍTULO V: CONCLUSIONES Y RECOMENDACIONES 170
6.1. Conclusiones 170
6.2 Recomendaciones 172
Referencias 174
ANEXOS 189

Índice de Figuras

Figura 1. Tipos de vectores 16
Figura 2. Dataset utilizado en el artículo de investigación 27
Figura 3. Metodología del pre procesamiento de los datos 27
Figura 4. Metodología del artículo de investigación 28
Figura 5. Región de interés de las alas de los insectos para la extracción de características 31
Figura 6. Imágenes de microscopio de larvas de Aedes Aegypti y Aedes Albopictus 32
Figura 7. Metodología de la técnica Grad-CAM 33
Figura 8. Gráfico del entrenamiento y validación del modelo DenseNet 121 34
Figura 9. Imágenes de los insectos de tipo Aedes y otras especies 35
Figura 10. Pasos de realizados en el pre procesamiento del Dataset 36
Figura 11. Metodología del artículo de investigación 37
Figura 12. Metodología del artículo de investigación 39
Figura 13. Data Augmentation del dataset de las especies Aedes, Anopheles y Culex utilizadas por el artículo de investigación 40
Figura 14. Dataset de los insectos Albopictus. 43
Figura 15. Metodología de la técnica Grad-CAM 44
Figura 16. Muestra de la técnica Grad-CAM 45
Figura 17. Etapas de desarrollo estándar de un modelo de inteligencia artificial 48
Figura 18. Distribución Global del Dengue, Zika y Chikungunya 50
Figura 19. Morfología de un insecto de tipo culícido 51
Figura 20. Tipos de serotipos del virus del dengue 53
Figura 21. Ciclo de vida del insecto de tipo culícido de la familia Aedes 54
Figura 22. Etapa de incubación del virus del Dengue en el ser humano 55
Figura 23. Número de casos de dengue por semana epidemiológica del 2018 al 2022 56
Figura 24. Número de casos de Chikungunya en el Perú 2018 - *2022 59
Figura 25. Número de casos de Zika en el Perú 2018 - *2021 60
Figura 26. Marco de referencia de la visión artificial 62
Figura 27. Ubicación del pixel en un sistema de coordenadas 63
Figura 28. Ubicación del pixel en un sistema de coordenadas 63
Figura 29. Ejemplo de carga de una imagen en código Python 64

Figura 30. Keypoints en una imagen a nivel vector 65
Figura 31. Esquema de análisis de imágenes en una red neuronal 65
Figura 32. Línea de tiempo y aparición del Deep Learning 66
Figura 33. Esquema del modelo de Deep Learning 67
Figura 34. Esquema de redes neuronales 68
Figura 35. Matriz de confusión 68
Figura 36. Curva ROC 70
Figura 37. Filtro especial para imagen en el pre procesamiento 70
Figura 38. Flujo de datos para la calidad 71
Figura 39. Conjunto de imágenes de insectos de la familia Culex 81
Figura 40. Metodología de la investigación 83
Figura 41. Imágenes de las especies Aedes y Culex 85
Figura 42. Imagen de la especie Aedes Aegypti y Aedes Albopictus 86
Figura 43. Imagen de la especie Aedes y Culex 86
Figura 44. Conjunto de imágenes de insectos de la familia Culex 87
Figura 45. Conjunto de imágenes de insectos de la familia Culex 89
Figura 46. Pseudocódigo del proceso de web scraping 89
Figura 47. Cálculo del redimensionamiento a partir de un punto pixel 91
Figura 48. Cálculo del balanceo de datos - SMOTE 93
Figura 48. Implementación del modelo VGG16 por Transferer Learning 99
Figura 49. Estructura del modelo VGG16 por Transferer Learning 99
Figura 50. Implementación del modelo VGG19 por Transferer Learning 102
Figura 51. Estructura del modelo VGG19 por Transferer Learning 103
Estructura del modelo VGG19 por Transferer Learning. 103
Figura 52. Implementación del modelo ResNet50 por Transferer Learning 105
Figura 53. Implementación del modelo Vision Transformers por Transferer Learning 107
Figura 54. Descripción de las actividades del presente proyecto de investigación 112

Índice de Tablas

Tabla 1. Dataset utilizado en el artículo de investigación 30
Tabla 2. Hiperparámetros del modelo DenseNet 121 33
Tabla 3. Resultados de Accuracy de los modelos de deep learning implementados.....37
Tabla 4. Resultados de otros indicadores de modelos de deep learning implementados 38
Tabla 5. Hiperparámetros del modelo implementado.....41
Tabla 6. Resultados de los modelos implementados 42
Tabla 7. Resultados de los modelos VGG16 y ResNet50.....45
Tabla 8. Número de casos de dengue por departamento 56
Tabla 9. Distribución de casos de dengue por grupos de edades en el Perú en el 2022...57
Tabla 10. Descripción de los datos originales y los datos aumentados 79
Tabla 12. Variables de la investigación.....94
Tabla 13. Información de presupuesto de la investigación 113
Tabla 14. Resumen de Resultado de Indicadores 113

Resumen

En los últimos años la presencia de la enfermedad del Dengue, el Zika y el Chikungunya ha aumentado en distintas regiones de Latinoamérica y ha provocado un gran número de pérdidas humanas debido al grado de mortalidad que tienen estas infecciones. En el Perú, la enfermedad del Dengue ha causado un gran número de muertes en distintas regiones del país. El origen del contagio de estas enfermedades es debido a la picadura de un tipo de mosquito denominado Aedes que trasmite estos 3 tipos de virus entre los seres humanos. Este mosquito suele estar presente en el mismo ambiente que el mosquito común denominado Culex. Por este motivo, el objetivo de esta investigación es desarrollar un modelo de inteligencia artificial que a partir de una imagen puede clasificar a los mosquitos Aedes y Culex. En la cual, mediante esta clasificación conocer la presencia del mosquito Aedes y tomar acciones preventivas. El presente trabajo es de tipo experimental, explicativo y de un enfoque cuantitativo con la implementación de las fases de adquisición de datos, pre procesamiento, construcción de 4 modelos de aprendizaje profundo que son VGG16, VGG19, ResNet50 y Vision Transformers. Por último como última fase la selección del mejor modelo. Los resultados indicaros que los 4 modelos son adecuados para esta tarea de clasificación. Sin embargo, el modelo de Vision Transformes es más óptimo por sus mejores resultados con un Accuracy de 97.98%, 97.93% de precisión, 97.73% de recall, 97.83% de F1-score, 97.94% de especificidad y un 97.99% de AUC.

Palabras claves: Dengue, Arbovirosis, procesamiento de imágenes, visión computacional, aprendizaje profundo.

Abstract

In recent years, the presence of Dengue, Zika and Chikungunya diseases has increased in different regions of Latin America and has caused a large number of human losses due to the degree of mortality that these infections have. In Peru, Dengue disease has caused a large number of deaths in different regions of the country. The origin of the contagion of these diseases is due to the bite of a type of mosquito called Aedes that transmits these 3 types of viruses between humans. This mosquito is usually present in the same environment as the common Culex mosquito. For this reason, the objective of this research is to develop an artificial intelligence model that can classify Aedes and Culex mosquitoes from an image. In which, through this classification, know the presence of the Aedes mosquito and take preventive actions. The present work is of an experimental, explanatory type and of a quantitative approach with the implementation of the phases of data acquisition, pre-processing, construction of 4 deep learning models that are VGG16, VGG19, ResNet50 and Vision Transformers. Finally, as the last phase, the selection of the best model. The results indicate that the 4 models are suitable for this classification task. However, the Vision Transformes model is more optimal due to its better results with an Accuracy of 97.98%, 97.93% precision, 97.73% recall, 97.83% F1-score, 97.94% specificity and 97.99% AUC.

Keywords: Dengue, Arbovirosis, image processing, computer vision, deep learning.

Introducción

El avance y desarrollo de la tecnología en años recientes, especialmente la ciencia computacional, ha tenido un gran impacto en distintos actividades humanas. En esa línea de desarrollo, una disciplina importante de ese campo que ha evolucionado a grandes pasos es la inteligencia artificial. Esta disciplina basada en procesamiento de imágenes, técnicas de visión artificial y deep learning se han desarrollado ampliamente que a la actualidad permiten procesar distintos tipos de imágenes con altos grados de complejidad en su clasificación. Según Forbes, la visión por computadora es la automatización de la vista humana, que representa el sentido más importante en cualquier actividad económica (Forbes US, 2021).

Los inicios de la inteligencia artificial se llevó a cabo en la década de 1930, desde esos años se buscaba darle solución a determinadas problemáticas. Se puede ejemplificar este concepto con la creación de la "Máquina de Turín", que tenía como fin identificar los patrones de las claves de comunicaciones que tenías los alemanes en la segunda guerra mundial. Por esta invención Alan Turín fue considerado el padre de la inteligencia Artificial. (Isabel Vegara, 2021). En 1941 en desarrolló la primera computadora programable y automática, considerado la primera computadora del mundo moderno. Pero el término de inteligencia artificial se definió en el año 1956 por el informático Jhon McCarthy en una conferencia científica, que luego en el año de 1957 Frank Rosenblat diseña la primera red neuronal artificial. (National Geographic, 2020).

La inteligencia artificial tuvo un cambio de dirección debido a un mejor nivel de hardware y arquitecturas de computadores que permitió procesar grandes volúmenes de datos que en años atrás sería imposible poder realzarlos. Este avance permitió un mejor procesamiento de imágenes y entrenamiento de modelos de aprendizaje profundo con una gran cantidad de datos.

En la actualidad la inteligencia artificial con sus sub áreas de visión computacional y aprendizaje profundo están presente en distintos ámbitos, desde sectores financieros hasta

sectores agrícolas. Entre ellas el sector de salud es el más crítico y en el que mayor impacto ha tenido. En esta área la lucha con tras las epidemias siempre ha sido un problema importante para muchos países. El Perú es uno de los países como mayor presencia de epidemias como el Dengue, Zika o Chikungunya. Especialmente el Dengue, en los últimos años a la actualidad a causado la muerte en distintas regiones del país (CDC, 2022).

El contagio de esta enfermedad infecciosa de persona en persona a través de la picadura de un mosquito de tipo Aedes que se encuentra presente en el mismo ambiente que los mosquitos comunes denominado científicamente Culex que no trasmiten este tipo de enfermedades mortales. Debido a esto, la presente investigación busca aportan una herramienta en basa a inteligencia artificial que mediante la clasificación automática de imágenes de estos mosquitos Aedes se pueda conocer la presencia de estos vectores de trasmisión y tomar medidas preventivas.

El presento trabajo está organizado de la siguiente manera: En el capítulo uno que tiene por nombre Planteamiento del problema, se detalla la formulación del problema, los objetivos de la investigación, la justificación y delimitación del estudio. En el segundo capítulo que tiene por nombre Marco teórico, se detalla los antecedentes de la investigación, las bases teóricas, el marco conceptual y la hipótesis de la investigación. En el tercer capítulo que tiene por nombre Metodología se detalla, el diseño de la investigación, la metodología implementada y los indicadores de medición de los modelos de inteligencia artificial. En el capítulo cuatro que tiene por nombre “Desarrollo del experimento”, se detalla el procedimiento del experimento, desde la obtención del dataset hasta la obtención de resultados y su debido análisis. Por ultimo en capítulo cinco que tiene por nombre “Conclusiones y recomendaciones”, se detalla las conclusiones de esta investigación y sus debidas recomendaciones.

Capítulo I: Planteamiento del Problema

En este capítulo se explicará los detalles de la realidad problemática, el planteamiento del problema principal, problemas específicos, objetivo principal, objetivos específicos, la justificación y delimitaciones de la investigación.

1.1. Descripción de la realidad problemática

El sector de salud en el Perú está conformado por instituciones y establecimientos médicos de carácter público y privado, que se encarga de la atención de servicios de salud que cubre las necesidades de la población. En el sector público, entre los servicios de salud más críticos están la atención y control de brotes de epidemias. En este ámbito los establecimientos están conformados por centros pertenecientes a Essalud, MINSA, Gobiernos Regionales, Municipalidades y las Fuerzas Armadas (Ministerio de Salud, 2021, como se citó en la Cámara de Comercio de Lima, 2022). El número total de estos establecimientos es de 9439 y en el sector privado se contabiliza un total de 14458 (Abad, 2021).

En situaciones de brotes epidémicos, las acciones de control y prevención entre las instituciones de salud y la población, son solamente informativas y en muchos casos son solo reactivos. Es decir, realizan sus funciones de control luego de una alerta de brote, que son evidenciados por los centros médicos ubicados en distintas locaciones geográficas. Para todas estas acciones se utilizan determinados sistemas a nivel web y móvil que cumplen solamente funciones informativas (Valencia, 2019).

En el Perú, según el Centro Nacional de Epidemiología, Prevención y Control de Enfermedades (CDC, 2023) menciona que a diciembre del 2022 se notificó 73888 casos de arbovirosis, de los cuales 98.8% fueron de dengue, 1.17% por Chikungunya y el 0.02% por Zika. Este grupo de arbovirosis comparado con el año 2021 tuvo un incremento del 63%. En este año a febrero del 2023, para los casos de dengue se tiene alrededor de 20017 casos de los

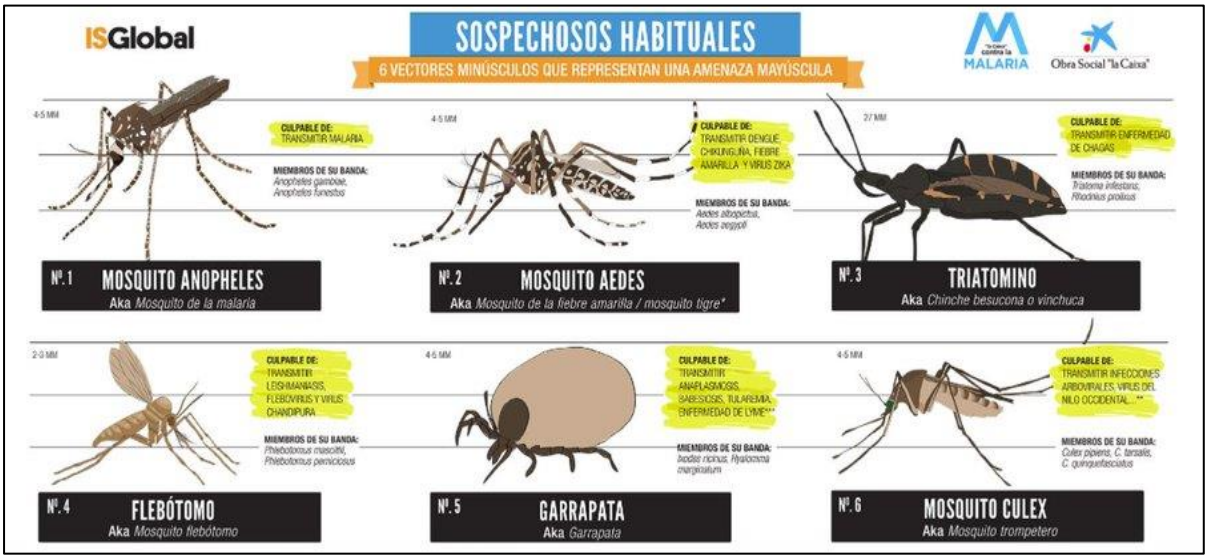
cuales se han registrado 25 casos mortales. Así mismo el 58.74% de los casos reportados tienen el rango de edad de 18-59 años. Es así que en estos últimos años los casos de arbovirosis en el Perú liderados por el dengue, tienen una comportamiento creciente y en específico la enfermedad del dengue en estos dos meses del 2023 ha aumentado 3 veces comparado con el año anterior en los mismos meses en cuestión. (CDC, 2023).

Los centros de atención de salud pública en el Perú, está estructurado por la magnitud y gravedad de las enfermedades que padece la población. Es decir se divide en tres niveles organizativos, el primer nivel está compuesto por postas médicas, centros de salud y policlínicos. Este tipo de centros es el primer contacto con la ciudadanía, donde más allá de la atención, tratamientos y diagnósticos médicos de cualquier índole, se realizan acciones de prevención y promoción para distintas epidemias. Así mismo se encuentran distintos especialistas de salud en estos tipos de centros. Las unidades de atención de segundo nivel están conformadas por hospitales y clínicas que reciben casos que son transferidos de los centros de primer nivel, en donde se encuentran profesionales de mayor especialidad. Por ultimo están los centros de tercer nivel que se diferencia de los anteriores por estar conformado por hospitales y clínicas con mayor número de equipamiento y profesionales especializados de distintas enfermedades. Esta estructura organizativa está enfocada en prevenir la saturación de estos centros de salud ante una situación de epidemia o pandemia.

En cada uno de estos centros de salud en sus distintos niveles, la unidad encargada de administrar los proceso de prevención y control de enfermedades endémicas a nivel nacional es del Centro Nacional de Epidemiología, prevención y control de enfermedades (CDC) que pertenece al Ministerio de Salud. Esta unidad tiene a cargo el sistema nacional de vigilancia epidemiológica de salud pública, quienes se encargan de realizar las campañas y acciones en contra de las enfermedades virales y enfermedades que son trasmitidas por organismos artrópodos, a este último grupo de enfermedades se les denomina arbovirosis.

El arbovirosis en el Perú está conformado principalmente por un grupo de virus, en los cuales el Dengue, Chikungunya y Zika son los que tienen mayores casos reportados. Estos virus son trasmitidos mediante un insecto de tipo culícido que pertenece a la familia Aedes. Estos organismos portadores de estos virus denominados vectores, es la raíz de todo contagio de arbovirosis en el Perú en los últimos años. (CDC, 2021).

Figura 1. Tipos de vectores



Fuente: Muños y Rodríguez. 2017. *Vectores minúsculos que suponen una amenaza mayúscula*. Centro de Reserva de Catalunya. p. 2

En situaciones de brotes epidemiológicos, las acciones realizadas por el CDC son enfocadas solo en campañas informativas y acciones post brote epidémico, es decir son reactivas y no proactivas. La necesidad de una herramienta tecnológica que se enfoque en acciones preventivas es fundamental en todo control eficiente de epidemias o pandemias.

La problemática de arbovirosis en el Perú radica en los insectos que propagan infecciones altamente letales para la salud humana. Estos insectos pertenecen al tipo culícidos de la familia Aedes que tienen la particularidad de trasmitir enfermedades con altos riesgos de mortalidad. Actualmente en el Perú, las acciones en el control y prevención en relación a estos tipos de insectos es insuficiente y se evidencia la falta de una herramienta tecnológica

que permite una clasificación rápida y autónoma de este tipo de insectos, que sean útiles en cualquier tipo de acción de control epidémico, debido a que la clasificación automática de estos insectos contribuiría a promover mejores acciones de control en las zonas endémicas de mayor riesgo de salud.

El nivel de soluciones tecnológicas en el Perú para este tipo de problemáticas es limitado y por los resultados negativos en el control de las enfermedades asociados a estos insectos se revela la falta de implementación de proyectos que trabajen con inteligencia artificial, que ofrezcan nuevas maneras de clasificar a estos insectos y que sirvan de herramienta en los procesos de prevención y control de enfermedades endémicas como el Dengue, Zika y Chikungunya.

Las iniciativas de tecnología basadas en inteligencia artificial permiten optimizar funcionalidades y crear modelos de clasificación autónomos a partir de distintos recursos como imágenes, que clasifiquen a los vectores de arbovirosis, que transportan las enfermedades del Dengue, Zika y Chikungunya.

Para el desarrollo de estos proyectos de inteligencia artificial es indispensable disponer de un óptimo conjunto de datos que permitan entrenar los diversos modelos de IA que se dispone actualmente. Este recurso en el Perú es ausente debido a que no se encuentra disponible de manera directa un adecuado dataset de imágenes de vectores de arbovirosis específicos de las especies que transportan las enfermedades del Dengue, Zika y Chikungunya, que puedan ser trabajados con modelos de aprendizaje profundo. De igual manera tampoco se dispone de una metodología a nivel de pre procesamiento donde se definan las técnicas adecuadas para tratar las imágenes a nivel morfológico de estos tipos de insectos.

Así mismo este tipo de imágenes luego de un pre procesamiento también necesitan de la aplicación de técnicas especiales de visión artificial, debido a que estas técnicas permiten

la extracción de patrones característicos que puedan diferenciar una imagen de un insecto de la familia Aedes y un insecto de la familia Culex. De igual manera para la clasificación de estos tipos de imágenes se necesitan la aplicación de modelos de deep learning que determinan si una imagen corresponde a una clase de insecto u otra. Estos modelos enfocados en esta problemática necesitan una evaluación que permitan comprobar su rendimiento y nivel de precisión.

En los últimos años la presencia de estos insectos de tipo Aedes ha aumentado en las diferentes regiones del país, el último departamento en registrar su presencia fue Tacna y para los próximos años se prevé que estos números aumenten y por consecuencia al aumentar la presencia este grupo de insecto, los casos de virus como el Dengue, Chikungunya y Zika también aumentaran. Así mismo esta información también puede comprobarse en las tendencias en el tiempo, de casos de arbovirosis en el Perú, en donde se revela que en los últimos años la distribución del número de casos confirmados de este grupo de enfermedades ha tenido un comportamiento con creciente y recurrente (CDC, 2021). Por esta razón, si las acciones de control y prevención se siguen ejecutando de la misma manera se tendrán lamentablemente en los próximos años un mayor número de casos mortales y la eventual saturación la saturación de atención de casos en los centros de salud de primera línea como las postas médicas y policlínicas en las regiones endémicas (Abad, 2020).

Las acciones que se deben realizar para controlar y prevenir de forma eficiente la arbovirosis en el Perú es controlando y cortando directamente el flujo de contagio de persona a persona. Para estos tipos de virus, el vector culpable del contagio es el grupo de insectos de tipo Aedes que transmiten esta enfermedad mediante su picadura y que por su condición de insecto suele confundirse con otros tipos de artrópodos similares. En esta situación, la gran mayoría de personas de distintas regiones del país no tienen el conocimiento para identificar a estos tipos de insectos. Es así que se necesita un modelo basado en inteligencia artificial que

ayude a clasificar de forma automática estos tipos de insectos y a partir de esto, informar a las personas para que puedan tomar las acciones correspondientes de prevención oportunamente. Así mismo, este modelo serviría como herramienta para generar un nexo entre la población y las instituciones de salud con respecto al control y prevención proactiva de epidemias o pandemias debido a la información que proporcionaría a la población acerca de la presencia de estos tipos de insectos. Contribuyendo a disminuir el número de casos reportados el Dengue, Chikungunya y Zika. De igual modo, los resultados de clasificación del modelo aportaría a evitar la saturación de centros médicos de primera línea debido a que se realizaría con anticipación y de manera oportuna las acciones de control epidémico a partir de la información proporcionada del modelo (CDC, 2021).

1.2. Formulación del problema

1.2.1. Problema General

¿Es posible implementar modelos de inteligencia artificial para la clasificación de vectores de arbovirosis del Dengue, Zika y Chikungunya?

1.2.2. Problemas Específicos

PE1: ¿Actualmente se encuentra disponible un dataset público de imágenes de vectores de arbovirosis detallados por sub especies de Aedes y Culex?

PE2: ¿Qué tipo de técnicas de pre procesamiento se pueden implementar en las imágenes de vectores de arbovirosis para mejorar la calidad de los datos?

PE3: ¿Qué tipo de modelos de Deep Learning se pueden implementar para la clasificación de imágenes de vectores de arbovirosis de tipo culícido de la familia Aedes y Culex?

PE4: ¿Qué métricas evalúan el desempeño de los modelos deep learning para clasificar las imágenes de los vectores de arbovirosis de tipo culícido de la familia Aedes y Culex?

1.3. Objetivos de la Investigación

1.3.1. Objetivos General

Desarrollar un modelo basado en inteligencia artificial para la clasificación de vectores de arbovirosis del Dengue, Zika y Chikungunya.

1.3.2. Objetivos Específicos

OE1: Crear un dataset de imágenes de vectores de arbovirosis de tipo culícido de la familia Aedes y Culex detallados por sub especies a partir de repositorios oficiales de entomología.

OE2: Aplicar técnicas de pre procesamiento en imágenes de vectores de arbovirosis de tipo culícido de la familia Aedes y Culex.

OE3: Implementar los modelos Deep Learning VGG16, VGG19, ResNet50 y Vision Transformers para la clasificación de imágenes de vectores de arbovirosis de tipo culícido de la familia Aedes y Culex.

OE4: Utilizar métricas para evaluar el desempeño de los modelos de deep learning para clasificar las imágenes de vectores de arbovirosis de tipo culícido de la familia Aedes y Culex.

1.4. Justificación de la Investigación

1.4.1. Teórico

Esta investigación tiene como propósito aportar una herramienta tecnológica basada en visión artificial y aprendizaje profundo que permitan prevenir y controlar los casos de arbovirosis en el Perú. Su significancia radica en desarrollar un modelo que clasifique de manera automática al grupo de insectos de tipo Aedes, que son los principales trasmisores del Dengue, Zika y Chikungunya en el Perú. Los casos positivos reportados de estas enfermedades se han incrementado en un 63% del 2021 al 2022 y en este año 2023 a la semana 9 se han confirmado 20146 casos de contagio a nivel nacional y se prevé que

superará al año anterior. De estas enfermedades la más peligrosa y con mayor presencia es el Dengue que ha provocado la muerte de 86 personas en el 2022 y 25 personas fallecidas de enero a abril en este año 2023. La gravedad de esta situación aumentaría debido a las inundaciones, huaycos y cambios climáticos que se han generado en este año, que por defecto provocaría un mayor número casos reportados de Dengue debido al aumento de la población de estos tipos de insectos Aedes (CDC, 2023).

Para enfrentar esta situación a corto y a largo plazo es necesario realizar acciones preventivas de control de estas epidemias que busquen la eficiencia mediante nuevas tecnologías, que vayan más allá de los procedimientos tradicionales actuales que son realizados de manera manual y que puedan estar enfocados en la unidad inicial de contagio que es el insecto Aedes. Donde cabe resaltar que una eficiente vigilancia entomológica de estos insectos afecta positivamente el control de los brotes epidémicos (Paho, 2021).

Lamentablemente en el Perú no se dispone de una herramienta tecnológica que contribuya a mejorar las actividades de prevención y control de epidemias que puedan estar enfocados en la clasificación e identificación temprana de los insectos Aedes dentro del ámbito de la vigilancia entomológica y que a partir de esta identificación se puedan implementar mejores acciones de control con mayor anticipación (CDC, 2023).

Las nuevas tecnologías basadas en aprendizaje profundo y técnicas de visión artificial en el campo de la entomología, han brindado grandes soluciones a nivel de procesamiento de imágenes de distintos tipos de insectos que permiten realizar con eficiencia la clasificación , el análisis y las observaciones entomológicas de las poblaciones de insectos (Hoye et al., 2021).

En el Perú las investigaciones de inteligencia artificial en la entomología son muy pocas y en lo que respecta al procesamiento de imágenes de insectos que trasmiten

enfermedades hacia los ser humanos no se han desarrollado ninguna investigación a la actualidad.

Por lo tanto es de prioridad para esta presente investigación brindar un aporte que contribuya al control preventivo de brotes de epidemias como el Dengue, Zika y Chikungunya mediante la clasificación automática del tipo de insecto que transporta estas enfermedades. Donde los resultados obtenidos podrán ser generalizados posteriormente en investigaciones científicas futuras.

1.4.2. Práctico

Al término de esta investigación se obtendrá un modelo basado en inteligencia artificial que clasificará por imágenes al grupo de insectos de tipo Aedes. Esta herramienta será de utilidad para la población y las instituciones de salud encargadas del control y prevención de epidemias. Es decir, para la población, esta herramienta mediante la clasificación de imágenes, brindará información de la presencia de estos tipos de insectos y con estos datos se podrá tomar acciones preventivas, evitar picaduras y la propagación de enfermedades. Así mismo para las instituciones de salud esta herramienta servirá de fuente de información en los controles de vigilancia entomológica que se realizan en distintas zonas endémicas para tomar acciones de prevención y de esta manera reducir el contagio de enfermedades como el Dengue, Zika y Chikungunya.

1.4.3. Metodológico

El desarrollo de esta tecnología mejorará las acciones de control y prevención de casos de arbovirosis en el Perú mediante la clasificación de insectos de tipo Aedes. Para esta investigación se realizará la implementación de modelos de deep learning utilizando internamente técnicas de visión artificial. Donde se utilizaran indicadores de evaluación que medirán la precisión de estos modelos. En la cual cada modelo implementado tendrá como objetivo la clasificación óptima de las imágenes de este tipo de insectos.

La implementación de este modelo estará compuesto por tres etapas principales. La primera etapa estará conformado por actividades de recolección de datos, en la que se obtendrán imágenes de los insectos de tipo culícidos de la familia Aedes y Culex de 5 fuentes distintas pertenecientes a varios centros de investigación entomológica, donde en cada una de las fuentes se encuentran distintas clases de insectos a nivel general y que para propósito de esta investigación solo se utilizaron de ese conjunto de imágenes aquellas que tienen las etiquetas de la familia Aedes y Culex.

De cada fuente, las imágenes fueron analizadas y etiquetadas por especialistas expertos en entomología. De esta manera se obtiene un conjunto de imágenes de dos tipos de clases, la primera clase pertenece a los insectos de tipo culícidos de la familia Aedes que corresponden a las sub especies de Aedes Aegypti, Aedes Albopictus. La segunda clase corresponde a los insectos de tipo culícidos de la familia Culex que corresponden a las sub especies Culex Pipiens, Culex Quinquefasciatus, Culex Tritaeniorhynchus y el Culex Anatomii.

En la segunda etapa se realizará las técnicas de pre procesamiento al conjunto de imágenes Aedes y Culex, para el adecuado tratamiento de las imágenes. Donde se utilizarán las técnicas de segmentación de fondo para las imágenes que requieres esta implementación, la aplicaciones normalización de datos y para todas las imágenes se realizaran un balanceo de datos para mantener el mismo número de etiquetas en ambas clases.

Por ultimo en la tercera etapa se realizará la implementación de los modelos de deep learning VGG16, VGG19 y ResNet50, donde posterior al entrenamiento del mejor modelo seleccionado se analizará los vectores característicos de visión artificial mediante la técnica de visualización de mapas de regiones discriminatorios para ser comparados con una análisis real que realizan los especialistas entomólogos en la práctica de las dos clases de insectos.

1.5. Delimitación del Estudio

En este parte se explicará las delimitaciones de este proyecto de investigación. Del cual será a nivel espacial, temporal y conceptual.

1.5.1. Espacial

Este proyecto de investigación se desarrollará a partir del conjunto de imágenes que serán recolectadas de 5 fuentes distintas pertenecientes a varios centros de investigación entomológica. En cada fuente, las imágenes fueron analizadas y etiquetadas por especialistas expertos en entomología. Este dataset agrupado está estructurado en dos clases, la primera clase pertenece a los insectos de tipo culícido de la familia Aedes (Aedes Aegypti, Aedes Albopictus) y la segunda clase pertenece a los insectos de tipo culícidos de la familia Culex (Culex Pipiens, Culex Quinquefasciatus, Culex Tritaeniorhynchus y el Culex Anatomii.). Todas las imágenes están en formato ".jpg" y muestran a los insectos de cuerpo entero en distintas posiciones y enfoques.

1.5.2. Temporal

Este tipo de proyectos de tecnología de inteligencia artificial requieren una amplia base de datos para una mejor tasa de aprendizaje. Es así que se recolectaran imágenes de insectos de tipo culícidos de la familia Aedes y Culex detallados por especies de 5 fuentes distintas y se formara un dataset estructurado que tiene como rango de fechas desde el año 2014 hasta la actualidad.

1.5.3. Conceptual

Este proyecto de investigación busca desarrollar un modelo que realice la clasificación por imágenes de insectos de tipo culícidos de la familia Aedes y Culex. Con el objetivo de reconocer y evidenciar la presencia del insecto Aedes que es el responsable del

contagio de enfermedades del Dengue, Zika y Chikungunya. Para ello se utilizará un dataset agrupado de 5 fuentes pertenecientes a varios centros de investigación entomológica. En este dataset se aplicaran técnicas de pre procesamiento y posteriormente estos datos serán entrenados en los modelos de deep learning VGG16, VGG19 y ResNet50 con el objetivo de clasificar los dos tipos de insectos (Aedes y Culex). La solución que plantea este proyecto de investigación se aplicará en el campo de la salud pública y control de epidemias aportando de esta manera una herramienta preventiva en el control de brotes endémicos.

Capítulo II: Marco Teórico

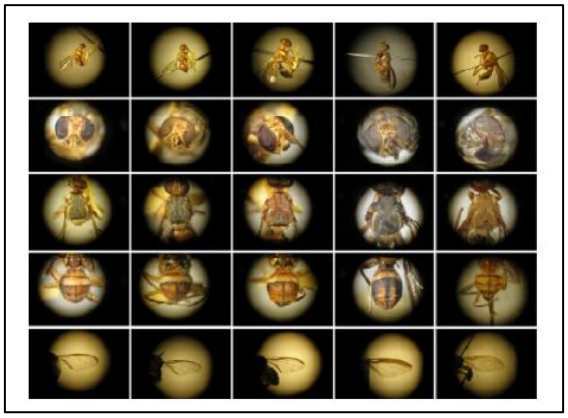
En este capítulo se detallan los antecedentes de la presente investigación y su correspondiente base teórica, del cual se fundamenta este trabajo. Así mismo también se analizará el marco conceptual, la hipótesis, las variables e indicadores.

2.1. Antecedentes de la Investigación

En esta parte se presentará un grupo de antecedentes que se analizaron con el objetivo de comprender como se ha tratado a nivel de objetivos, procedimientos y resultados este tema de investigación en otros casos y en distintos países. Es importante mencionar que a la fecha actual no se tiene conocimiento que exista una investigación de estas características en el Perú.

En el artículo de investigación desarrollado en Malasia y publicado en el año 2014 que tiene por título Vision Based Entomology: A Survey y que tiene por autores a Asian Hassan, Nadiah Syakira, Zaw Htike y Shoon Lei Win. Explican que la alta presencia de insectos de distintos tipos que se ubican en varias partes del mundo provoca múltiples enfermedades que perjudican la salud humana y su entorno. Así mismo este perjuicio también alcanza a las zonas productivas de sembríos donde es importante identificar las diversas especies en las áreas de cultivos. Este artículo desarrolló un modelo basado en KNN con componentes PCA y LDA para la clasificación de imágenes de insectos que transportan enfermedades en sembríos.

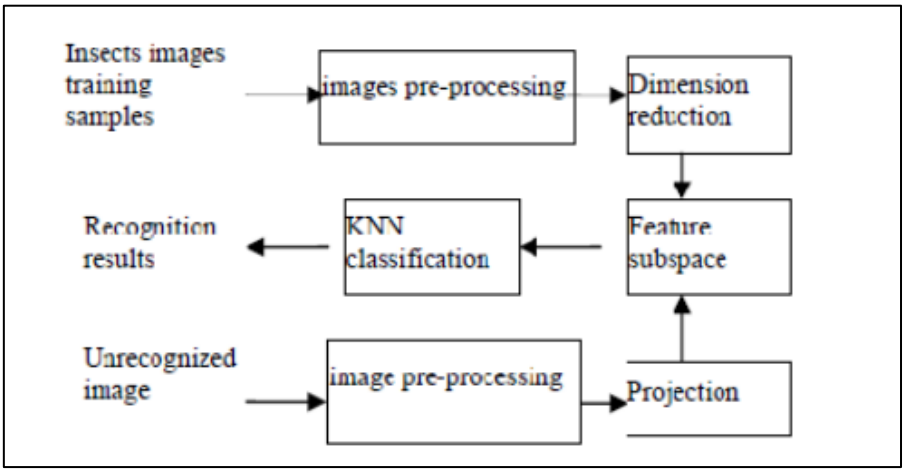
Figura 2. Dataset utilizado en el artículo de investigación



Fuente: Hassan et al., 2014. *Vision Based Entomology: A Survey*. p. 6

La metodología implementada por este artículo inicia con una primera etapa donde se realiza la obtención de los datos que son las imágenes de la mosca de la fruta denominada Tephritidae y sus determinadas subespecies. Luego se realizó la aplicación de técnicas de pre procesamiento donde se aplicaron determinados filtros y redimensionamientos a las imágenes.

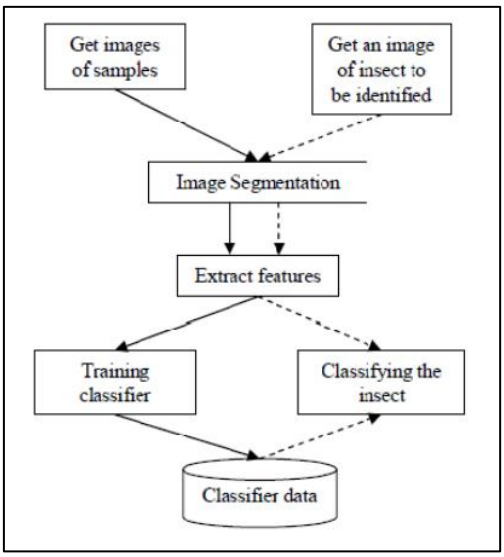
Figura 3. Metodología del pre procesamiento de los datos



Fuente: Hassan et al., 2014. *Vision Based Entomology: A Survey*. p. 10

En esta misma etapa también se realizó la extracción de características mediante la transformación de características de escala invariante (SIFT). Los vectores SIFT se generan a partir de las diferentes partes morfológicas del cuerpo del insecto, mismas que será la entrada al modelo KNN.

Figura 4. Metodología del artículo de investigación



Fuente: Hassan et al., 2014. *Vision Based Entomology: A Survey*. p. 6

En la segunda etapa se realiza la parte experimental del modelo KNN utilizando los componentes de entrada PCA y LDA. La función LDA busca ampliar la distribución de las muestras, es decir la dispersión interna se mantiene alejada entre sí. Así mismo el análisis de componentes principales (PCA) es utilizado también en la metodología de este artículo para la comparación de resultados de un KNN con PCA o un KNN con un LDA. Es así que en la última etapa se presentan los resultados del modelo donde se obtuvo un Accuracy del 90% que pertenecen a un KNN con un LDA.

Como conclusión en este artículo se recomienda seguir mejorando este campo de investigación, implementando a futuro nuevas técnicas de aprendizaje automático para

mejorar el rendimiento de los modelos en la clasificación de este tipo de imágenes de insectos que trasmiten enfermedades.

En el artículo de investigación desarrollado en Brasil y publicado en el año 2015 que tiene por título *Artificial Neural Network applied as a methodology of mosquito species identification* y que tiene por autores a Camila Lorenz, Sergio Ferraudo y Lincoln Suesdek. Explican que en el mundo existen cerca de 200 especies de mosquitos de tipo culícido que son causantes de la trasmisión de enfermedades hacia los seres humanos. Es así que identificar las especies de mosquitos que son altamente peligrosos es esencial para realizar efectivos controles de las enfermedades. Para este tipo de identificación es necesario el análisis de expertos entomólogos que realizan tareas laboriosas para el cumplimiento de la clasificación manual de estos insectos. Es así que el artículo busca clasificar mediante imágenes y la forma de las alas a 17 especies del género Anopheles, Aedes y Culex. Para este proceso utilizaron una red neuronal artificial Perceptrón Multicapa.

La metodología implementada estuvo dividida en tres etapas. El primer paso se realizó la adquisición de los datos que consta de 388 especímenes hembra, donde contienen las 17 especies. De estas especies se fotografiaron las alas mediante una cámara digital.

Tabla 1

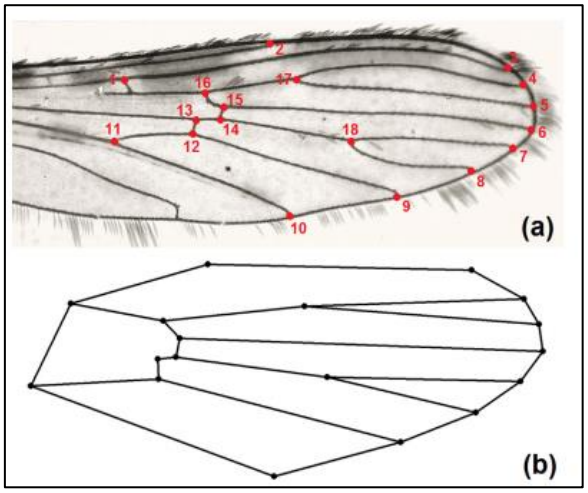
Dataset utilizado en el artículo de investigación

Familia	Especies	N	Sitio de muestreo
Aedes	Ae. Aegypti	35	Sau Paulo
	Ae. Albopictus	35	Sau Paulo
Anopheles	An. Homunculus	24	Pariquera-Acu
	An. Trianmilatus s.l.	23	Minas Gerais
	An. Albitarsis s.l.	18	Minas Gerais
	An. Aquasalis	36	Sau Paulo
	An. Bellator	36	Pariquera
	An. Cruzii	24	Pariquera
	An. darlingi	28	Manaus
Culex	Cx. Sacchetae	9	Pariquera
	Cx. Usquatus	17	Pariquera
	Cx. Atratus	10	Rio de Janeiro
	Cx. Coronator	16	Manaus
	Cx. Ribeirensis	15	Rio de Janeiro
	Cx. Cornirger	27	Sau Paulo
	Cx. Nigripalpus	19	Manaus
	Cx. Quinquefasciatus	16	Sau Paulo

Fuente: Lorenz et al., 2015. *Artificial Neural Network applied as a methodology of mosquito species identification*. p. 3

En esta misma etapa se realizó posteriormente la extracción de características de las diferentes partes de las especies de donde se obtuvo 32 componentes principales que fueron las entradas al modelo Perceptrón Multicapa. Donde se utilizaron 300 imágenes para el entrenamiento y 88 imágenes para el test.

Figura 5. Región de interés de las alas de los insectos para la extracción de características



Fuente: Lorenz et al., 2015. *Artificial Neural Network applied as a methodology of mosquito species identification*. p. 3

En la segunda etapa se realizó la etapa experimental de la implementación de la red neurona, donde en la primera fase se configuró 100 épocas bajo el modelo de *back propagation.* En la segunda fase se configuró 500 épocas bajo el modelo de gradiente descendiente. El modelo a modo experimental utilizó una taza de aprendizaje de un rango de 0.1 a 0.5 y la función logística fue la función de activación utilizada.

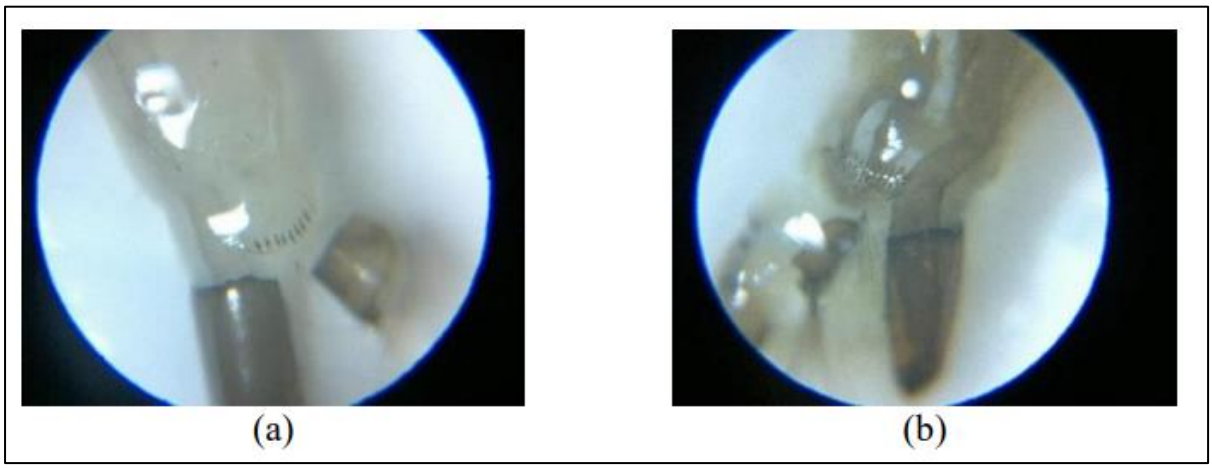
En la tercera etapa se presentaron los resultados del modelo perceptrón multicapa donde obtuvo un 85.7%. Así mismo se concluyó que es necesario realizar mejores implementaciones de modelos para alcanzar una mejor precisión en la clasificación de las imágenes de estos tipos de insectos.

En el artículo publicado en el año 2019 que tiene por título *Mosquito Larvae Image Classification based on DenseNet and Guided Grad-CAM* y que tiene por autores a Zaira García, Keiji Yanai, Mariko Nokano, Antonio Arista, Laura Cleofas y Hector Perez. En la cual explican que las enfermedades como el Dengue, Zika y Chikungunya tienen un mayor número de propagación y contagios en los últimos años, donde la vigilancia de los mosquitos

Aedes Aegypti y Aedes Albopictus en lo que respecta a métodos de identificación de sus larvas requiere altos costos de trabajo, equipos especiales y entomólogos especializados. Por este motivo en este artículo se propone un enfoque de clasificación de imágenes de larvas de Aedes Aegypti y Aedes Albopictus basado en un modelo de redes neuronales profundas DenseNet 121.

La metodología empleada en este artículo de investigación tiene como primera etapa la construcción de la base de datos y su pre procesamiento, en la cual se utilizó 760 imágenes de larvas de las especies de Aedes Aegypti y Albopictus. De conjunto de imágenes se dispuso 600 imágenes para el entrenamiento, 60 imágenes para el test y 100 imágenes para la validación. En esta misma etapa se realizó el proceso de *data augmentation* en los datos de entrenamiento que tuvo transformaciones de rotación, cambio de en las dimensiones y zoom en cada imagen.

Figura 6. Imágenes de microscopio de larvas de Aedes Aegypti y Aedes Albopictus



Fuente: García et al., 2019. *Mosquito Larvae Image Classification Based on DenseNet and Guided Grad-CAM*. p. 4

En la segunda etapa se realizó el desarrollo del modelo DenseNet 121 donde se utilizaron determinados hiperparámetros de pruebas para obtener los mejores resultados con la técnica de *Grid Search*.

Tabla 2

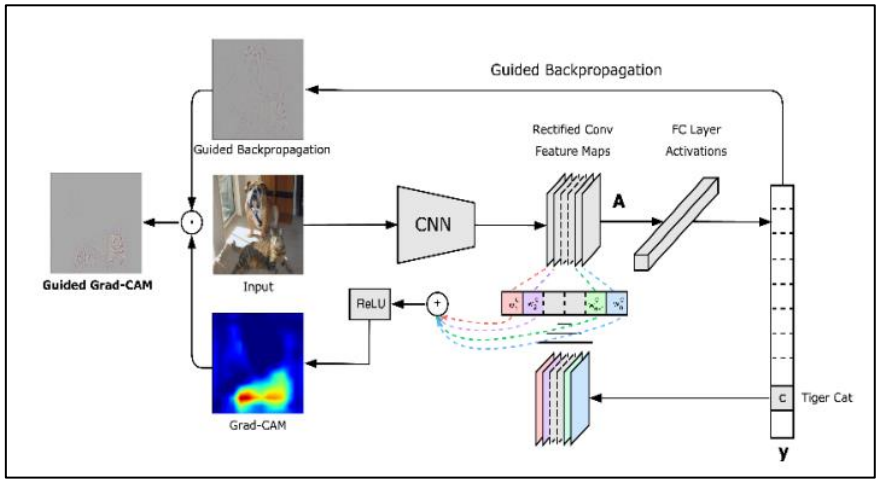
Hiperparámetros del modelo DenseNet 121

Hyperparámetros	Valores
Épocas	150
Batch Size	10
Taza de Aprendizaje	0.01
Momentum	0.2
Decay	$10e^{-5}$

Fuente: García et al., 2019. *Mosquito Larvae Image Classification Based on DenseNet and Guided Grad-CAM*. p. 5

Así mismo para conocer las partes principales de los mapas de características que emplea el modelo en cada imagen se utilizó Grad-CAM, que es una técnica que permite visualizar mediante mapas de calor las partes de las imágenes que está identificando la red como puntos característicos para la clasificación.

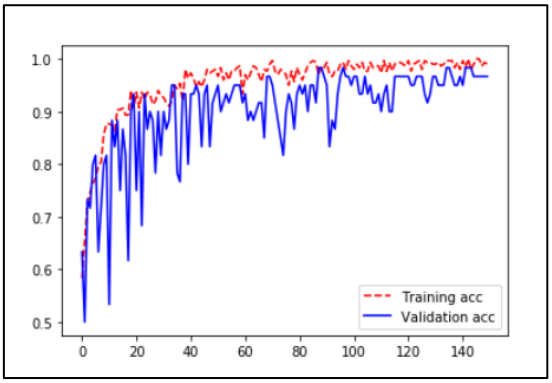
Figura 7. Metodología de la técnica Grad-CAM



Fuente: García et al., 2019. *Mosquito Larvae Image Classification Based on DenseNet and Guided Grad-CAM*. p. 4

Por último en la tercera etapa se presentaron los resultados del modelo implementado donde se obtuvo un Accuracy del 97%, una sensibilidad del 97% y una especificidad del 96% al término de las 150 épocas.

Figura 8. Gráfico del entrenamiento y validación del modelo DenseNet 121



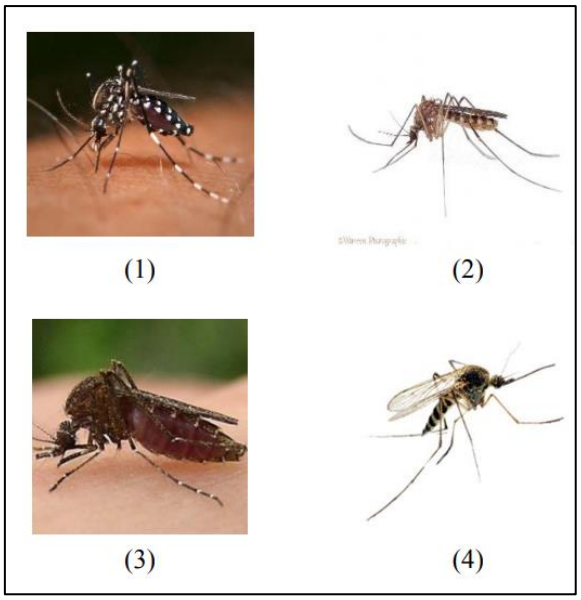
Fuente: García et al., 2019. *Mosquito Larvae Image Classification Based on DenseNet and Guided Grad-CAM*. p. 5

Este artículo concluye que el modelo DenseNet 191 tiene un gran rendimiento en la clasificación de larvas de mosquitos así como también la técnica Grad-CAM permite visualizar lo que internamente analiza la red a nivel de característica de la imagen y que determine la clasificación del mismo.

En el artículo de investigación publicado en el año 2020 que tiene por título *CNN Architectures Performance Evaluation for Image Classification of Mosquito in Indonesia* y que tiene por autores a Brilian Amiruddin y Adbul Rusdhianto. Se explica que las trasmisiones de enfermedades producidas por mosquitos son altamente peligrosas en diferentes partes del mundo y la propagación de enfermedades a causa de estos insectos tiene una tendencia en el tiempo cada vez más alta. En indonesia existe alrededor de 125 especies y 18 géneros de mosquitos, en la cual algunos de ellos trasmiten enfermedades peligrosas para el ser humano Esta investigación busca remplazar las funcionalidades realizadas por un especialista y un microscopio mediante el uso de las redes neuronales profundas, en la cual

consiste en reconocer y clasificar las imágenes de mosquitos por su tipo, donde se utilizó como modelo una red neuronal convolucional (CNN) y un conjunto de imágenes obtenidas de internet.

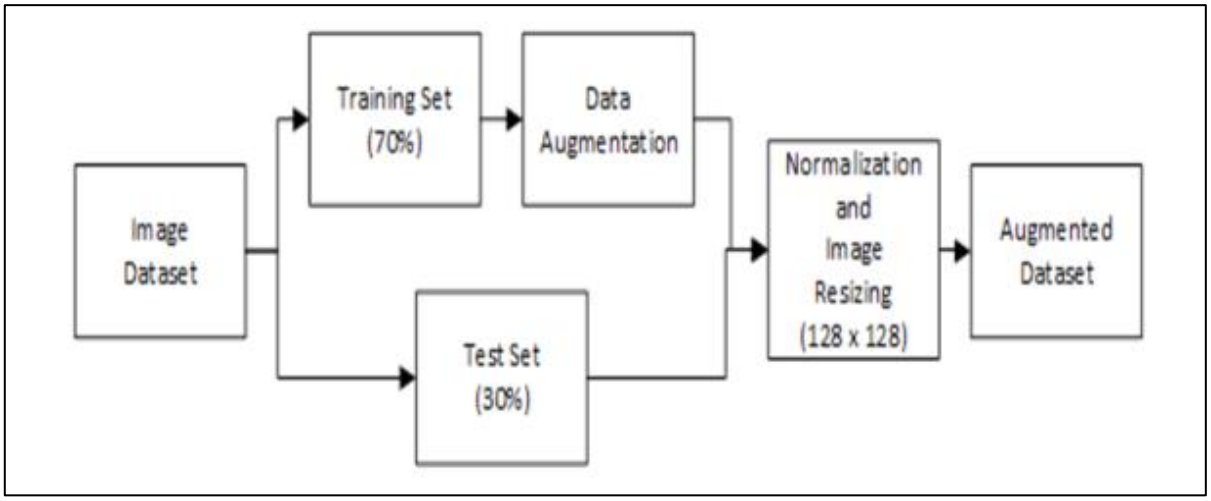
Figura 9. Imágenes de los insectos de tipo Aedes y otras especies



Fuente: Brilian y Rusdhianto, 2020. *CNN Architectures Performance Evaluation for Image Classification of Mosquito in Indonesia*. p. 2

La metodología aplicada en esta investigación comienza con una primera etapa, donde se realiza la adquisición de la data, en la cual se obtuvo 200 imágenes de mosquitos directamente de la web y se dividió en 50 imágenes por cada especie en general (Aedes, Anopheles, Culex y Mansonia). Debido a la poca cantidad de imágenes disponibles se realizó la técnica de *Data Augmentation* para aumentar el número de imágenes por especie, esta técnica solo se aplicó a los datos de entrenamiento que resultó en un total de 560 imágenes destinadas solo para el entrenamiento del modelo. Después de este procesos se aplicaron técnicas de normalización y redimensionamiento de las imágenes a 128 x 128 del entrenamiento y prueba. En el cual se trajo con 70% para la data de entrenamiento y 30% para la test.

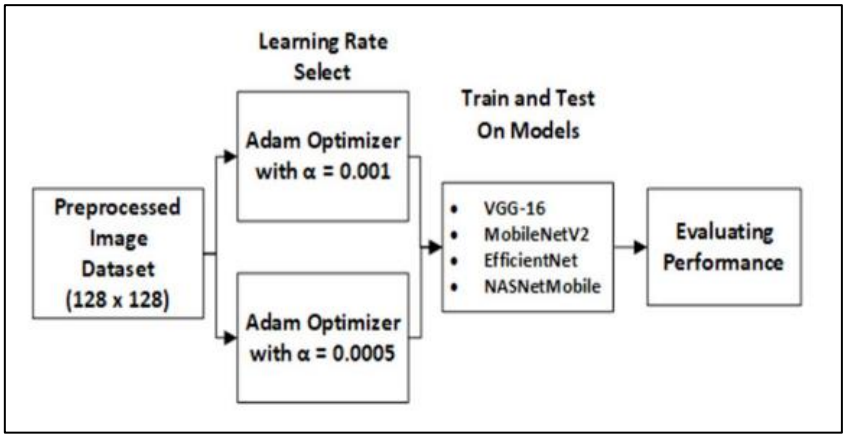
Figura 10. Pasos de realizados en el pre procesamiento del Dataset



Fuente: Brilian y Rusdhianto, 2020. *CNN Architectures Performance Evaluation for Image Classification of Mosquito in Indonesia*. p. 1

En la segunda etapa se desarrolló el proceso experimental donde se subdividió implementaron los modelos VGG16, MobileNetV2, EfficientNet-B1 y NASNetMobile que se entrenaron najo la técnica de aprendizaje por transferencia donde utilizaron el conjunto de datos de ImageNet. Para este estudio se modificó la capa de salida de 1000 nodos a 512 nodos y con una función de activación ReLU, un Dropout de 0.5 para evitar el problema de sobreajuste. También en la capa de salida se agregaron 4 nodos que son las cuatro clases de mosquito que se están clasificando, donde estas capas son activadas por una función Softmax. En cada modelo implementado se utilizó 100 épocas. Se realizó un primer escenario utilizando el optimizador Adam con una taza de aprendizaje correspondiente al 0.0005 y otro escenario con una taza de aprendizaje de 0.001.

Figura 11. Metodología del artículo de investigación



Fuente: Brilian y Rusdhianto, 2020. *CNN Architectures Performance Evaluation for Image Classification of Mosquito in Indonesia*. p. 2

En la última etapa se presentaron los resultados, en la cual el modelo MobileNetV2 obtuvo el mejor Accuracy en los dos escenarios de taza de aprendizaje con respecto a los dos modelos.

Tabla 3.

Resultados de Accuracy de los modelos de deep learning implementados

		Tiempo y Accuracy			
Modelos CNN	**α**	**Train Acc.**	**Test Acc.**	**Best Epoch**	**Time**
VGG-16	0.0005	76%	78%	77	483 s
	0.001	82%	78%	74	473 s
MobileNetV2	0.0005	90%	80%	90	462 s
	0.001	91%	78%	60	440 s
EfficientNet-B1	0.0005	82%	75%	70	437 s
	0.001	81%	78%	79	428 s
NASNetMobile	0.0005	86%	73%	77	488 s
	0.001	78%	73%	69	489 s

Fuente: Brilian y Rusdhianto, 2020. *CNN Architectures Performance Evaluation for Image Classification of Mosquito in Indonesia*. p. 3

Así mismo también se analizaron los resultados de cada modelo utilizando las métricas de Precisión, Recall, F1-Score y el AUC. El que obtuvo mejores resultados en promedio en todas las métricas y con una taza de aprendizaje del 0.0005 fue MovilNetV2 con una precisión de 0.81, un Recall de 0.8, F1-score de 0.8 y un valor de AUC de 0.91.

Tabla 4.

Resultados de otros indicadores de modelos de deep learning implementados

	Tiempo y Accuracy				
Modelos CNN	**α**	**Precisión**	**Recall**	**F1-Score**	**AUC**
VGG-16	0.0005	0.79	0.78	0.78	0.88
	0.001	0.79	0.78	0.78	0.88
MobileNetV2	0.0005	0.81	0.80	0.80	0.91
	0.001	0.79	0.78	0.78	0.93
EfficientNet-B1	0.0005	0.77	0.75	0.75	0.89
	0.001	0.79	0.78	0.78	0.91
NASNetMobile	0.0005	0.75	0.73	0.73	0.86
	0.001	0.73	0.73	0.73	0.89

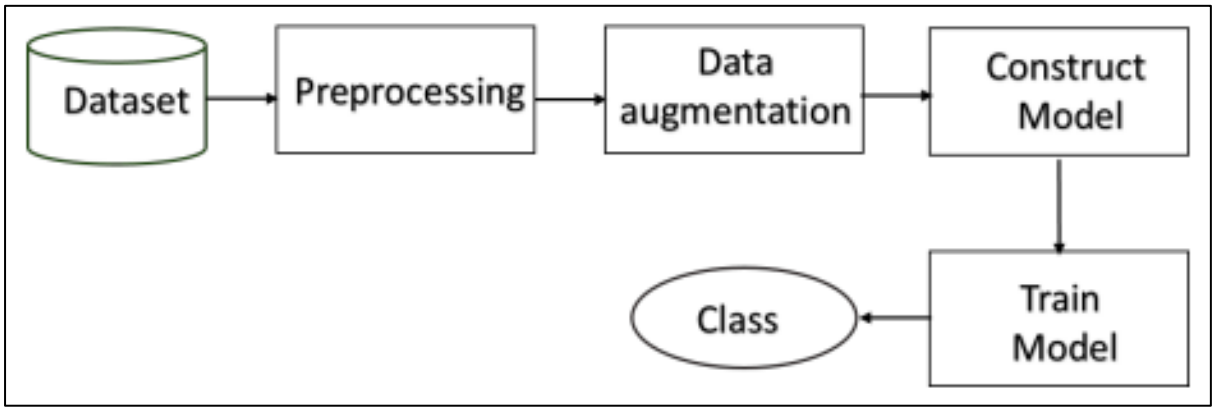
Fuente: Brilian y Rusdhianto, 2020. *CNN Architectures Performance Evaluation for Image Classification of Mosquito in Indonesia*. p. 3

Con estos resultados se concluye que el mejor modelo en este proceso de clasificación fue el MobileNetV2 que logró un 80% de Accuracy. Además los autores de esta investigación sugieren seguir con una mayor aplicación de modelos para obtener mejores resultados de clasificación.

En el artículo desarrollado en Bangladesh y publicado en el 2020, que tiene por título Mosquito *Classification using Convolutional Neural Network with Data Augmentation* y que tiene por autores Mehenika Akter, Mohammad Shahadat, Tawsin Ahmed y Karl Andersson. Esta investigación se enfoca en la problemática de las infecciones provocadas por la

transmisión de enfermedades epidemiológicas. Este tipo de afecciones afecta críticamente al país de Bangladesh, que es el lugar donde se desarrolla este artículo, en la cual se plantean clasificar los mosquitos que trasmiten enfermedades mediante imágenes correspondientes a mosquitos altamente contagiosos de la familia Aedes, Anopheles y Culex , de esta manera prevenir las trasmisiones y evitar la exposición del ser humano a estos moquitos. El conjunto de datos utilizado proviene de internet de búsquedas generales sin ninguna sub clasificación específica y sin verificación comprobada de las especies. Este artículo implementó un modelo de red neuronal convolucional y se compararon con otros modelos como el VGG16, Random Forest, XGboost y SVM.

Figura 12. Metodología del artículo de investigación

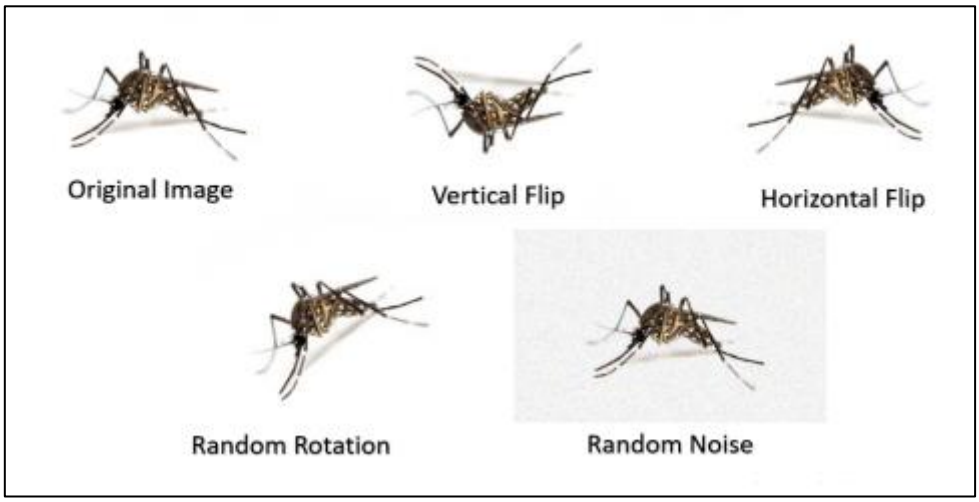


Fuente: Mehenika et al., 2020. *Mosquito Classification using Convolutional Neural Network with Data Augmentation*. p. 4

La metodología implementada tiene tres etapas, la primera está conformado por la recolección de la data y la aplicación de técnicas de Data Augmentation. Para la adquisición de la base de datos se obtuvieron un conjunto de imágenes recolectadas de internet de las especies en general de Aedes, Anopheles y Culex, sin ninguna sub clasificación en específica de cada grupo de mosquitos ni revisión de cada clase por expertos. Para el grupo Aedes se recolectaron en total 188 imágenes, 130 imágenes para Anopheles y 124 imágenes para el

tipo Culex. A esta base de datos se aplicaron la técnica de Data Augmentation (solo al conjunto de entrenamiento) con el objetivo de ampliar un mayor número de datos y mejorar el entrenamiento en los modelos. Luego de aplicación de aumento de datos se tiene 1200 imágenes por cada grupo.

Figura 13. Data Augmentation del dataset de las especies Aedes, Anopheles y Culex utilizadas por el artículo de investigación



Fuente: Mehenika et al., 2020. *Mosquito Classification using Convolutional Neural Network with Data Augmentation*. p. 8

En la segunda etapa se realizaron la técnica del pre procesamiento de las imágenes. En esta etapa se utilizó la librería de OpenCV para el procesamiento de las imágenes. Donde se aplicó las técnicas del ROI a cada imagen y posterior se eliminó el fondo de cada imagen. Luego, a estas imágenes de entrenamiento le aplicaron un filtro de escala gris y posteriormente pasó por un proceso de normalización y extracción de características.

En la tercera etapa se realizó el proceso experimental donde se implementaron los modelos VGG16, Random Forest, XGboost y SVM. Para la construcción de nuestro modelo en cada capa se utilizó la función ReLU para la activación, un valor de dropout de 0.5 y un tamaño de Kernel de 2. El modelo incluye 4 capas convolusionales. Donde la primera capa

convolucional tiene 16 filtros de tamaño 2x2, la segunda capa convolucional tiene 32 filtros de tamaño 2x2, la tercera capa tiene 64 filtros y la cuarta capa tiene 128 capas. Así mismo después de cada capa oculta hay una capa de abandono que desactiva el 50% de los nodos de la capa oculta para evitar el sobre ajuste. Además en la capa de salida se utilizó la función de activación Softmax y como optimizador del modelo se utilizó Adam y la función de perdida es de tipo *Categorical Crossentropy*.

Tabla 5.

Hiperparámetros del modelo implementado.

Contenido del modelo	**Detalles**
1ra Capa Convolucional 2D	16 filtros de tamaño 2X2 ReLU
1ra Capa Max Pooling	Pooling de tamaño 2X2
Capa Dropout	50% de neuronas excluidas en aleatorio
2da Capa Convolucional 2D	32 filtros de tamaño 2X2 ReLU
2da Capa Max Pooling	Pooling de tamaño 2X2
Capa Dropout	50% de neuronas excluidas en aleatorio
3ra Capa Convolucional 2D	64 filtros de tamaño 2X2 ReLU
3da Capa Max Pooling	Pooling de tamaño 2X2
Capa Dropout	50% de neuronas excluidas en aleatorio
4ta Capa Convolucional 2D	128 filtros de tamaño 2X2 ReLU
4ta Capa Max Pooling	Pooling de tamaño 2X2
Capa Dropout	50% de neuronas excluidas en aleatorio
Promedio Global de la capa Pooling	N/A
Capa de salida	Tres salidas para las tres clases, Softmax
Función de Optimización	Adam
CallBack	ModelCheckPoint

Fuente: Mehenika et al., 2020. *Mosquito Classification using Convolutional Neural Network with Data Augmentation*. p. 5

En los resultados de los modelos implementados el modelo propuesto en este artículo tuvo un mejor Accuracy del 93% seguido del modelo VGG16 con 91%.

Tabla 6.

Resultados de los modelos implementados

Modelo	Accuracy	Precisión	Recall	F1-Score
CNN propuesto	0.93	0.93	0.93	0.923
VGG16	0.91	0.909	0.901	0.905
Random Forest	0.83	0.833	0.833	0.83
XGboost	0.69	0.690	0.687	0.687
SVM	0.66	0.677	0.660	0.663

Fuente: Mehenika et al., 2020. *Mosquito Classification using Convolutional Neural Network with Data Augmentation*. p. 12

En las conclusiones de este artículo de investigación recomiendan poder continuar este tipo de investigación con un mayor número de datos, debido a que este trabajo se realizó con muy pocas imágenes. Así mismo se comprueba que la aplicación de *data augmentation* aumenta el nivel de clasificación en los entrenamientos de modelos de aprendizaje automático donde no se dispone de una amplio y diverso dataset.

En el artículo de investigación publicado en el año 2021 y que tiene por título *A deep convolutional neural network for classification of Aedes Albopictus mosquitoes* y que tiene por autores a Gereziher Adhane, Mohammad Mahdi Dehshibi, and David Masip. Se analiza la problemática de la propagación de mosquitos que trasmiten enfermedades mediante picaduras como el Dengue, fiebre amarilla, Zika y Chikungunya. Entre las especies de mosquito que trasmiten estas enfermedades está el mosquito tigres (Aedes Albopictus), que es la especie de estudio en este paper de científico, donde se busca clasificar mediante imágenes al mosquito Aedes Albopictus de otras especies de Aedes. Para este objetivo de

investigación se desarrolló dos modelos de aprendizaje profundo como el VGG16 y ResNet 50.

La metodología empleada se basó en la formulación de un modelo de clasificación binaria a partir de imágenes del mosquito de tipo Aedes Albopictus, donde la red neuronal objetivo clasificará como valor de 1 cuanto la imagen corresponda al mosquito de Aedes Albopictus y cuando el valor sea 0 se clasificará a la imagen como un mosquito de tipo no Aedes. En la primera etapa se realizó el proceso de recolección de las imágenes, en la cual el dataset utilizado pertenece al portal web Mosquito Alert que proporciona imágenes de varios tipos de mosquitos que fueron etiquetadas por expertos en entomología. Este conjunto de datos contiene imágenes etiquetadas como no comprobadas, probables y no confirmadas. Para esta investigación se utilizó 3334 imágenes etiquetadas como casos confirmados de Aedes Albopictus y 3014 imágenes de otras especies que no son Aedes Albopictus.

Figura 14. Dataset de los insectos Albopictus.



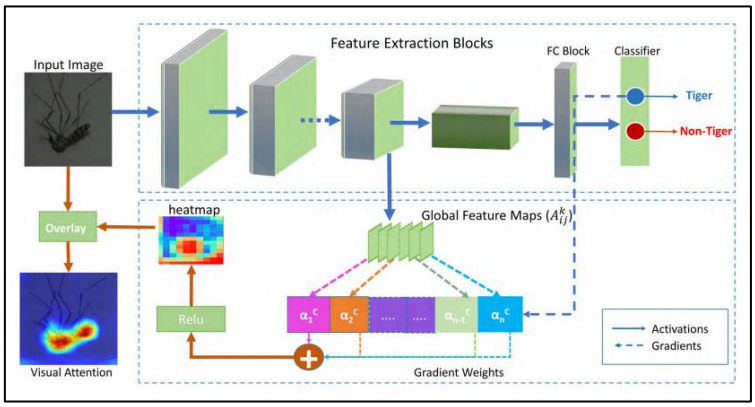
Fuente: Adhane et al., 2021. *A deep convolutional neural network for classification of Aedes albopictus mosquitoes*. p. 5

En esta misma etapa también se realizó el pre procesamiento de las imágenes aplicando dimensiones de 224x224, donde se realizó la normalización de estos datos mediante z-score y

la aplicación de validación cruzada (5-Fold) en el porcentaje de datos de entrenamiento y prueba.

En la segunda etapa se realizó los procesos experimentales donde primero se implementó una arquitectura VGG16 bajo el enfoque de aprendizaje por transferencia que fue pre entrenado previamente con datos de ImageNet. Esta arquitectura consta de 13 capas convolusionales, a cada capa le sigue un capa de max-pooling una función de activación ReLU. Así mismo la cola de esta arquitectura contiene dos capas completamente conectadas (ReLU y dropout) y por último se añadió un módulo softmax que esta seguido de una función de pérdida de entropía para identificar el vector de características en cada clase. Adicionalmente en esta etapa también se utilizó la técnica de Grad-CAM que permite visualizar los puntos característicos analizados por la red.

Figura 15. Metodología de la técnica Grad-CAM.

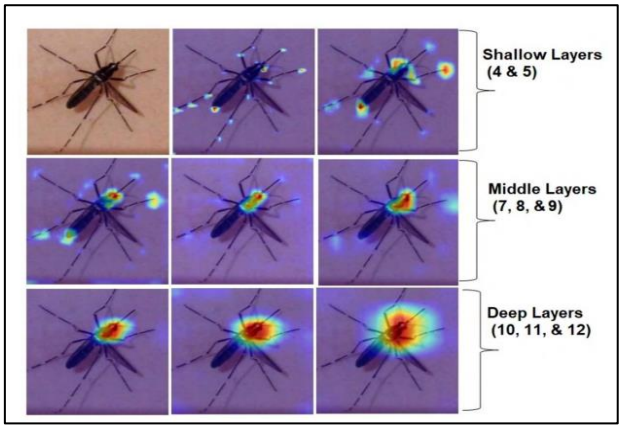


Fuente: Adhane et al., 2021. *A deep convolutional neural network for classification of Aedes albopictus mosquitoes*. p. 3

El modelo VGG16 y ResNet50 fueron trabajados con los mismos parámetros, donde fueron entrenados en todo el proceso con PyTorch, una taza de aprendizaje de aprendizaje del 0.001, mini lotes de 64 y 32 épocas. Posterior al entrenamiento de estos modelos con datos

etiquetados y confirmados de mosquitos de tipo Aedes Albopictus se realizó también el entrenamiento con datos etiquetados como probables. Por último se realizó la observación de las características que analiza la red mediante Grad-CAM.

Figura 16. Muestra de la técnica Grad-CAM



Fuente: Adhane et al., 2021. *A deep convolutional neural network for classification of Aedes albopictus mosquitoes*. p. 7

En la última etapa se presentaron los resultados de los modelos, en la cual el modelo VGG16 obtuvo un Accuracy de 93.86% en comparación al modelo ResNet50 que obtuvo 93.21%.

Tabla 7

Resultados de los modelos VGG16 y ResNet50

Método		**VGG16**	**ResNet50**
5-Fold Cross Validation			
Train	Accuracy (%)	94.61 ± 0.24	93.64 ± 0.29
	Loss	0.13	0.18
Test	Accuracy (%)	93.86 ± 0.54	93.21 ± 0.62
	Loss	0.208	0.207

Fuente: Adhane et al., 2021. *A deep convolutional neural network for classification of Aedes albopictus mosquitoes*. p. 5

Como conclusión en este artículo de investigación mencionan que la clasificación en este tipo de imágenes de insectos que propagan enfermedades sirve de un apoyo importante para identificar y clasificar a las especies de mosquito como el Aedes Albopictus en la trasmisión de enfermedades. Así mismo el modelo VGG16 para este tipo de clasificación logró un gran equilibrio entre eficiencia y el número de parámetros reflejando óptimos resultados en la precisión del modelo.

2.2.Bases Teóricas

2.2.1. Inteligencia Artificial

Los sistemas basados en inteligencia artificial, tienen la finalidad de emular las acciones y respuestas humanas para llevarlas a su máxima expresión y capacidad. Estos sistemas están compuestos por algoritmos que buscan ejecutar procedimientos y paralelamente aprender del resultado de esos procedimientos, básicamente como lo haría el cerebro humano. Los algoritmos de inteligencia artificial son altamente técnicos a nivel matemático y diversamente estructural en su composición que tiene el fin de mejorar y apoyar en los ámbitos donde el ser humano tiene limitaciones.

2.2.1.1. La inteligencia artificial en la actualidad

Como se mencionó en párrafos anteriores, los modelos matemáticos de inteligencia artificial se crearon muchos años atrás, estos modelos durante décadas estuvieron paralizados debido a que los algoritmos se planteaban de manera teórica y por las limitaciones de poder computacional y de datos no se podían implementar de forma práctica. Pero todo eso cambió gracias al crecimiento de la tecnología a nivel de hardware en los últimos años, es decir cada año en los mercados tecnológicos se encuentran mejores procesadores y computadoras que

soportan estos tipos de algoritmos matemáticos. Por otro lado en la actualidad debido al internet y otros tipos de tecnología, se generan grandes cantidades de datos y muchos de ellos están en la web con fácil accesos y disponibilidad para trabajos en diversos proyectos de inteligencia artificial. Mismos proyectos que son impulsados por instituciones públicas y privadas para el desarrollo de la ciencia de datos en diversas partes del mundo.

A la actualidad se desarrollaron múltiples sistemas basados en inteligencia artificial, que se implementaron en distintos sectores de la sociedad. Por ejemplos para las búsquedas en internet se utiliza Google, una plataforma que se optimiza mediante la inteligencia Artificial. En el sector de seguridad se encuentran aplicaciones de reconocimiento facial basado en inteligencia artificial. Así mismo en las organizaciones utilizan los famosos RPA, que son sistemas que automatizan las distintas áreas de negocio y sus correspondientes procesos.

Al respecto (Rouhiainen, 2021) menciona que "…la inteligencia artificial busca hacer que casi todo sea más barato, en menos tiempo, más efectivo y cambiará profundamente sectores como el de la conducción , viajes, salud , educación , comercio , agricultura, finanzas, ventas y marketing…"

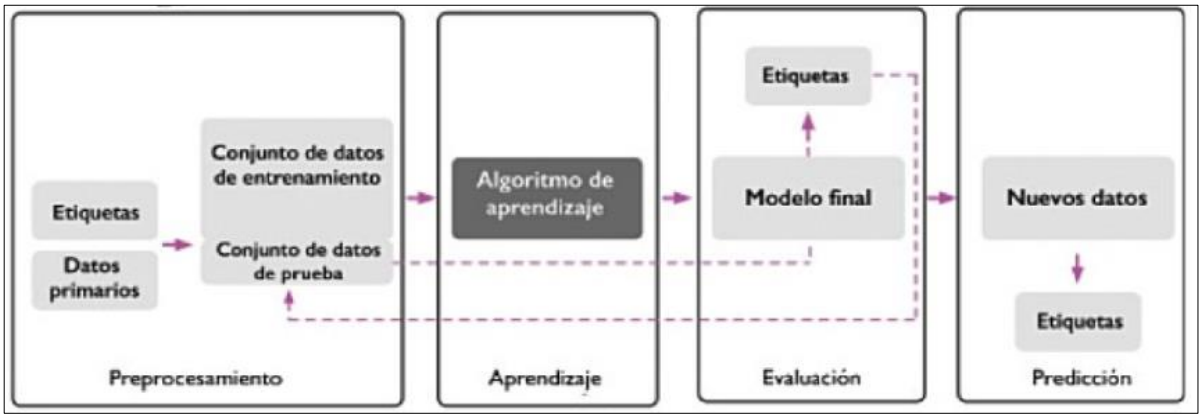
Como se ha podido mostrar, las aplicaciones de inteligencia artificial desde sus orígenes más allá de la contribución a la ciencia, siempre han tenido un objetivo de solución a una problemática, de buscar nuevas formas de generar una solución eficiente mediante resultados óptimos.

2.2.1.2. Fases del desarrollo de los modelos de inteligencia artificial

El primer componente de todo sistemas basados en inteligencia artificial parten desde su propia alimentación de datos, en la actualidad debido a la tecnología incorporada en

distintos dispositivos de uso diario y por ayuda de distintas instituciones públicas y privadas alrededor del mundo, se logran construir conjuntos masivos de datos que son la alimentación directa para le generación de modelos de inteligencia artificial. Con esta información se pueden trabajar diversas arquitecturas de machine learning y deep learning, que dependiendo de los datos que se dispone se puede aplicar distintas técnicas como el aprendizaje supervisado, no supervisado y aprendizaje por refuerzo. Esta estructura está presente en la aplicación de esta tecnología en distintos sectores estratégicos.

Figura 17. Etapas de desarrollo estándar de un modelo de inteligencia artificial



Fuente: Por S. Vahid, 2020, Python Machine Learning. P.15

2.2.2. Vectores de arbovirosis

2.2.2.1. Situación actual de las enfermedades trasmitidas por insectos

En los últimos años las enfermedades trasmitidas por insectos ha aumentado drásticamente. En lo que respecto a la salud pública, las enfermedades que provienen de contagios de vectores de arbovirosis que son los zancudos que representan el 17% de todas las infecciones y cada año causan más de 1 millón de muertes en la región de América central y América del sur. Así mismo los arbovirosis como el Dengue, Chikungunya y el Zika representan más del 60% de todas las enfermedades de trasmisión mediante un vector, que en

muchos de los casos son un insecto (Organización Mundial de la Salud, 2020, como se citó en Moncada, 2021). Estos casos de contagios son provocados por factores biológicos, sociales y económicos que afecta gravemente a la población en zonas endémicas (Moncada, 2021).

2.2.2.2. Vectores

Son los organismos que transportan en sus cuerpos distintos tipos de enfermedades que pueden contagiarse y proliferar una enfermedad hacia otro ser vivo. Estos organismos denominados vectores transportan la enfermedad a nivel patógeno. El procedimiento de una trasmisión por vectores se genera a causa de la propia interacción ecológica de este organismo con otros seres vivos, del cual esta trasmisión es de tipo horizontal debido a que el este organismo sirve de intermediario de contagios de múltiples enfermedades infecciosas entre varios seres vivos. (Rovid et al, 2011).

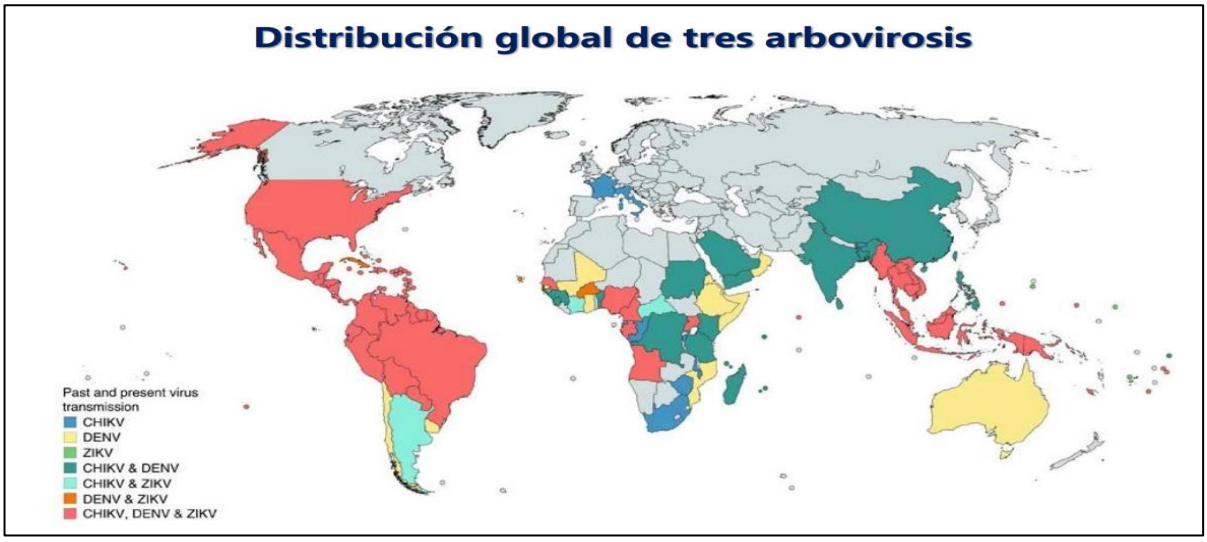
En la actualidad existen varias clases de vectores biológicos como los mosquitos, garrapatas, ácaros, caracoles, piojos, flebótomos y triatóminos, Pero la clase de vector que más transmisión de enfermedades ha generado son los mosquitos, conocidos científicamente como insectos de tipo culícidos (Organización Mundial de la Salud, 2020).

2.2.2.3. Vectores de arbovirosis

Los vectores de arbovirosis son los organismos biológicos de tipo artrópodos que transmiten distintos tipos de virus, que en la actualidad existen un aproximado de 500 tipos de virus. De este conjuntos de virus, solo 150 provocan enfermedades en las personas (Jiménez citado en CDC, 2021). Estos virus se dividen en tres familias, la primera llamada Bunyaviridae, donde se encuentran el virus de la encefalitis de La Crosse, Hantavirus y Fiebre del Peluche. La segunda familia denominada Flaviviridae, donde se encuentran los

virus del Dengue, Fiebre Amarilla y Zika. La última familia tiene por nombre Togaviridae, donde se encuentran los virus del Chikungunya y Mayaro. De estos virus los que tienen mayor número de contagio a nivel mundial son el Dengue, Zika y Chikungunya (Espinoza citado en CDC, 2021).

Figura 18. Distribución Global del Dengue, Zika y Chikungunya

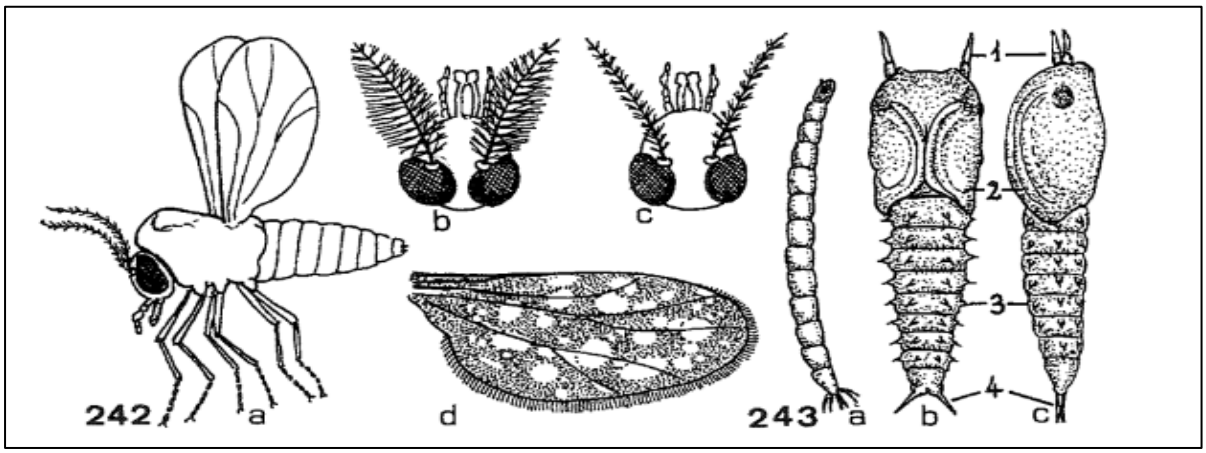


Fuente: Rückert, C., Weger-Lucarelli, J., Garcia-Luna, S. et al. Impact of simultaneous exposure to arboviruses on infection and transmission by Aedes Aegypti mosquitoes. 2017. p3

2.2.2.4. Vectores de arbovirosis de tipo culícidos

Son las especies de insectos que tiene un tamaño variado y mayormente se les conoce como mosquito. Esta especie tiene como principal foco de nido las áreas húmedas, donde dependiendo de su etapa de reproducción las hembras implantan en estas zonas sus huevos (García, 2020).

Figura 19. Morfología de un insecto de tipo culícido



Fuente: Gállego. 2007. *Manual de Parasitología. Morfología y biología de los parásitos de interés sanitario*. Edicions Universitat Barcelona. p. 448

2.2.2.5. Transmisión de vectores de arbovirosis de tipo culícidos

Los vectores de arbovirosis de tipo culícidos se encuentran presentes en regiones tropicales y subtropicales de distintos países del mundo. Su hábitat está alineado a altos niveles de temperatura y climas cambiantes. Estas características están presentes en áreas urbanas y suburbanas; el contagio comienza directamente cuando este vector pica a una persona que tiene el virus y luego de picarlo transporta el virus hacia otra persona mediante otra picadura. Estos vectores realizan estos hábitos de contagio por su propio naturales de necesidad de sangre (CDC, 2021).

2.2.2.6. Clasificación de vectores de arbovirosis de tipo culícido

Para una correcta clasificación de vectores de arbovirosis, es necesario conocer e identificar todos los aspectos físicos del vector. La identificación de vectores implica el conocimiento de la estructura anatómica y la fisiológica por cada tipo de insecto para entender correctamente el funcionamiento de la trasmisión de agentes virales que producen enfermedades como el Dengue, Chikungunya y Zika. Por ello se busca analizar cada detalle

del cuerpo del insecto, es decir si es alargado, cilíndrico y el tipo de texturas de color que presentan en sus cuerpos. Así mismo todo insecto está compuesto por tres regiones corporales cabeza, tórax y abdomen, cada una de ellas con características singulares por tipo de especie. (García, 2020).

Es así que en todo proceso de clasificación de vectores de arbovirosis participa activamente el reconocimiento de las características físicas del insecto del cual se debe analizar su estructura física a detalle. Así como también su comportamiento en el medio ambiente. La familia de insectos de tipo culícidos que más enfermedades peligrosas trasmite es la familia Aedes que tiene una mayor presencia en el Perú (CDC, 2022).

2.2.2.7. Familia Aedes

Estos mosquitos culícidos que pertenecen a la familia Aedes, son en su mayoría insectos que pican a las personas u otros animales buscando alimentarse de sangre. Las hembras de esta especie busca la sangre para alimentar a sus huevos y reproducirse. Esta especie se divide principalmente en dos tipos, Aedes Albopictus, Aedes Aegypti. Todas estas clasificaciones de la familia Aedes son los responsables de trasmitir las enfermedades del Dengue, Zika y Chikungunya (Keystone J, Kozarsky P, Connor B, Nothdurft H, Mendelson M, Leder K, 2020).

2.2.2.8. Familia Culex

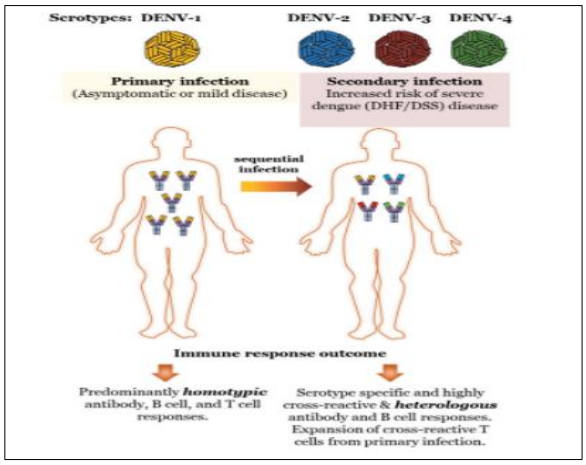
Tienen por nombre científico Culex Pipiens y es el tipo de insecto culícido más común en los hogares domésticos que pertenecen a las regiones tropicales y sub tropicales. Estos insectos también trasmiten enfermedades pero no son letales para el ser humano. Así mismo su reproducción se genera en ambientes contaminados y recipientes de aguas estancadas (Álfau, 2012).

2.2.2.9. Enfermedades endémicas en el sector de Salud del Perú

2.2.2.9.1. Enfermedad del Dengue

El dengue es una enfermedad de tipo infección viral que es causada por el "virus del dengue". Este tipo de virus genera 4 formas de serotipos que son DENV-1, DENV-2, DENV-3 y DENV-4, que se encuentran presentes en la región de las américas (OMS, 2022). El contagio consecutivo de cada uno de estas variaciones del virus puede desencadenar en tres tipos de manifestaciones, el dengue con manifestación de síntomas, dengue sin manifestación de síntomas y dengue grave. (CDC, 2022). Además a la actualidad no existe procedimientos o tratamientos especialmente para los casos de dengue, por ende solo se aplican seguimientos y asistencia médica de apoyo para pacientes con sintomatología (Rosman y Cardellach, 2020).

Figura 20. Tipos de serotipos del virus del dengue

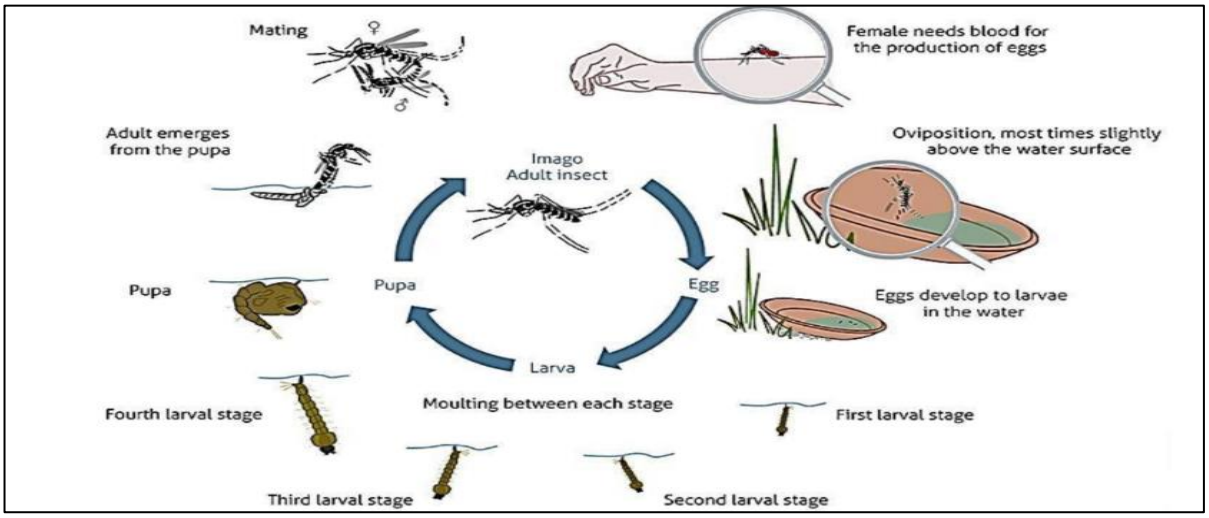


Fuente: Aparicada, M., Saxena, S. 2020 *Dengue Fever in a One Health Perspective*. p.62

2.2.2.9.2. Formas de contagio del dengue.

El virus del dengue se trasmite directamente mediante un insecto de tipo culícido que pertenece a la familia Aedes. De este grupo son dos especies en específico que trasmiten el virus, el Aedes Aegypti y el Aedes albopictus (Mayor et. al, 2019). Las fases de transmisión de este enfermedad inicia con la picadura de estos vectores a una persona que tiene el virus del dengue, este insecto se infecta y al picar a otras personas propaga la enfermedades en distintas zonas geográficas. Pero una propagación mayor de contagio también se genera cuando este zancudo infectado con el virus, se reproduce en zonas de aguas estancadas y en ambientes contaminados, aumentando un mayor número de insectos infectados y que por consecuencia provocaran un mayor número de contagios (Instituto Nacional de Salud, 2020).

Figura 21. Ciclo de vida del insecto de tipo culícido de la familia Aedes

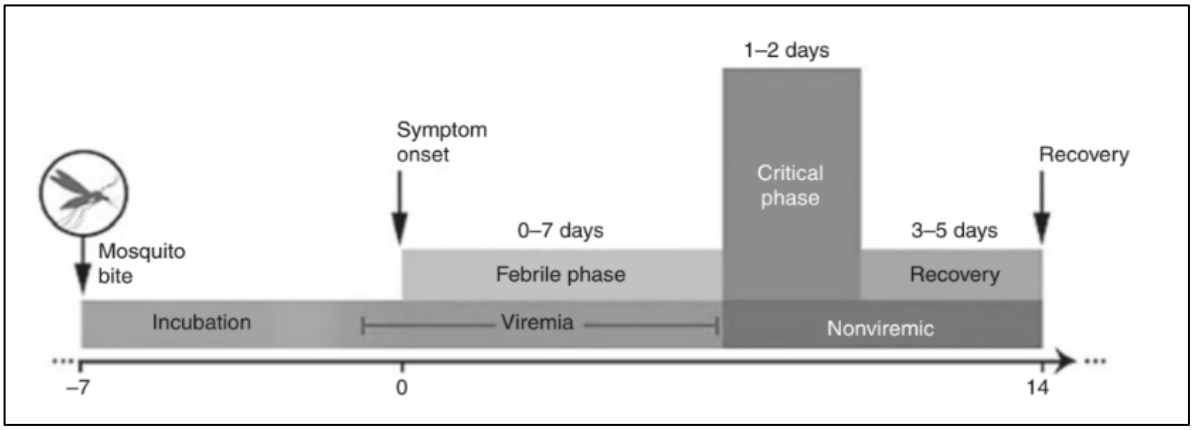


Fuente: Ram, A. *Spatial Distribution of Dengue Vectors.* 2021. (p.6)

En los casos de dengue con o sin manifestación de síntomas pasa por un procesos de incubación y contagio vía picadura de estos insectos, en donde tiene una duración de 14 días

todo el proceso del virus en el cuerpo del ser humano (Hussain et. al, 2020). La figura N° 1 muestra las etapas en estos tipos de casos.

Figura 22. Etapa de incubación del virus del Dengue en el ser humano

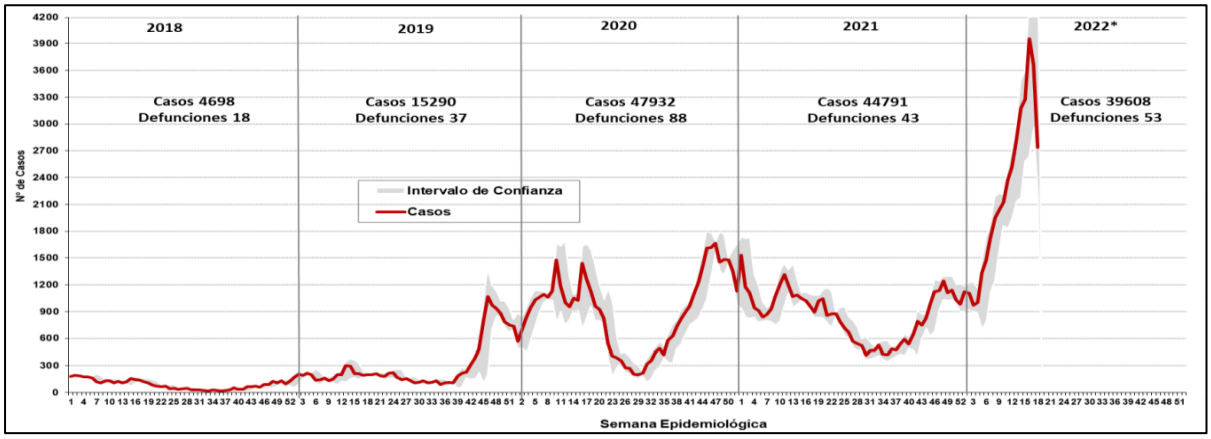


Fuente: Hussain et. al 2020. *Expanded Dengue Syndrome*.p.4.

2.2.2.9.3. Situación del dengue en el Perú

En la actualidad el Dengue en el país tiene una presencia continua y de altos niveles de contagio, en las zonas de la región selva, sierra y parte del norte del país. En este año 2022 se tiene 39608 casos reportados a nivel nacional, de los cuales se tiene 53 casos de fallecimiento. El departamento con mayor número de casos de esta epidemia es Piura con 4072 casos, y en este año las cifras aumentaron a 10198.

Figura 23. Número de casos de dengue por semana epidemiológica del 2018 al 2022



Nota. Centro Nacional de Epidemiología, Prevención y Control de enfermedades. *Sala de situación de Salud – Dengue*. 2022. (p.2)

A nivel regional son 19 departamentos que tienen presencia de contagios de este virus. El top 5 de los departamentos que tienen altos registros de esta enfermedad son Piura, Loreto, Ica, Ucayali y Cusco.

Tabla 8

Número de casos de dengue por departamento

Casos Acumulados						
Departamentos	**2017**	**2018**	**2019**	**2020**	**2021**	**2022***
Piura	44275	525	70	125	4072	10198
Loreto	1089	1833	2547	7800	5115	4346
Ica	4384	127	51	7144	4208	3967
Ucayali	779	317	213	9781	3140	2405
Cusco	537	79	54	2499	1982	2239
Cajamarca	420	6	398	266	4015	2239
Lambayeque	1579	4	770	568	843	2203
San Martin	460	98	1969	4955	4532	2202
Madre de Dios	565	1234	7398	3233	1333	2023
Ancash	1720	6	20	0	28	1767
Junín	220	51	482	3738	5090	1547
Tumbes	4145	64	508	3093	1455	989
Huánuco	92	25	35	1133	2811	925
Amazonas	93	109	164	873	2369	921
Lima	362	11	46	350	1190	746
Ayacucho	1657	202	95	1583	1663	375
Pasco	4	3	36	377	616	341
La Libertad	5904	3	366	390	262	164
Puno	0	0	65	20	51	9

Callao	5	0	0	4	9	1
Arequipa	0	0	0	0	7	1
Moquegua	0	1	0	0	0	0
Huancavelica	0	0	0	0	0	0
Tacna	0	0	0	0	0	0
Apurímac	0	0	0	0	0	0
Total	68290	4698	15287	47932	44791	39608

Nota.* Es el acumulado solo del año 2022. Fuente: Centro nacional de Epidemiología, Prevención y Control de Enfermedades (CDC). 2022

El departamento de Lima no es ajeno a estos contagios por dengue, el distrito con mayor número de casos contagio correspondiente al año 2022 es comas con 197 casos reportados. Si se analiza el impacto de los contagios a nivel de edades en el Perú, las personas de 30 a 59 años tienen el mayor porcentaje de contagios con un 36.87% y los que tienen menos incidencia de contagios son los adultos mayores con un 8.34% (CDC, 2022).

Tabla 9

Distribución de casos de dengue por grupos de edades en el Perú en el 2022

Según grupo de edad	N° de casos	%
Niños (0 - 11 años)	6841	17%
Adolecentes (12- 17 años)	5616	14%
Joven (18- 29 años)	9244	23%
Adulto (30-59 años)	14603	37%
Adulto Mayor (60 + años)	3304	8%
Total	39608	100%

Nota. Fuente: Centro nacional de Epidemiología, Prevención y Control de Enfermedades (CDC). 2022

2.2.2.9.4. Enfermedad del Chikungunya

Es un virus de tipo ácido ribonucleico que pertenece al género “alphavirus” correspondiente a la familia “Togaviridae” que tuvo su origen en Tanzania en el año de 1952, que ha tenido presencia en distintas partes del mundo como en Asia, África y América. En varios estudios realizados a este virus, se han encontrado tres tipos de cepas diferentes, el del oeste de África, el asiático y el del centro-sur de Asia. Para la enfermedad que genera este

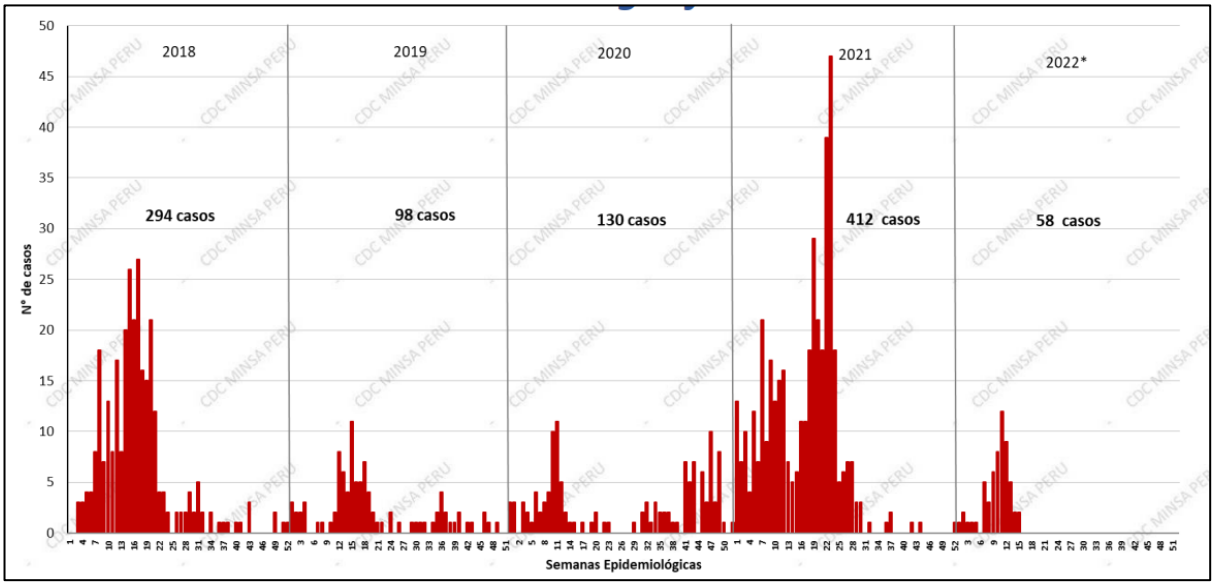
virus no existe ningún tratamiento especializado ni ninguna vacuna que contraste este virus (Rosman y Cardellach, 2020). Así mismo en la población está presente en dos tipos de casos, caso probable y grave de Chikungunya (CDC, 2021).

2.2.2.9.5. *Formas de contagio del Chikungunya.*

Este virus también se trasmite mediante la picadura de insectos de tipo culícidos que pertenecen a la familia Aedes, es decir los Aedes Aegypti y Albopictus. El proceso de transmisión de este virus es el mismo al virus del Dengue que se trasmite a través de la picadura de estos tipos de insectos denominados vectores. Otra forma de propagación esta enfermedad es mediante la reproducción de estos zancudos, en los cuales sus nidos son aguas estancadas y lugares con contaminación ambiental. Además posee un ciclo de transmisión de mosquito-humano-mosquito de aproximadamente 7 días (OMS, 2020).

En el Perú la presencia de este virus se ha incrementado en años recientes, para el año 2020 se reportaron 130 casos y para noviembre del 2021 se reportaron 412 casos a nivel nacional y para este año al mes de abril se reportaron 58 casos. De estos casos el 86.21% se encuentran en el departamento de Piura y el resto de porcentaje se encuentra presente en Tumbes, Ancash, Ucayali, Lambayeque, Madre de Dios, Loreto y San Martin (CDC, 2022).

Figura 24. Número de casos de Chikungunya en el Perú 2018 - *2022



Nota. *Hasta Abril del 2022. Fuente: Centro Nacional de Epidemiología, Prevención y Control de enfermedades. Sala de situación de Salud – Dengue. 2022. p.1

2.2.2.9.6. Enfermedad del Zika

Este tipo de virus fue registrado por primera vez en África en el año de 1947, desde esa fecha se han identificado diversos brotes en distintas parte s del mundo. En Latinoamérica se han registrado mayor número de casos en Brasil pero lamentablemente el Perú por su propio clima tropical de la región selva también se ha identificado casos de este virus (OMS, 2018).

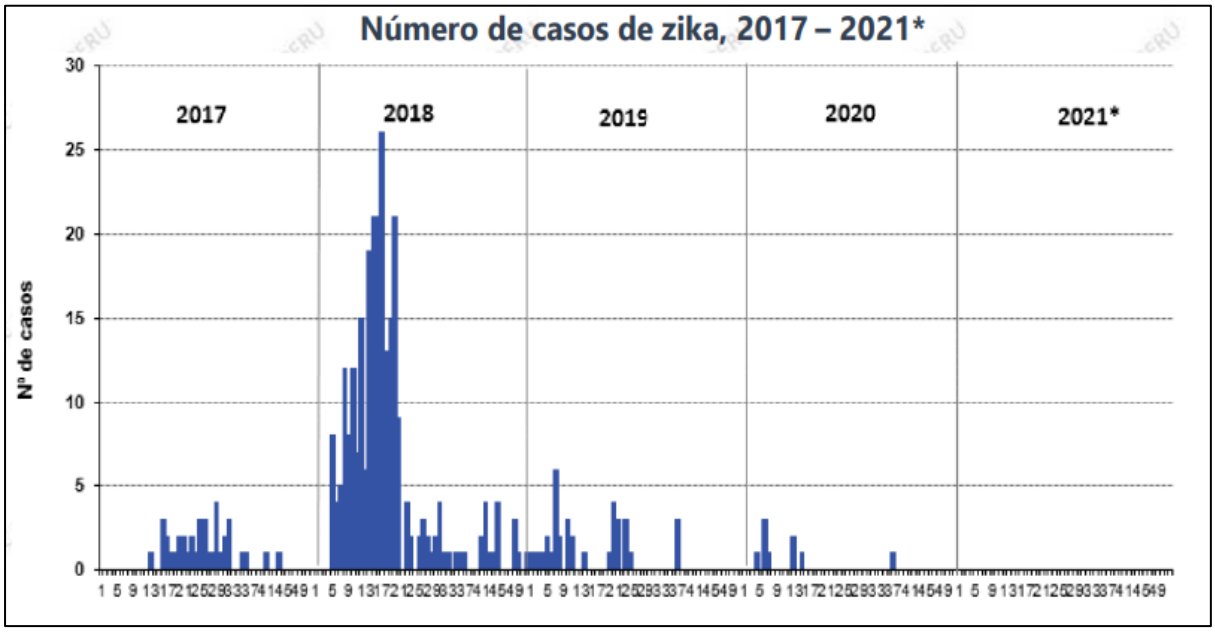
2.2.2.9.7. *Formas de contagio del Zika.*

La trasmisión de este virus es la misma que la del virus del Dengue y Chikungunya, que son trasmitidas por la picadura de un vector de la familia culícido de tipo Aedes Aegypti y Aedes Albopictus. Así mismo el virus también se puede trasmitir de madre a hijo en un contexto de un embarazo.

En los últimos reportes del Centro Nacional de Epidemiología, Prevención y Control de Enfermedades del Perú entre los años 2017 y 2021 se han registrado 1067 casos

confirmados. El año 2018 tuvo mayor número de casos de contagio de esta enfermedad (CDC, 2021).

Figura 25. Número de casos de Zika en el Perú 2018 - *2021



Nota. *Hasta Marzo del 2021. Fuente: Centro Nacional de Epidemiología, Prevención y Control de enfermedades. Sala de situación de Salud – Dengue. 2022. p.7

2.2.2.9.8. Procedimientos de control de las arbovirosis

A nivel de la vigilancia entomológica el procedimiento de control se realizan en distintas etapas. En la etapa de prevención, este proceso consiste en una serie de pasos y procedimientos que buscan supervisar las potenciales amenaza de brotes epidémicos que de distintas enfermedades. En estos procedimientos se encuentra la obtención de información de vectores de arbovirosis, número de personas contagiadas, número de familias contagiadas entre otros datos de cada área geográfica (CDC, 2021).

En cada procedimiento de control realizada por un especializa en entomología, se realiza las inspecciones y levantamiento de información presencial en las zonas con potencialidad de brote endémico. Estos especialistas toman muestras de los mosquitos y

realizan un análisis a nivel morfológico para identificar las especies encontradas (CDC, 2021).

2.2.2.9.9. Recolección de información en la vigilancia entomológica

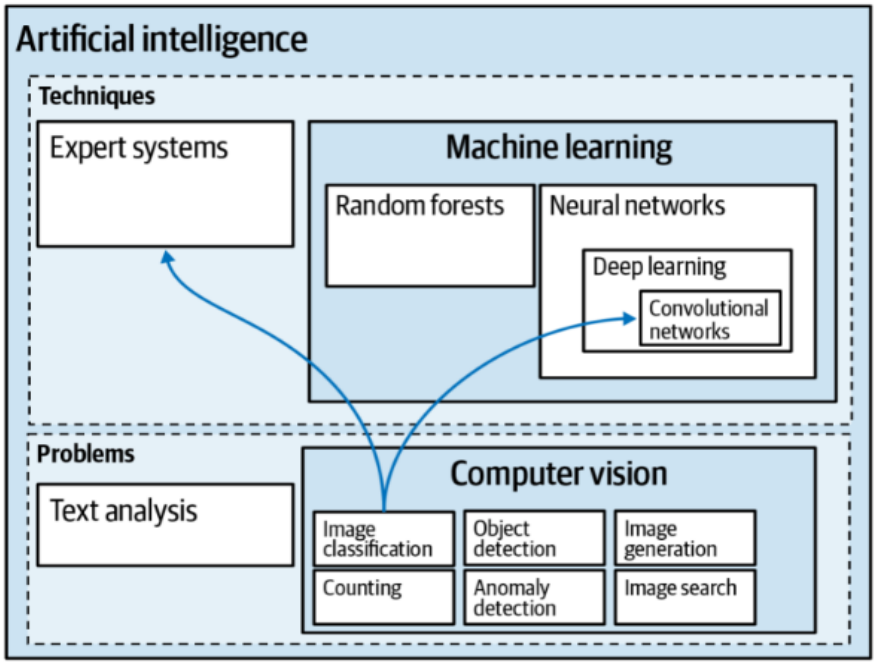
En el proceso de vigilancia entomológica se realizan distintos funciones desde la obtención de información de especies de vectores hasta los casos confirmados por personas y familias de una determinada localidad. Para estos procedimientos en necesario la recolección de la data de manera eficiente. En la necesidad de información de especies de vectores, lo realizan los entomólogos de manera física en las zonas endémicas o potencialmente endémicas, del cual el procedimiento es manual y recogen la data mediante la muestra del propio vector como una imagen de ese insecto, del cual lo llevan a un laboratorio posterior para ser analizado y clasificado por especie y determinar el grado de peligro que puede causar a la población (CDC, 2021).

Esta recolección de datos a nivel entomológico tiene que ser clasificadas por especie, en la cual este procedimiento actual es manear manual realizada por profesionales especialistas.

2.2.3. Visión artificial

Es el conjunto de técnicas y pasos enfocados en la extracción de información a nivel de caracteres de una imagen o video para el reconocimiento o identificación de determinadas figuras. Este proceso emula la capacidad visual del cerebro que constantemente está pasando por procesos de reconocimiento por imágenes. Entre sus procesos encontramos el análisis a nivel de pixeles para extraer determinados vectores característicos correspondientes a cada región de una imagen o video.

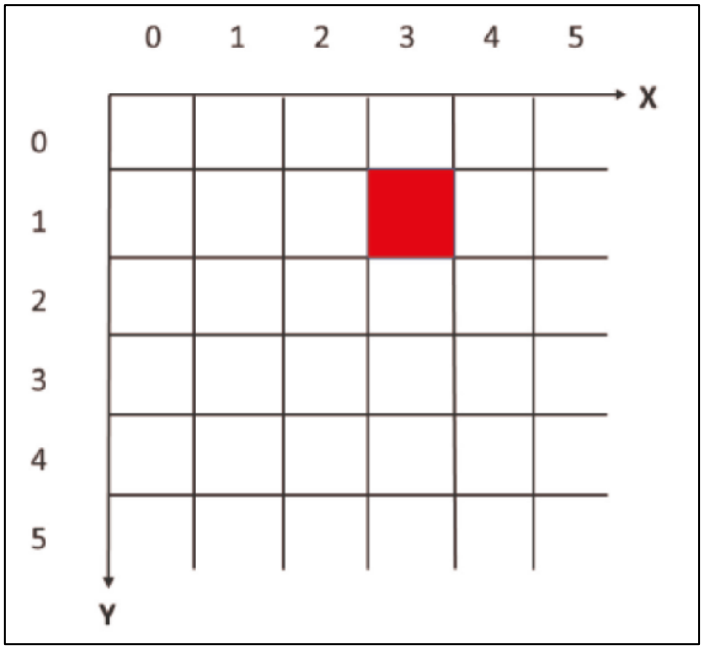
Figura 26. Marco de referencia de la visión artificial



Fuente: Lakshmanam et al, 2021. *Practical Machine Learning for Computer Vision*. O'Reilly Media, Inc. p. 3

Estas técnicas están basadas en tres principales pilares en el que radica toda su funcionalidad para el tratamiento de las imágenes. El primer pilar es adquisición, este pilar se enfoca en recibir a nivel numérico cada imagen. El segundo pilar es el procesamiento que es la aplicación de algoritmos matemáticos en cada capa de la imagen para resaltar los rasgos re regiones más importantes de la imagen. Por último el tercer pilar es el análisis de la imagen, en la cual también se aplica algoritmos matemáticos, donde se reconocerá cada objeto, clasificarlo, identificarlo y seguimiento en cada imagen. Así mismo a nivel técnico cada pixel está relacionada a su posición bajo un sistema de coordenadas (x, y) (Domínguez, 2021).

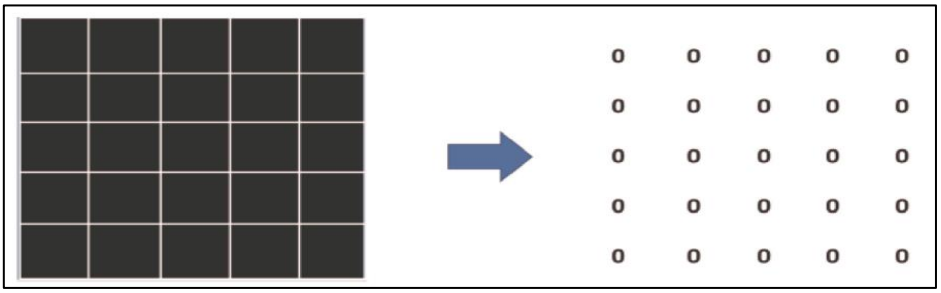
Figura 27. Ubicación del pixel en un sistema de coordenadas



Fuente: Domínguez. 2021. Visión artificial: Aplicaciones prácticas con OpenCV - Python. p. 10.

Cada imagen a nivel de código tiene su correspondiente matriz numérica, donde cada elemento de cada posición de la matriz indica la intensidad de luz de la imagen. Estos valores están entre 0 y 255. Es decir si se quiere tener una imagen totalmente negra, sería una matriz de puros ceros. Así mismo si se quisiera una imagen total mente blanca sería una matriz de puros 255 (Domínguez, 2021).

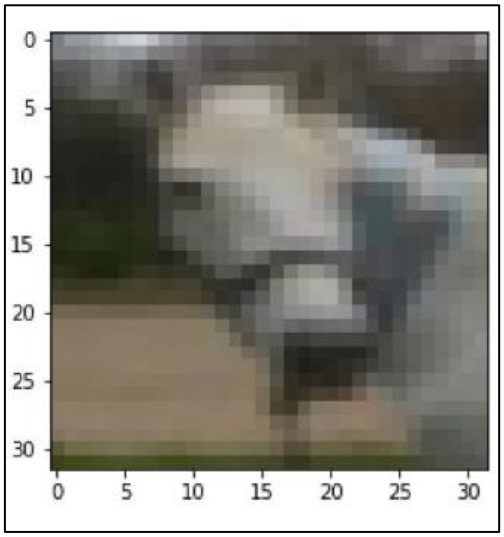
Figura 28. Ubicación del pixel en un sistema de coordenadas



Fuente: Domínguez. 2021. Visión artificial: Aplicaciones prácticas con OpenCV - Python. p. 22.

De esta manera se va visualizando e interpretando cada pixel con su correspondiente valor en su matriz numérica.

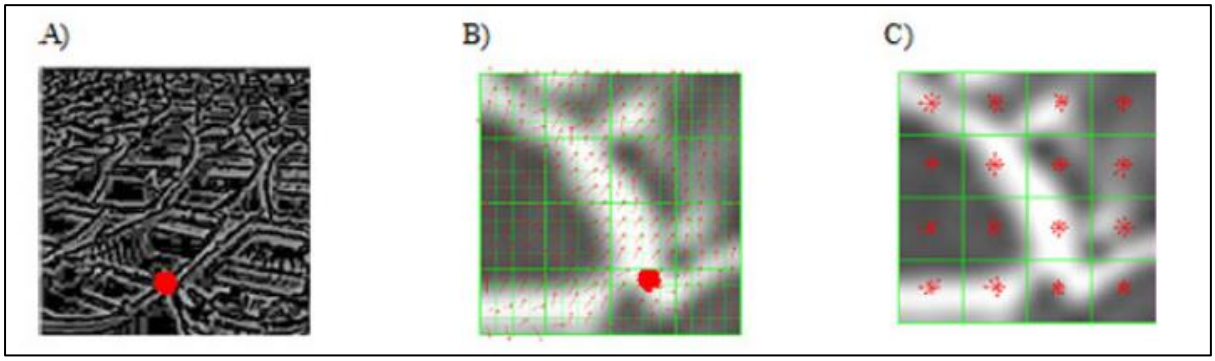
Figura 29. Ejemplo de carga de una imagen en código Python



Fuente: Martinez. 2021. *TensorFlow 2.0 Computer Vision Cookbook: Implement machine learning solutions to overcome various computer vision challenges*. p. 14.

Para la extracción de características se necesita la técnica de transformación de características invariantes de escala (SIFT) que son los que detectan los puntos característicos de los cuales no varía ante un cambio a nivel de traslación, escala, rotación, iluminación y traslación (Rosado, 2015).

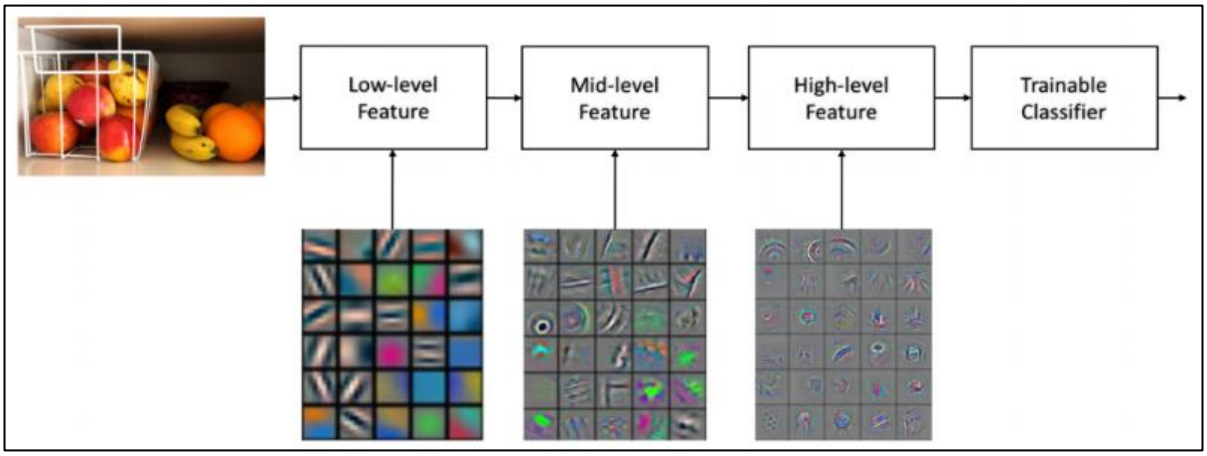
Figura 30. Keypoints en una imagen a nivel vector



Fuente: Rosado. 2015. *La visión artificial, un nuevo aliado para el análisis de imágenes artísticas*. Universidad de Barcelona p. 4

Además, luego de extracción de características son incorporadas en las entradas de los modelos aprendizaje automático o aprendizaje automático profundo, del cual se describe en la siguiente figura.

Figura 31. Esquema de análisis de imágenes en una red neuronal



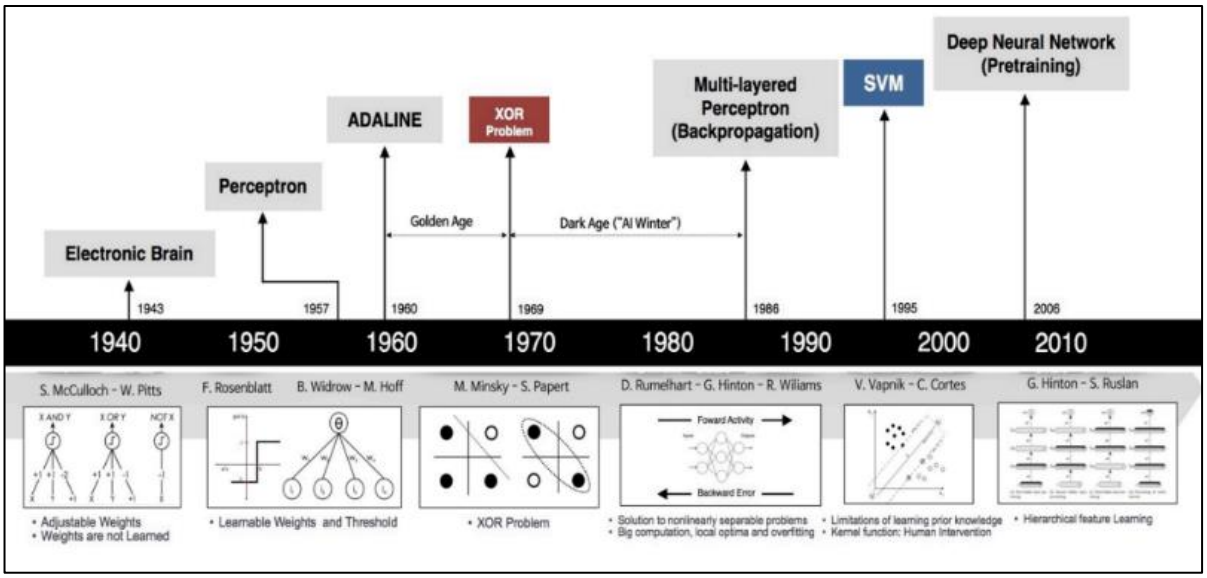
Fuente: Bhardwaj et al, 2018. *Deep Learning Essentials: Your hands-on guide to the fundamentals of deep learning and neural network modeling*. Packt Publishing Ltd. p. 9

2.2.4. Deep Learning

Según lo define IBM: “Es un subconjunto de machine learning (que a su vez es parte de la inteligencia artificial), donde las redes neuronales, algoritmos inspirados en cómo funciona el cerebro humano, aprenden de grandes cantidades de datos.” (IBM, 2018).

Estos modelos tuvieron su inicio en el año 2010, después de varias mejoras y modelos de redes neuronales.

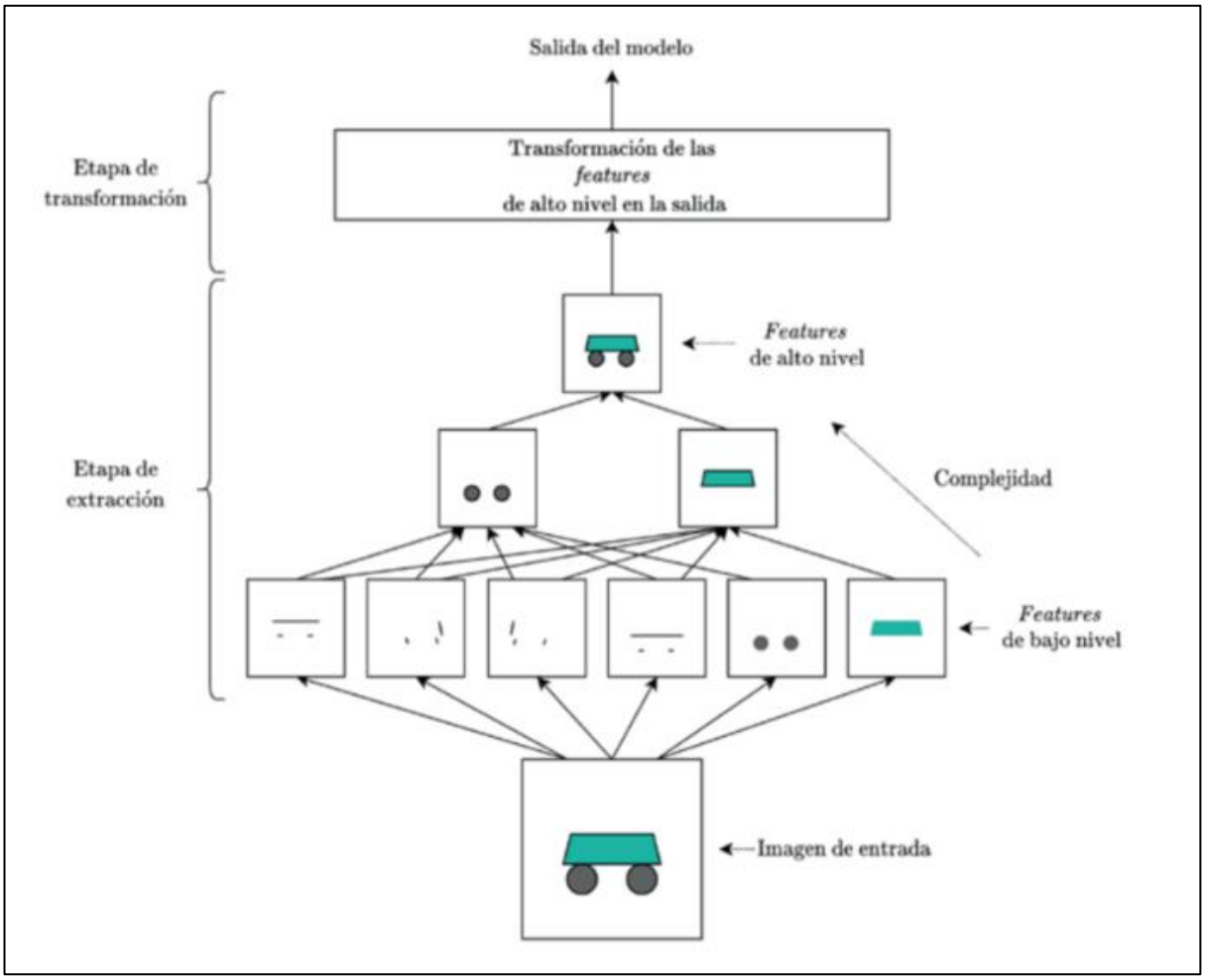
Figura 32. Línea de tiempo y aparición del Deep Learning



Fuente: Bhardwaj et al, 2018. *Deep Learning Essentials: Your hands-on guide to the fundamentals of deep learning and neural network modeling*. Packt Publishing Ltd. p. 14

Este sub campo de Machine Learning consiste en una arquitectura de red neuronal que contienen varias capas profundas con el objetivo de tener mejores resultados. Mejorando los pesos en cada nodo y mejorando la estructura de la red.

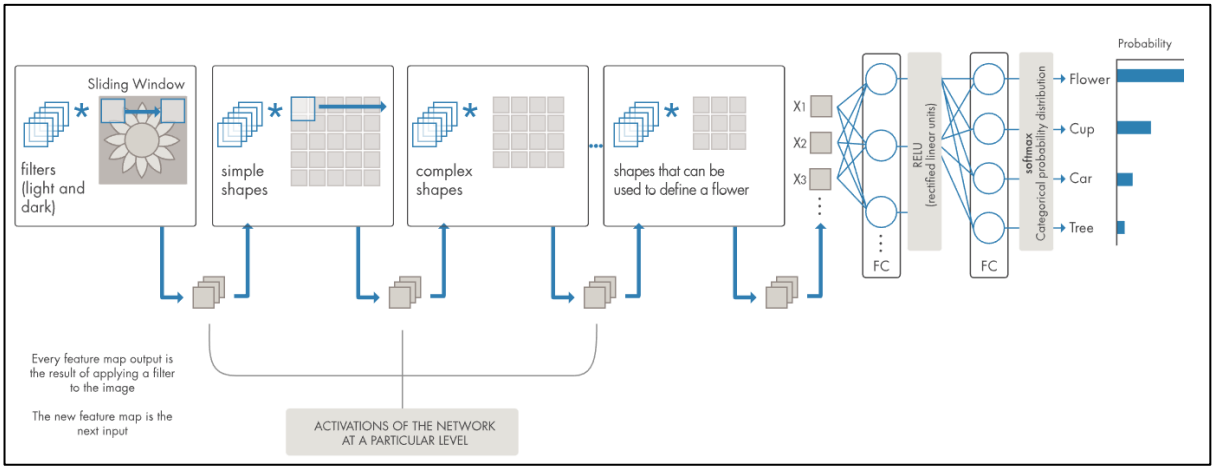
Figura 33. Esquema completo de un modelo de Deep Learning con sus diferentes etapas



Fuente: Pérez y Gegúndez. 2021. *Deep Learning*. Servicio de Publicaciones de la Universidad de Huelva. p. 196

Este tipo de arquitecturas tienen variaciones que han surgido en años recientes con el propósito de mejorar el rendimiento a nivel de resultados de predicción y clasificación. Estos modelos son las redes neuronales convolusionales, redes de memoria a corto plazo, redes neuronales recurrentes, redes neuronales adversativas, redes neuronales de función de base lineal, redes perceptron multicapa, autocodificadores, redes de creencias profundas y las redes neuronales transformadoras. La mayoría de estas redes neuronales están basadas en la siguiente estructura para la clasificación de imágenes (Biswal, 2022).

Figura 34. Esquema de redes neuronales con múltiples capas profundas convolusionales

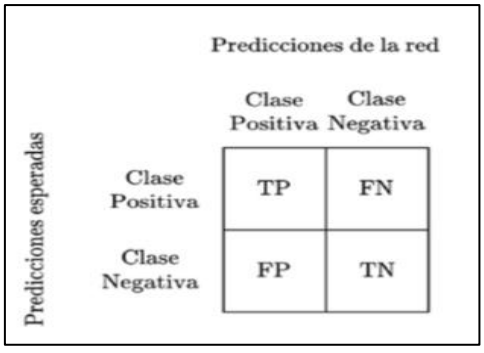


Fuente: MathWorks.com. 2021. *Deep Learning: Tres cosas que es necesario saber*. Matlab. p. 7

Es así que el aprendizaje profundo consiste en la aplicación de redes neuronales de capas profundas para un mejor aprendizaje y mayor precisión en los modelos de predicción y clasificación (IBM, 2018).

Estos modelos son medibles mediante sus métricas de evaluación, estas métricas son el despliegue de la denominada matriz de confusión, del cual se obtienen el *Accuracy*, *Recall*, *Precision*, *Especificidad y el AUC.* Donde TP significa "Verdaderos positivos", TN significa Verdaderos negativos, FP significa "Falsos positivos" y FN significa "Falsos negativos".

Figura 35. Matriz de confusión



Fuente: Pérez y Gegúndez. 2021. *Deep Learning*. Servicio de Publicaciones de la Universidad de Huelva. p. 105

De esta matriz mostrada en la figura 39, se calcula el Accuracy del modelo que consiste en evaluar el porcentaje de predicciones correctas realizadas por el modelo y que se obtiene con la fórmula siguiente:

$$\mathbf{Accaracy} = \frac{\mathbf{TP} + \mathbf{TN}}{\mathbf{TP} + \mathbf{TN} + \mathbf{FP} + \mathbf{FN}}$$

Donde TP = *Verdaderos positivos,* TN = *Verdaderos negativos,* FP = *Falsos positivos y* FN = *Falsos negativos*

Así mismo la métrica del Recall muestra el porcentaje de predicciones que son acertadas correctamente de una determinada clase y se calcula de la siguiente manera:

$$\mathbf{Recall} = \frac{\mathbf{TP}}{\mathbf{TP} + \mathbf{FN}}$$

Donde TP = *Verdaderos positivos y* FN = *Falsos negativos*

La métrica de precisión indica el porcentaje de valores acertados que son positivos con respecto al total de valores positivos y se calcula de la siguiente manera:

$$\mathbf{Presicion} = \frac{\mathbf{TP}}{\mathbf{TP} + \mathbf{FP}}$$

Donde TP = *Verdaderos positivos y* FP = *Falsos positivos*

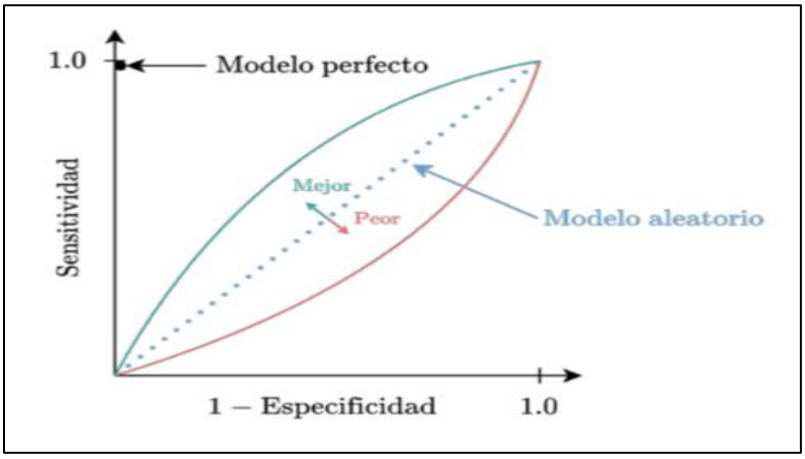
La métrica de especificidad indica el porcentaje de valores acertados que son negativos con respecto al total de valores negativos y se calcula de la siguiente manera:

$$\mathbf{Especificidad} = \frac{\mathbf{TN}}{\mathbf{TN} + \mathbf{FN}}$$

Donde TN = *Verdaderos negativos y* FN = *Falsos negativos*

Cada modelo tiene que tener una buena puntuación a nivel de resultados en cada métrica descrita anteriormente, considerando la especificidad y sensibilidad.

Figura 36. Curva ROC

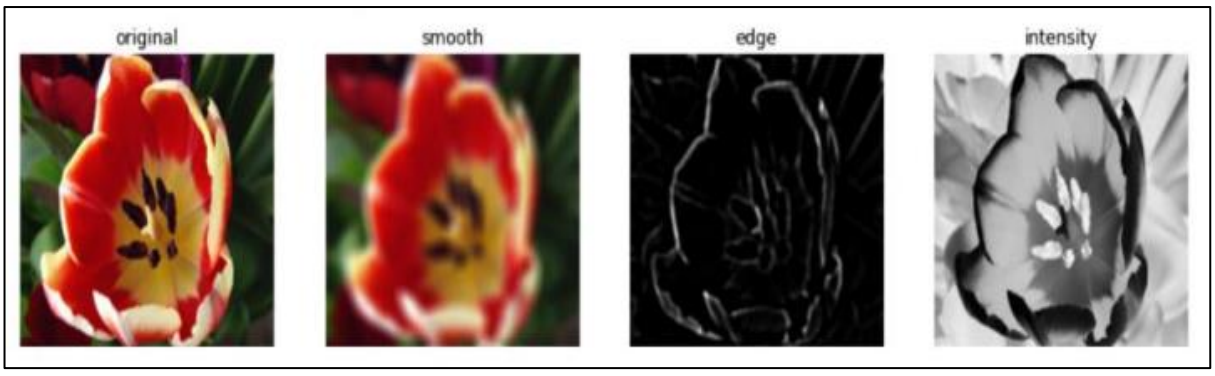


Fuente: Pérez y Gegúndez. 2021. *Deep Learning*. Servicio de Publicaciones de la Universidad de Huelva. p. 105

2.2.5. Pre procesamiento de imágenes

Es el grupo de técnicas que busca disminuir los ruidos y anormalidad que tienen un conjunto de datos. Así como también prepara la información de forma filtrada para que se encuentre listo para ingresar a un modelo de inteligencia artificial. Estos procesos son fundamentales y necesarios para aumentar la precisión del modelo de IA y obtener mejores resultados.

Figura 37. Filtro especial para imagen en el pre procesamiento

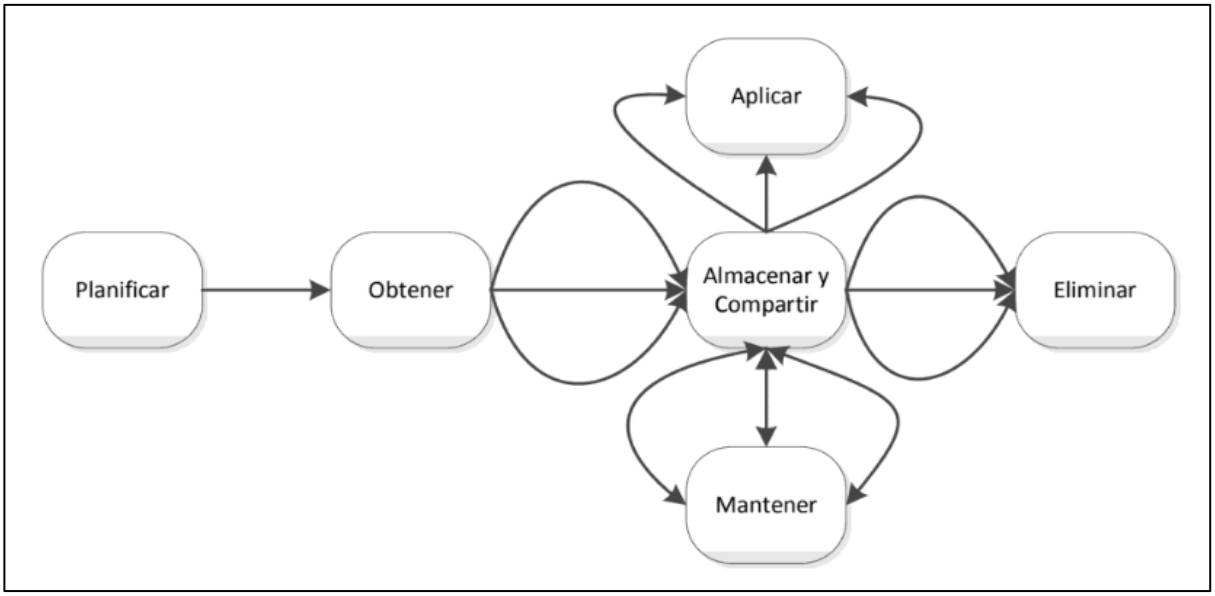


Fuente: Lakshmanam et al, 2021. *Practical Machine Learning for Computer Vision*. O'Reilly Media, Inc. p. 68

2.2.6. Calidad de datos

La calidad de datos consiste en ajustar los datos enfocados en el propósito de investigación. Los datos a nivel de inteligencia artificial deben tener unicidad, representatividad y ser completos a nivel de información que puedan servir a mejorar los modelos de IA para aumentar la precisión y su poder clasificatorio a nivel binario.

Figura 38. Flujo de datos para la calidad



Fuente: Piattini, M. 2018. *Calidad de datos*. p. 31.

2.3. Marco Conceptual o Contexto de Investigación

2.3.1. Redes Neuronales Artificiales

Es una arquitectura o modelo de inteligencia artificial que imita el comportamiento de las redes neuronales biológicas del cerebro humano. Esta arquitectura está compuesta por un conjunto de nodos de entrada que constituyen una capa. La red en general está compuesta por una capa de entrada, una o varias capas ocultas y una capa de salida, en la que cada nodo está asociado a un determinado peso (Kamath et al, 2022).

2.3.2. Aprendizaje Supervisado

Es un subconjunto de técnicas dentro del marco de Machine Learning que está enfocado en el procesamiento de datos, en la que cada registro está asociado a un target. Estas técnicas son aplicadas para la clasificación de múltiples clases.

2.3.3. Codificador de Transformador

Es el componente de una red neuronal transformadora que tiene por objetivo convertir entradas de secuencias de datos en un vector para la debida clasificación del modelo transformador. Es decir este componente toma una secuencia de longitud variable y la convierte en una salida de longitud fija (Kamath et al, 2022).

2.3.4. Tokens de clasificación

Son bloques de división de las regiones de las imágenes a procesar dentro del entorno del modelo de Vision Transformers, estos tokens son carga de pixeles y se mapean en vectores de características, que son las entradas para el codificador Transformers (Kamath et al, 2022).

2.3.5. Auto Atención

Es el concepto más importante de la arquitectura de redes neuronales transformadores, la auto atención es una capa que convierte un vector de entrada en tres vectores de salida que es la consulta, la clave y el valor. Estos vectores se obtienen proyectando el mismo vector en el tiempo mediante matrices de tiempo (Kamath et al, 2022).

2.3.6. Encoder

Es uno de los componentes del modelo de redes neuronales transformadoras que reciben los vectores tokenizados de características y las transforma en vectores unidireccionales de estado oculto (Kamath et al, 2022).

2.3.7. Dedocer

Es el componente que recibe los vectores de estado oculto, estos vectores de salida contienen la variable de contexto y la secuencia de salida decodificadas (Kamath et al, 2022).

2.3.8. Matriz de confusión

Es la matriz que indica el rendimiento del modelo implementado mediante determinados indicadores. Esta matriz está compuesta por el número de resultados que el modelo predice la etiqueta 1, número de resultados que el modelo predice la etiqueta 0. Así como también el número de resultados que predice de manera incorrecta la etiqueta 1 y la etiqueta 0.

2.3.9. Accuracy

Es una métrica que se desprende de la matriz de confusión, que se calcula como la dividiendo la suma de los verdaderos positivos y los verdaderos negativos entre la suma de estos mismos, la suma de los falsos positivos y falsos negativos. Esta métrica evalúa la exactitud del modelo en su función de clasificación.

2.3.10. Precisión

Es una métrica que también se desprende de la matriz de confusión, que se calcula como la división entre los verdaderos positivos y la suma de estos mismos con los falsos

positivos. Esta métrica permite conocer que tan bien el modelo clasifica los valores de etiqueta 1.

2.3.11. AUC

Tiene por nombre como “área bajo la curva”. Esta métrica ayuda a conocer el nivel de clasificación de datos por clases del modelo implementado. El valor se defino como toda la región de clasificación posible de valores, misma que se muestra en su gráfico correspondiente (Berlingerio et al, 2019).

2.3.12. Balanceo de datos

Es la técnica que se aplica en casos donde el número de registros por clases esta desnivelado, lo que provoca que el modelo pierda su nivel de precisión. Esta técnica se puede aplicar de varias formas como la generación sintética de datos, que básicamente es aumentar el número de registros de la clase minoritario e igualarla con la clase mayoritario.

2.3.13. Normalización de datos

Todo modelo de inteligencia artificial necesita datos pre procesados, parte de ese pre procesamiento es la técnica de normalización que se aplica aquellos datos que tienen variables que están bajo distintos rangos de medición. Esta técnica aliena los rangos de distribución de todos los registros para que el modelo de IA pueda procesarlos de mejor manera.

2.3.14. Culícidos

Según Basurco y Villanueva: “Los mosquitos culícidos son ectoparásitos que se posan temporalmente en la piel de un ser vivo para alimentarse. Son capaces importantes patógenos”. En este tipo de insectos se encuentra la familia Culex y Aedes.

2.3.15. Arbovirosis

Es el conjunto de virus que son trasmitidos por artrópodos, es decir para el caso de los virus como el Dengue, Zika y Chikungunya son trasmitidos por los insectos de tipo culícidos de la familia Aedes (CDC, 2021).

2.3.16. Entomología

Según (Rogg, 2001) “la entomología es la ciencia del estudio de los insectos”. En donde se analizar sus aspectos de comportamiento y morfología de todas las clases de insectos en el mundo.

2.3.17. Vigilancia Entomológica

Es uno de los procesos principales dentro del marco de prevención de enfermedades impulsadas por el Centro Nacional de Epidemiología, Prevención y Control de enfermedades. Este proceso se encarga de recoger y analizar toda la información a nivel entomológica en las zonas con potencialidad de brote epidémico (CDC, 2021).

2.4.Hipótesis

2.4.1. Hipótesis General

Mediante el desarrollo de un modelo de clasificación utilizando inteligencia artificial se logra clasificar los vectores de arbovirosis del Dengue, Zika y Chikungunya.

2.4.2. Hipótesis General Nula

Mediante el desarrollo de un modelo de clasificación utilizando inteligencia artificial no se logra clasificar los vectores de arbovirosis del Dengue, Zika y Chikungunya.

2.4.3. Hipótesis Específicos

HE1: La creación de una base de datos de imágenes de vectores de arbovirosis tipo culícido de la familia Aedes y Culex detallado por especies contribuirá en el desarrollo de modelos de inteligencia artificial.

HEN1: La creación de una base de datos de imágenes de vectores de arbovirosis tipo culícido de la familia Aedes y Culex detallado por especies no permitirá el desarrollo de modelos de inteligencia artificial.

HE2: La aplicación de técnicas de pre procesamiento de imágenes de vectores de arbovirosis tipo culícido de la familia Aedes y Culex favorecerá la calidad de los datos para la construcción del modelo de inteligencia artificial.

HEN2: La aplicación de técnicas de pre procesamiento de imágenes de vectores de arbovirosis tipo culícido de la familia Aedes y Culex no favorecerá la calidad de los datos para la construcción del modelo de inteligencia artificial.

HE3: La implementación de técnicas de Deep Learning VGG16, VGG19, ResNet50 y Vision Transformers permite una óptima clasificación de imágenes de arbovirosis tipo culícido de la familia Aedes y Culex.

HEN3: La implementación de técnicas de Deep Learning VGG16, VGG19, ResNet50 y Vision Transformers no mejora la clasificación de imágenes de arbovirosis tipo culícido de la familia Aedes y Culex.

HE4: La utilización de métricas de rendimiento favorece de forma positiva la evaluación de los modelos de deep learning para la clasificación de imágenes de arbovirosis tipo culícido de la familia Aedes y Culex.

HEN4: La utilización de métricas de rendimiento no evalúa los modelos de deep learning para la clasificación de imágenes de arbovirosis tipo culícido de la familia Aedes y Culex.

Capítulo III: Metodología

En este capítulo se desarrollará los aspectos metodológicos de la investigación como el enfoque, tipo y alcance. Así mismo se detallará la población, muestra y unidad de análisis. También se explicara los procedimientos de recolección de datos, sus determinadas fuentes y el cronograma de actividades.

3.1. Diseño de la Investigación

En esta sección de la presenta investigación se detallará el tipo de enfoque, tipo de diseño de investigación y el alcance que tendrá el trabajo.

3.1.1. Diseño

Según define (Marin y Novoa, 2014), en el diseño experimental se manipula deliberadamente al menos una variable independiente para analizar el efecto y la relación con una o más variables dependientes.

Bajo esta definición, la presente investigación utilizará un diseño experimental debido a que se contralará cada vector característico por cada clase de imágenes y se medirá su efecto en la variable binaria de clasificación, es decir los resultados de los modelos con valor de 1 corresponden a la clase Aedes y los resultados con valor de 0 corresponden a la clase Culex.

3.1.2. Alcance

Según (Ñaupas et al, 2014), el alcance explicativo está basado en problemas que buscan la relación causa y efecto, que explican el efecto de las variables independientes en la variable dependiente.

Bajo esta definición, la presente investigación aplicará un alcance explicativo debido a que si se encuentra un patrón de reconocimiento en la imagen del insecto se podrán

clasificar al insecto que causes las enfermedades de arbovirosis y conocer la densidad de su población.

3.1.3. Enfoque

Según (Ñaupas et al, 2014), el enfoque cuantitativo utiliza la recolección y análisis de datos a nivel estadístico para probar las hipótesis que se formulan en un investigación.

Bajo esta definición, la presente investigación aplicará un enfoque cuantitativo que se ve reflejado en la recolección de los datos que corresponden a las imágenes de cada especie de insecto y las mediciones estadísticas de cada modelo de inteligencia artificial que mostraran los niveles de precisión en la clasificación de estos tipos de imágenes.

3.1.4.Selección de la muestra

3.1.4.1.Población. Basado en el artículo de investigación relacionado a la clasificación de insectos mediante imágenes que tiene por título Mosquito Classification using Convolutional Neural Network with Data Augmentation y publicado en Springer donde se trabajó con 442 imágenes para el entrenamiento de los modelos (Mehenika et al., 2020).

Bajo esta referencia, la presente investigación trabajará con 4513 imágenes de insectos de tipo culícidos de la familia Aedes y Culex (2014 – actualidad). Para la familia Aedes se seleccionaron las subespecies Aedes Aegypti y Aedes Albopictus, que son especies que trasmiten enfermedades peligrosas y letales como el Dengue, Zika y Chikungunya. De la misma manera para la familia Culex se seleccionaron las sub especies Culex Pipiens, Culex Quinquefasciatus, Culex Tritaeniorhynchus y Culex Inatomii, que son especies de vectores que también trasmiten enfermedades pero no son letales para el ser humano.

3.1.4.2. Unidad de Análisis. La unidad de análisis es una imagen del insecto de tipo culícido que pertenece a la familia Aedes o Culex.

3.1.4.3.**Muestra.** Para esta investigación se utilizará un tipo de muestreo no probabilístico, del cual se utilizará toda la población de 4513 imágenes recolectas de 5 fuentes de información publicadas en sus respectivas páginas web que son validadas y confirmadas por expertos entomólogos. Posterior a ello se aplicará la técnica de "Data Augmentation" solo a los datos de entrenamiento que resulta en un total de 15344 imágenes de las dos especies. Es decir, el total de los datos serán particionados en dos grupos: entrenamiento (85%) y validación (15%).

Tabla 10

Descripción de los datos originales y los datos aumentados

Clases	Sub especies	BD1	BD2	BD3	BD4	BD5	Totales Original	Balanceo de Datos	Data Augmentation
Aedes	Aedes Aegypti	239	603	-	-	98	2432	2432	8268
	Aedes Albopictus	239	600	600	-	53			
Culex	Culex Pipiens	-	-	600	163	93	2081	2432	8268
	Culex Tritaeniorhynchus	-	-	594	-	53			
	Culex Quinquefasciatus	246	-	-	-	133			
	Culex Inatomii	-	-	199	-	-			
Totales		724	1203	1993	163	430	4513	4864	16536

Elaboración propia

3.1.5. Operacionalización de Variables

Tabla 11

Variables de la investigación

Definición de variables		
VARIABLE Y DEFINICIÓN	INDICADOR	FÓRMULA DE INDICADOR
VECTORES DE ARBOVIROSIS Son insectos que trasmiten enfermedades hacia los seres humanos	Proporción de vectores identificados del Dengue, Zika y Chikungunya	$\frac{Número\ de\ vectores\ indentificados\ como\ Aedes\ mediante\ imágenes}{Total\ de\ imágenes\ de\ vectores\ recolectados}$
INTELIGENCIA ARTIFICIAL Es un sistema basado en redes neuronales artificiales que buscan predecir o clasificar determinados objetivos.	Tiempo de inferencia	Total de tiempo que el modelo de IA realiza una predicción
	Huella de memoria	Cantidad total de memoria principal que el modelo de IA utiliza en su ejecución
MODELO DE CLASIFICACIÓN Es un modelo que está estructurado para clasificar un conjunto de datos etiquetados por clases.	Accuracy	$\frac{TP + TN}{TP + FP + FN + TN}$
	Precisión	$\frac{TP}{TP + FP}$
	Recall	$\frac{TP}{TP + FN}$
	F1-score	$2 * \frac{Presicion * Recall}{Presicion + Recall}$
	Especificidad	$\mathbf{Especificidad} = \frac{\boldsymbol{TN}}{\boldsymbol{TN + FN}}$
	AUC	$P(score(x^+) > score(x^-)$

Donde:

TP = True Positives

FP = False Positives

FN = False Negatives

TN = True Negatives

3.1.6. Técnicas de recolección de datos

En esta sección se detallará las características que deben tener las imágenes de los insectos de tipo culícidos tanto Aedes y Culex. Esto debido a que cada imagen tiene que ser de cuerpo entero y en distintas posiciones para entrenar los modelos con todas los detalles posibles tanto de la cabeza, tórax y alas del insecto. Estas partes son fundamentales por que determinaran a qué clase de pertenecen cada imagen.

Figura 39

Conjunto de imágenes de insectos de la familia Culex



Fuente: Elaboración propia

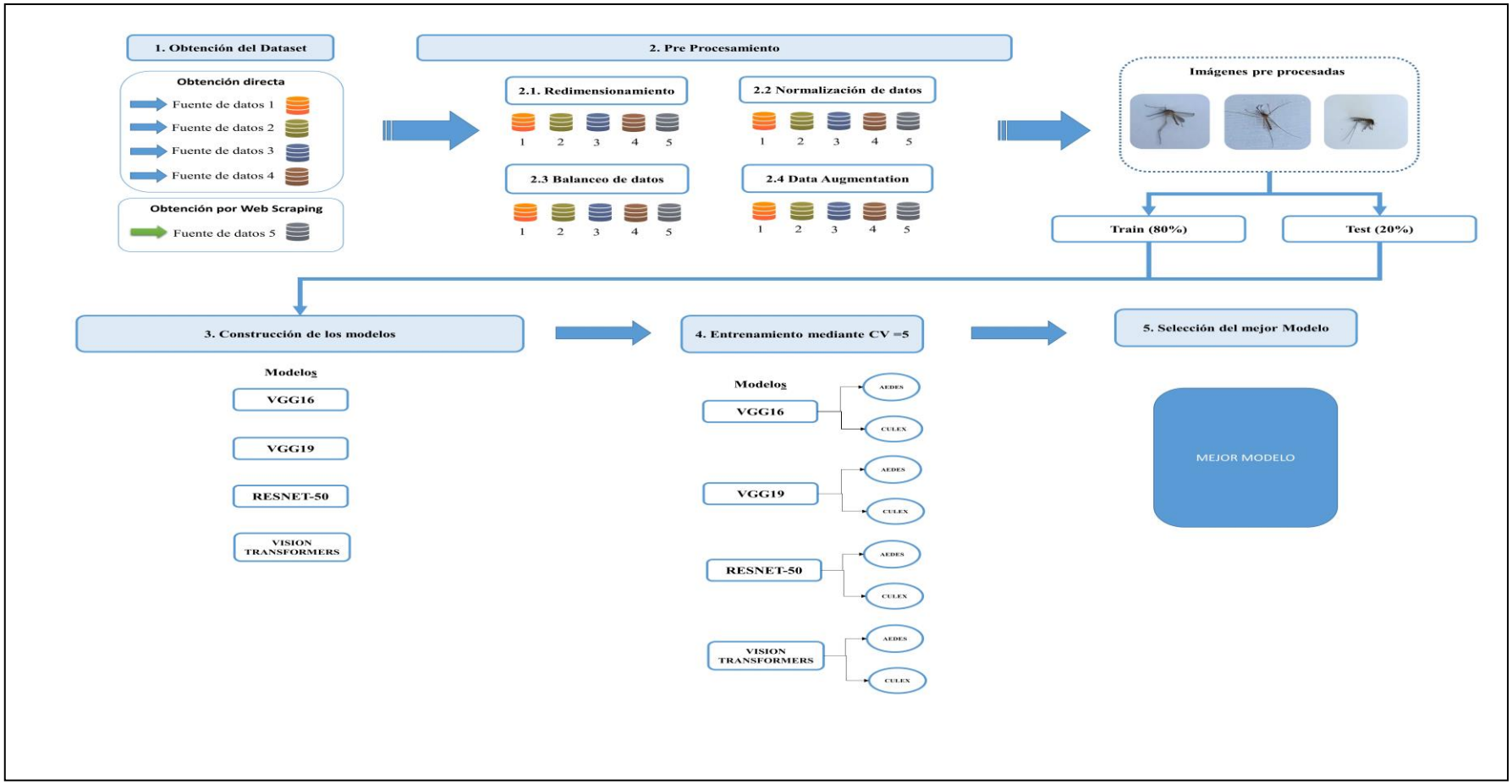
3.2.Metodología de la implementación de la solución

La implementación de una arquitectura completa de inteligencia artificial está compuesta por dos sub campos importantes denominados visión artificial y modelos de clasificación de Machine Learning o Deep Learning. Estos dos componentes trabajan juntos. Primero la visión artificial para la extracción de características de cada imagen y posteriormente en este caso los modelos de deep learning para la clasificación de las imágenes. En esta investigación se implementará una metodología que tomará como referencia la metodología de dos artículos que se mostraron en los antecedentes. La razón es debido a que en el primer artículo incorpora una serie de pasos donde implementan una etapa de pre procesamiento y la aplicación de técnicas de *Data Augmentation*. El segundo artículo

referenciado es debido a la aplicación de aprendizaje por modelos conjuntos para la búsqueda de mejores resultados a nivel de Accuracy. De esta manera la metodología implementa en esta investigación se muestra en la figura 40.

Figura 40

Metodología de la investigación



Fuente: Elaboración propia

3.2.1.Obtención del Dataset

En la presente etapa se detalla la adquisición de cada dataset que se utilizó para esta investigación. En la cual se explica a continuación las actividades que se realizaron la obtención de las 5 bases de datos de imágenes de Aedes y Culex.

3.2.1.1. **Búsqueda y selección de las primeras cuatro bases de datos.** En esta actividad se realizará la identificación y selección de imágenes de insectos de tipo culícido de la familia Aedes y Culex. Esta búsqueda se realizará solo en repositorios oficiales de investigación entomológica o en artículos científicos de entomología. En la cual se detallan las siguientes tareas que se realizaron por cada una de las cuatro primeras base de datos:

- Examinar y seleccionar las bases de datos correspondientes a imágenes de insectos de tipo culícido de la familia Aedes y Culex.
- Corroborar que la fuente de cada base de datos este previamente etiquetada por expertos entomólogos y evaluados en laboratorios.
- Descargar cada conjunto de base datos clasificados por especie Aedes y Culex.

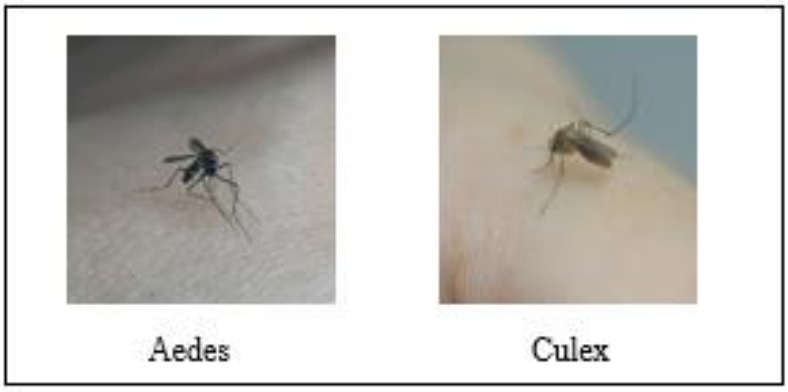
Estas tres tareas serán realizadas en cada una de las primeras 4 base de datos. Para la especie Aedes se seleccionaron 2 tipos de subespecie, Aedes Aegypti y Aedes Albopictus que son los principales responsables de la trasmisión del Dengue, Zika y Chikungunya. En la cual la primera subespecie se encuentra presente en el Perú. Para la especie Culex se seleccionaron las subespecies Culex Pippiens, Culex Tritaeniorhynchus, Culex Quinquefasciatus y Culex Inatomi. Que son sub especies que no trasmiten enfermedades letales hacia los seres humanos. De esta manera se seleccionó 4 base de datos que se detallan a continuación.

3.2.1.1.1. Primera fuente de datos. Conjunto de imágenes elaboradas en el laboratorio de la Universidad de Sabah de Malasia que se encuentra acreditado por la OMS.

- Detalle:
  - Aedes Aegypti: 239 imágenes.
  - Aedes Albopictus: 239 imágenes.
  - Culex Quinquefasciatus : 246 imágenes

Figura 41

Imágenes de las especies Aedes y Culex



Fuente: Universidad Sabah de Malasia

3.2.1.1.2. Segunda fuente de datos. Conjunto de imágenes elaboradas con insectos identificados por una cantidad de expertos de 30 entomólogos de la Universidad de Malasia.

- Detalle:
  - Aedes Aegypti: 603 imágenes
  - Aedes Albopictus: 600 imágenes

Figura 42

Imagen de la especie Aedes Aegypti y Aedes Albopictus



Fuente: Universidad de Malasia KDU Penang

3.2.1.1.3.Tercera fuente de datos. Conjunto de imágenes elaborado por la Universidad Nacional de Incheon de Coreo del Sur.

- Detalle:
  - Aedes Albopictus: 600 imágenes
  - Culex Pipiens: 600 imágenes
  - Culex. Tritaeniorhynchus: 594 imágenes
  - Culex. Inatomii: 199 imágenes

Figura 43

Imagen de la especie Aedes y Culex



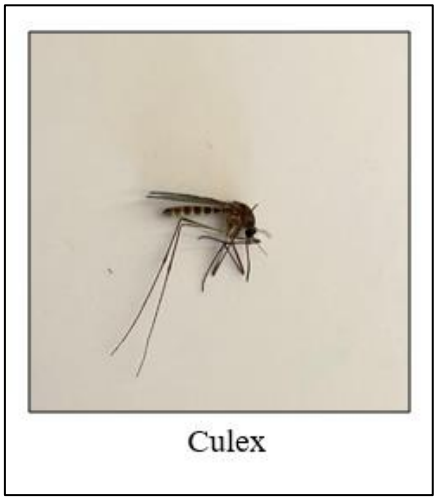
Fuente: Universidad Nacional de Icheon de Core del Sur

3.2.1.1.4.Cuarta fuente de datos. Conjunto de imágenes elaborado por el observatorio ciudadana en coordinación con diferentes instituciones públicas de investigación. Cada imagen fueron evaluadas por expertos entomólogos confirmando las etiquetas de las imágenes.

- Detalle:
  - Culex Pipens: 163 imágenes

Figura 44

Conjunto de imágenes de insectos de la familia Culex



Fuente: Observatorio Biodiverse

3.2.1.2. Extracción y construcción de la quinta base de datos. En esta segunda actividad se realizará la construcción de una base de datos a partir de técnicas de web scraping realizadas al sitio web International Barcode of life Project. En la cual se obtendrán imágenes clasificadas por especies Aedes y Culex. En la cual se detallan las siguientes tareas que se realizaran en esta etapa:

- Examinar y seleccionar las páginas que contengan las imágenes de tipo culícido de la familia Aedes y Culex.

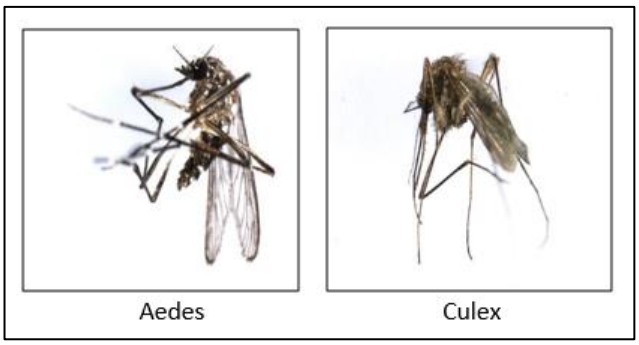
- Corroborar que la fuente de cada página de las imágenes este previamente etiquetada por expertos entomólogos y evaluados en laboratorios.
- Abrir el entorno de trabajo de Jupyter para ejecutar los bloques de código.
- Importar las librería de BeautifulSoup y librerías de sistema operativos para la gestión de archivos.
- Conectarse por solicitud “Get” al servidor del sitio web.
- Validar el estado de la conexión mediante el protocolo 200.
- Implementar los bloques de código necesario para automatizar la navegación y extracción de las imágenes que corresponden a las clases Aedes y Culex.
- Extraer de manera automatizada cada imagen por clase Aedes y Culex
- Etiquetar cada imagen y guardar en una carpeta específica.

El sitio web elegido corresponde al International Barcode of life Project, que es una organización especializada en proyectos de entomología, donde analizan una gran cantidad especies de insectos a nivel clínico mediante el ADN de cada especie con el objetivo de determinar la clasificación correcta de cada especie y sub especie de insectos. De esta manera luego de analizar los bloques de imágenes de las especies, se encontró las siguientes sub especies detalladas a continuación:

- Culex pipiens: 93 imágenes
- Culex Tritaeniorhynchus: 53 imágenes
- Culex Quinquefasciatus: 133 imágenes
- Aedes Aegypti: 98 imágenes
- Aedes Albopictus: 53 imágenes

Figura 45

Conjunto de imágenes de insectos de la familia Culex



Fuente: Elaboración Propia

A continuación se presenta el Pseudo código de la aplicación de la técnica de web scraping donde se especifican los pasos a seguir para la obtención de las imágenes clasificadas por especie.

Figura 46

Pseudocódigo del proceso de web scraping

```
#Algoritmo de Web Scraping
Variables:
url -> Es la dirección de la página de Barcode.
page -> Html de las páginas que contienen las imagenes de las especies.
page_secciones -> Son las secciones específicas donde se ubican la segmentacion la url de cada imagen.
title_de_especies -> Es una lista de los nombres de las sub especies de Aedes y Culex.
lista_nombres -> Es una lista de todos los nombres con código que corrpesonde a cada especie.
image_url -> La dirección directa de la imagen de por cada especie.

Inicio
|
|   Repetir
|   |   title_de_especies:
|   |    page <- request(url)
|   |    page_secciones <- BeautifulSoup(page)
|   |    Repetir
|   |    |    page_secciones:
|   |    |         si especies_nombres == title_de_especies[i]:
|   |    |              lista_nombres <- especies_nombres
|   |    Hasta no encontrar mas nombres de especies en la page_secciones
|   |
|   |    Repetir
|   |    |      lista_nombres:
|   |    |           image_url <- url + lista_nombre[i]
|   |    |           image <- request(image_url)
|   |    |           image_list <- image
|   |   Hasta terminar todos los nombres por especie.
|   |
|   Hasta terminar todos los titulos por sub especie de Aedes y culex.
|
Fin
```

3.2.2.Pre procesamiento

En esta fase se procedió a realizar las técnicas de pre procesamiento a cada base de datos detalladas en el punto anterior. Por consiguiente, se describe a continuación las técnicas realizadas por orden:

3.2.2.1. Redimensionamiento. Esta técnica consiste en estandarizar las dimensiones de las imágenes de las 5 base de datos para tener una sola medida de ancho y alto en todas las imágenes que es un requisito necesario para los entrenamientos de los modelos de deep learning. (Kumar et. al, 2023).

En base a los artículos descritos en los antecedentes de esta investigación se utilizará la dimensión de 224x 224.

Por consiguiente, para transformar las imágenes y convertirlas a una dimensión de 256 x 256 es necesario analizar la mejor técnica para realizar un óptimo proceso. Según el artículo de investigación publicado en el año 2013 llamado "A Comparison Study On Different Interpolation Methods Based On Satellite" desarrollado por la facultad de Ingeniería y Tecnología de la Universidad de Viswajyothi en la India cuyos autores son Jency Titus y Sebastian George, indicaron en sus conclusiones que luego de varias pruebas de entrenamiento y comparación entre varias otras técnicas de cambio de dimensiones, la "interpolación bicúbica" genera mejores resultados de calidad de imagen en un proceso de mayor tiempo.

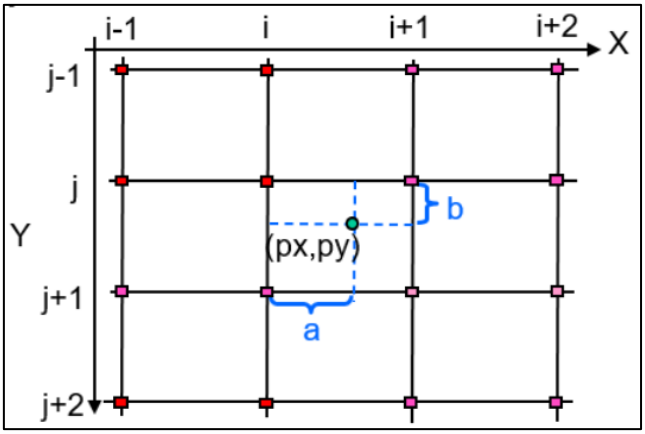
La técnica de interpolación bicúbica consiste en un algoritmo que selecciona lo 16 punto pixeles más cercanos para la estimación del pixel desconocido. (Titus y George, 2013, p.2).

Este artículo de investigación concluyó que "El método de interpolación bicúbica es mejor que los modelos de vecino más cercano y el método de interpolación bilineal. Estos dos modelos tienen menos tiempo de procesamiento pero la interpolación bicúbica produce una mejor salida". (Titus y George, 2013, p.4).

Con esta referencia solida se implementará la interpolación bicúbica donde su cálculo correspondiente se muestra en la figura N° 47.

Figura 47

Cálculo del redimensionamiento a partir de un punto pixel



Fuente: García. 2018. Procesamiento de Imágenes. Universidad de Murcia. p. 17

Donde:

Sea p el punto que se desea interpolar:

$$A'(px, py) = \sum_{n=-1..2} \times \sum_{n=-1..2} \times A((i+n),(j+m)) \times P(n-a) \times P(b-m)$$

Siendo:

$$P(k) = \frac{1}{6} \times (C(k+2)^3 - 4C(k+1)^3 + 6C(k)^3 - 4C(k-1)^3)$$

$$C(k) = \max(0, k)$$

La implementación de esta técnica se realizará mediante la librería PIL importando la función de “resize” donde se especificará la técnica bicúbica y se aplicará la conversión a las 5 bases de datos. Las tareas realizadas se detallan a continuación

- Abrir el entorno de trabajo de Google Colab para ejecutar los bloques de código en Python.
- Importar la librería “OS” para la carga de las imágenes almacenadas en drive”
- Importar la librería de “PIL” para el tratamiento de las imágenes de cambios de dimensiones.

- De la libería “PIL” se importará el módulo “image” donde se obtendrá la función “resize” que permitirá el cambio de dimensiones por cada imagen.
- Mediante un bucle “for” se procedió a cargar las imágenes y con la función “resize” se cambió el tamaño de cada imagen a 224x224.

De esta manera se obtendrá el primer entregable que son las imágenes estandarizadas con las dimensiones de 224 X 224.

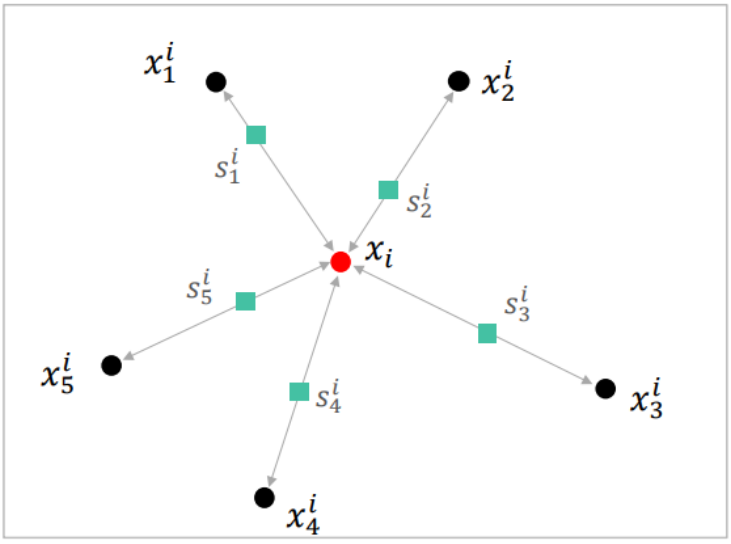
3.2.2.2. Balanceo de datos. Esta técnica consiste en nivelar las cantidades de imágenes por tipo de clase y mejorar el proceso de entrenamiento en los modelos de inteligencia artificial. Existen varios métodos de balanceo de datos, tanto a nivel de sobre muestreo como de sub muestreo. En la cual las técnicas más importantes son el SMOTE, ADASYN, ROSE entre otros. (Brownlee, 2020).

Según el artículo de investigación realizado por la Universidad de Ingeniería de Iran llamado “Effective Class-Imbalance Learning Based on SMOTE and Convolutional Neural Networks” que tiene como autores a Javad Hassannataj Joloudari , Abdolreza Marefat, Mohammad Ali Nematollahi, Solomon Sunday Oyelere y Sadiq Hussain. Concluye que la técnica SMOTE proporciona mejores resultados en comparación a otros tipos de técnicas, debido a su estructura y complejidad en la generación de datos sintéticos a partir de los originales que impactan positivamente en un mejor rendimiento en el entrenamiento de los modelos de inteligencia artificial.

El cálculo para generar datos sintéticos a partir de los originales está basado en instancias, es decir “Si tiene un objeto X_i y se selecciona sus 5 vecinos más cercanos $\{X^i_1, X^i_2, X^i_3, X^i_4, X^i_5\}$ en una primera iteración de un proceso SMOTE a nivel de sobre muestreo es posible la creación de 5 nuevas instancias $\{S^i_1, S^i_2, S^i_3, S^i_4, S^i_5\}$”. (García, 2021).

Figura 48

Cálculo del balanceo de datos - SMOTE



Fuente: García .2021. Comparativo de técnicas de balanceo de datos. p. 26

Donde:

X_{min} → Representa el conjunto de instancias de la clase minoritaria

S → Representa el conjunto de instancias sintéticas generadas

k → Representa el conjunto de instancias de la clase minoritaria

$$S_j^i = X_i + \lambda . (X_j^i - X_i), J = 1, \dots , k$$

Siendo:

$$\lambda \in [0, 1]$$

En base a esta referencia, para el presente proyecto de investigación se implementará la técnica de SMOTE a nivel de sobre muestreo que implicar generar y nivelar el número de imágenes de la clase minoritaria.

Tabla 12

Variables de la investigación

Antes de la técnica de balanceo de datos		Después de la técnica de balanceo de datos	
Clase	Cantidad	Clase	Cantidad
Aedes	2432	Aedes	2432
Culex	2081	Culex	2432
Total	4513	Total	4864

Fuente: Elaboración propia

Los pasos detallados a realizar para el desarrollo de esta técnica de balanceo de datos para esta investigación son los siguientes:

- Abrir el entorno de trabajo en Google Colab para ejecutar los bloques de código en Python.
- Importar la librería “OS” para cargar las imágenes almacenadas en drive.
- Importar la librería “imblearn.over_sampling” para obtener el módulo “SMOTE”.
- Mediante la librería “OS” cargar las etiquetas que correspondientes a las imágenes de Aedes y Culex y guardar en dos variables temporales de tipo lista con los valores de 0 (Culex) y 1 (Aedes).
- Mediante la función “SMOTE” generar una instancia en código.
- Mediante un bucle “for” cargar las imágenes en dos arrays distintos que correspondan a los tipos Aedes y Culex respectivamente.
- Aplicar el sobre muestreo para generar datos sintéticos de la clase minoritaria e igualar a la clase mayoritaria.
- Cargar en una variable temporal de tipo lista los “*arrays*” que corresponden a cada imagen de las clases Aedes y Culex.
- Guardar las imágenes balanceadas en drive para el siguiente pase del pre procesamiento.

3.2.2.3.Aumento de datos. Esta técnica consiste en aumentar el número de imágenes a partir de cambios en el eje estructural y la forma de las imágenes originales con el objetivo de tener un mayor número de muestras en el entrenamiento y mejorar los resultados en los modelos de inteligencia artificial debido a que estas estructuras requieren un mayor número de datos para obtener una precisión mejor. (Aboul y Ashraf, 2020).

Esta técnica implica hacer transformaciones en las formas de las imágenes originales para tener un mayor número de imágenes a partir de una sola imagen original. En la cual se detallan los cambios aplicados al conjunto de datos de esta investigación:

3.2.2.3.1.Shear range. Esta transformación consiste en aplicar una inclinación a las imágenes originales donde su cálculo se realiza de la siguiente manera:

$$x' = x + shear_range * y$$

$$y' = y$$

Donde:

(x,y) representan a las coordenadas punto de una imagen y sus correspondientes (x',y') son las trasformación de inclinación generadas en la imagen

3.2.2.3.2. *Zoom range*. Esta transformación consiste en aumentar el acercamiento en determinadas zonas de la imagen en la cual su cálculo se realiza de la siguiente manera:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} Xscale & 0 \\ 0 & Yscale \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Donde:

(x,y) representan a las coordenadas punto de una imagen y sus correspondientes (x',y') son las trasformación de inclinación generadas en la imagen

3.2.2.3.3. Horizontal Flip. Esta transformación consiste en girar la imagen en el eje horizontal, en la cual su cálculo se realiza de la siguiente manera:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Donde:

(x,y) representan a las coordenadas punto de una imagen y sus correspondientes (x',y') son las trasformación de inclinación generadas en la imagen

3.2.2.3.4.Vertical Flip. Esta transformación consiste en girar la imagen en el eje horizontal, en la cual su cálculo se realiza de la siguiente manera:

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

Donde:

(x,y) representan a las coordenadas punto de una imagen y sus correspondientes (x',y') son las trasformación de inclinación generadas en la imagen

Para esta investigación se detalla a continuación los pasos a seguir para la aplicación del aumento de datos considerando las 4 trasformaciones explicadas anteriormente.

- Abrir el entorno de trabajo de Google Colab para ejecutar los bloques de código en Python.
- Importar la librería de "Keras" para obtener la función de "ImageDataGenerator".
- Importar la librería "OS" para cargar las imágenes de entrenamiento y prueba en una variable temporal de tipo lista.

- Mediante la función “ImageDataGenerator” definir las 4 variaciones en la que se aumentaran los datos originales de los datos de entrenamiento.
- Incorporar las instancias de “ImageDataGenerator” de las variaciones establecidas en el generador de entrenamiento de la función “flow_from_directory” que son los elementos necesarios para el entrenamiento de los modelos de inteligencia artificial.

3.2.2.4. Normalización de datos

Por ultimo en esta etapa de procesamiento se realiza la normalización de los datos que consiste en convertir la distribución de los valores de una imagen de rango de 0 a 255 en valores de 0 y 1 con el objetivo de valores de rangos estandarizados y que los modelos de inteligencia artificial pueden ser entrenados con mayor eficiencia. (Aboul y Ashraf, 2020).

En base al artículo de investigación realizado por el departamento de computación e ingeniería de software de Canada llamado “Realistic Image Normalization for Multi-Domain Segmentation” que tiene por autores a Pierre-Luc Delislea , Benoit Anctil-Robitaillea, Christian Desrosiersa , Herve Lombaerta, concluyeron que la técnica de normalización en los datos de entrada para el entrenamiento de modelos de inteligencia artificial de capas profundas mejoran notablemente los resultados.

El cálculo para normalizar las imágenes del conjunto de datos es el siguiente:

$$x_{nom} = \frac{x}{255}$$

Donde:

X: Representa el valor del punto pixel de la imagen original

X_{norm}: Represente el valor del pixel normalizado

Para esta investigación se detalla a continuación los pasos a seguir para la aplicación de la normalización de las imágenes de las 5 bases de datos, en la que a continuidad de la técnica de aumento de datos se realiza lo siguiente:

- En el mismo entorno de trabajo de Google Colab mediante "ImageDataGenerator" agregar el parámetro de rescale que normalizará a las imágenes a valores entre 0 y 1.
- Incorporar las instancias de "ImageDataGenerator" de las variaciones establecidas con la normalización implementada en el generador de entrenamiento de la función "flow_from_directory" que son los elementos necesarios para el entrenamiento de los modelos de inteligencia artificial.

3.2.3.Aplicación y entrenamiento de los modelos de IA

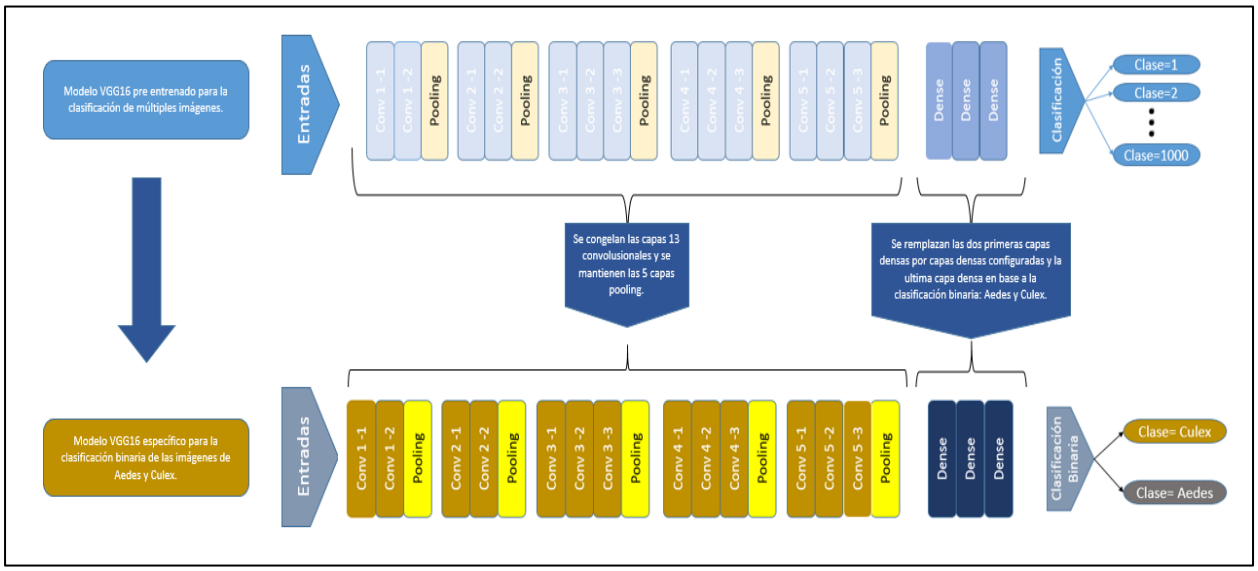
En esta segunda etapa de la metodología de esta investigación se describirá como se implementará cada modelo de inteligencia artificial y su correspondiente estructura para el entrenamiento y test con el total de imágenes etiquetados por Aedes y Culex.

Como primer paso en esta etapa del enteramiento de los modelos de deep learning, se realiza la división de los datos en entrenamiento y prueba, aplicando un 85% para el entrenamiento y el 15% para la prueba. Los datos se guardaron en una carpeta aparte divida por entrenamiento y prueba. En cada carpeta está divida por cada etiqueta de Aedes y Culex.

3.2.3.1. Modelo VGG16. La estructura base del modelo VGG16 está configurado por defecto para la clasificación de 1000 clases de imágenes que corresponden al entrenamiento de la base de ImageNet que contiene un aproximado de 14 millones de imágenes. En la que cada peso de esa red de 16 capas profundas contiene la información de caracteres de identificación de distintos tipos de figuras y rasgos. De esta manera para esta investigación se implementará el aprendizaje por transferencia cargando los pesos pre entrenados, congelando las 13 capas convolusionales, remplazando las dos primeras capas densas y configurando la última capa de salida para obtener la clasificación binaria de las etiquetas Aedes y Culex.

Figura 48

Implementación del modelo VGG16 por Transferer Learning.

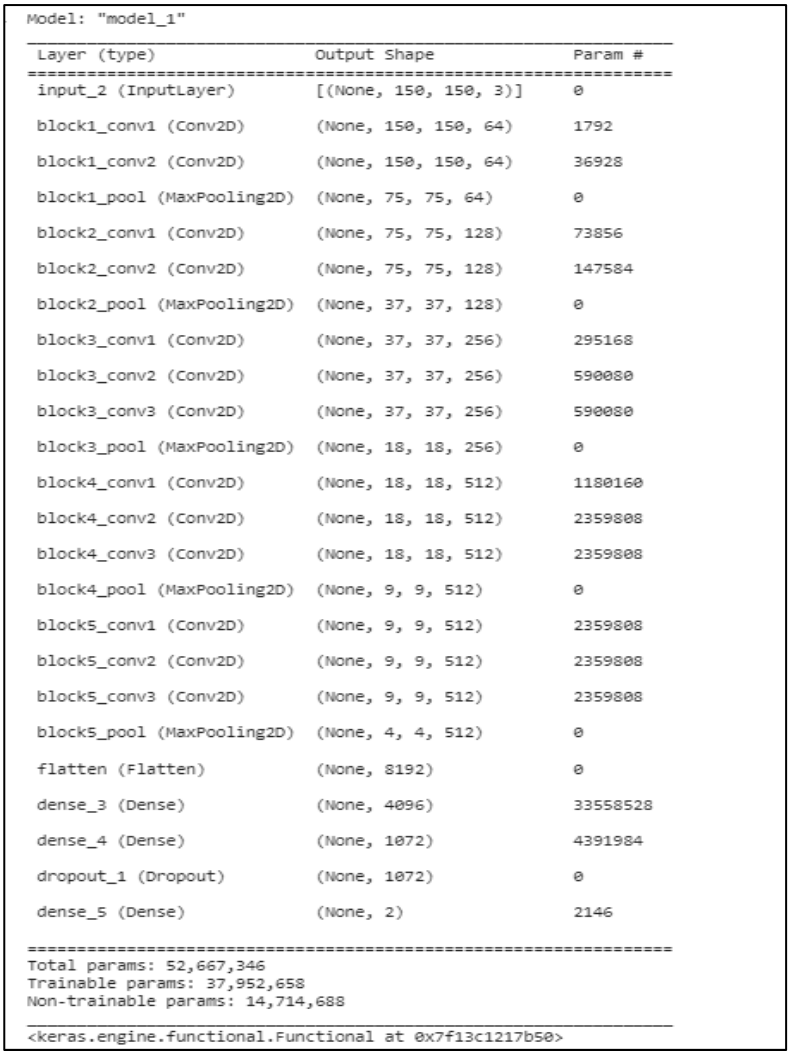


Fuente: Elaboración propia

A continuación se muestra la estructura de implementación del modelo vgg16:

Figura 49

Estructura del modelo VGG16 por Transferer Learning.



```
Model: "model_1"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 input_2 (InputLayer)        [(None, 150, 150, 3)]     0

 block1_conv1 (Conv2D)       (None, 150, 150, 64)      1792

 block1_conv2 (Conv2D)       (None, 150, 150, 64)      36928

 block1_pool (MaxPooling2D)  (None, 75, 75, 64)        0

 block2_conv1 (Conv2D)       (None, 75, 75, 128)       73856

 block2_conv2 (Conv2D)       (None, 75, 75, 128)       147584

 block2_pool (MaxPooling2D)  (None, 37, 37, 128)       0

 block3_conv1 (Conv2D)       (None, 37, 37, 256)       295168

 block3_conv2 (Conv2D)       (None, 37, 37, 256)       590080

 block3_conv3 (Conv2D)       (None, 37, 37, 256)       590080

 block3_pool (MaxPooling2D)  (None, 18, 18, 256)       0

 block4_conv1 (Conv2D)       (None, 18, 18, 512)       1180160

 block4_conv2 (Conv2D)       (None, 18, 18, 512)       2359808

 block4_conv3 (Conv2D)       (None, 18, 18, 512)       2359808

 block4_pool (MaxPooling2D)  (None, 9, 9, 512)         0

 block5_conv1 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_conv2 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_conv3 (Conv2D)       (None, 9, 9, 512)         2359808

 block5_pool (MaxPooling2D)  (None, 4, 4, 512)         0

 flatten (Flatten)           (None, 8192)              0

 dense_3 (Dense)             (None, 4096)              33558528

 dense_4 (Dense)             (None, 1072)              4391984

 dropout_1 (Dropout)         (None, 1072)              0

 dense_5 (Dense)             (None, 2)                 2146

=================================================================
Total params: 52,667,346
Trainable params: 37,952,658
Non-trainable params: 14,714,688
_________________________________________________________________
<keras.engine.functional.Functional at 0x7f13c1217b50>
```

Fuente: Elaboración propia

A continuación se detalla las tareas realizadas en la preparación y aplicación de este modelo:

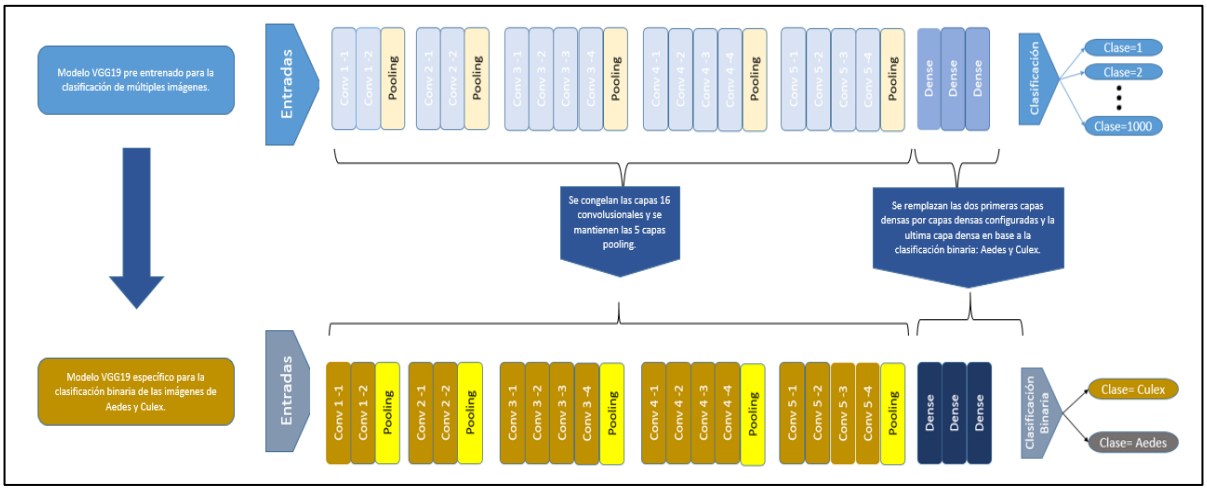
- Abrir el entorno de trabajo en Google Colab para ejecutar los bloques de código en Python.
- Importar la librería “OS” para cargar las imágenes almacenadas en drive.
- Se realiza la selección de los mejores hiperparámetros en la cual se importa de la librería de “sklearn.model_selection” con su módulo “RandomizedSearchCV”.

- Se carga en 4 variables temporales de tipo “lista” las etiquetas separadas de Aedes y Culex por Train y Test.
- Se carga en 4 variables temporales de tipo “lista” los valores de las imágenes separadas de Aedes y Culex por Train y Test.
- Se convierte a valores de 0 y 1, las 4 variables temporal de las etiquetas de Culex y Aedes. Posteriormente se guarda el total de valores 0 en una sola variable lista y lo mismo para los valores 1.
- Se convierte las varables de tipo lista que almancenan los valores de las imágenes en variables de tipo “arrays”.
- Se agregan en 4 variables los valores de hiperparáemtros para el tuneo:
    - Learning Rate = [0.01,0.001,0.0001]
    - Dropout = [0.2, 0.3, 0.5]
    - Batch Size = [8, 16, 32]
    - Épocas = [60, 80, 120]
- Se implemnata la carga de los imperparametros en la instancia de “RandomizedSearchCV” configuando adicionalmente un Cross-Validation de 10.
- Se guarda el modelo entrenado con sus mejores hiperparámetros.

3.2.3.2. Modelo VGG19. La estructura base del modelo VGG19 está configurado por defecto para la clasificación de 1000 clases de imágenes que corresponden al entrenamiento de la base de ImageNet que contiene un aproximado de 14 millones de imágenes y que es una variante del modelo VGG16. En la que cada peso de esa red de 19 capas profundas contiene la información de caracteres de identificación de distintos tipos de figuras y rasgos. De esta manera para esta investigación se implementará el aprendizaje por transferencia cargando los pesos pre entrenados, congelando las 16 capas convolusionales, remplazando las dos primeras capas densas y configurando la última capa de salida para obtener la clasificación binaria de las etiquetas Aedes y Culex.

Figura 50

Implementación del modelo VGG19 por Transferer Learning.

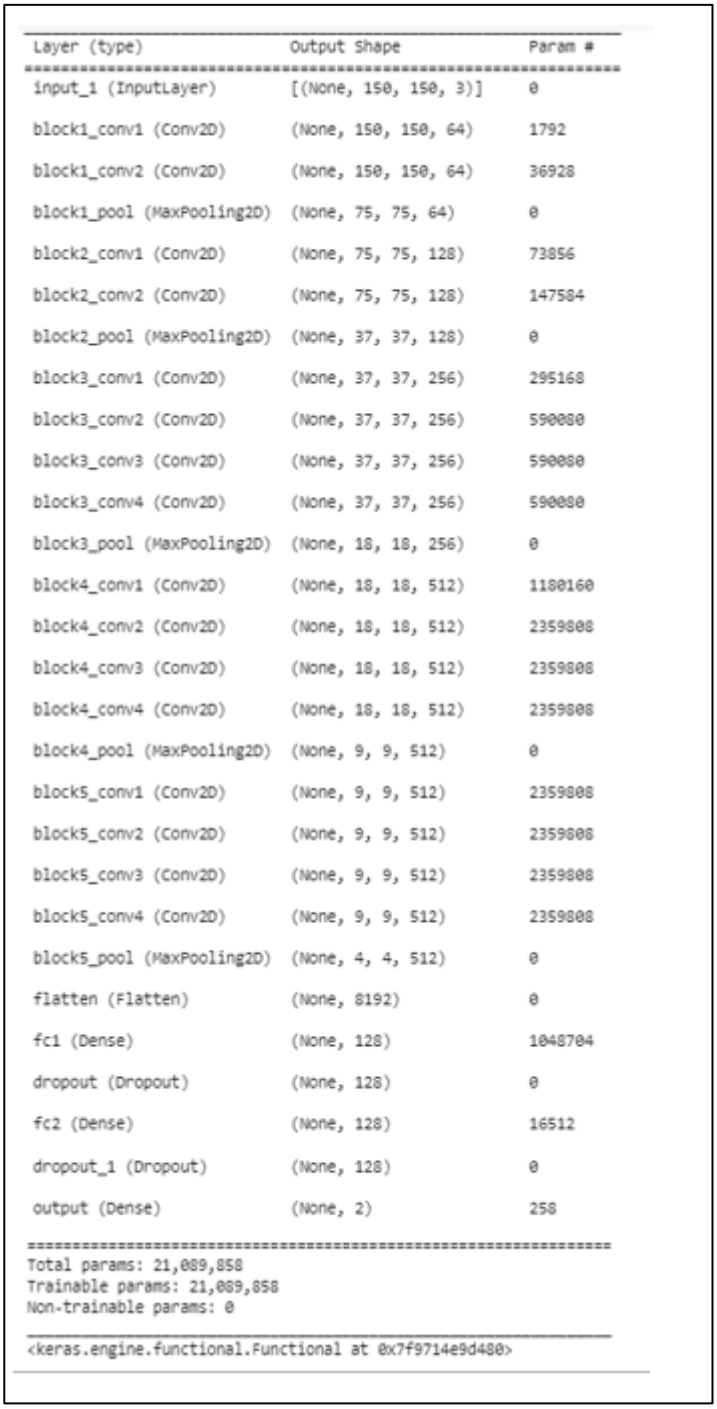


Fuente: Elaboración propia

A continuación se muestra la estructura de implementación del modelo vgg19:

Figura 51

Estructura del modelo VGG19 por Transferer Learning.



Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 150, 150, 3)]	0
block1_conv1 (Conv2D)	(None, 150, 150, 64)	1792
block1_conv2 (Conv2D)	(None, 150, 150, 64)	36928
block1_pool (MaxPooling2D)	(None, 75, 75, 64)	0
block2_conv1 (Conv2D)	(None, 75, 75, 128)	73856
block2_conv2 (Conv2D)	(None, 75, 75, 128)	147584
block2_pool (MaxPooling2D)	(None, 37, 37, 128)	0
block3_conv1 (Conv2D)	(None, 37, 37, 256)	295168
block3_conv2 (Conv2D)	(None, 37, 37, 256)	590080
block3_conv3 (Conv2D)	(None, 37, 37, 256)	590080
block3_conv4 (Conv2D)	(None, 37, 37, 256)	590080
block3_pool (MaxPooling2D)	(None, 18, 18, 256)	0
block4_conv1 (Conv2D)	(None, 18, 18, 512)	1180160
block4_conv2 (Conv2D)	(None, 18, 18, 512)	2359808
block4_conv3 (Conv2D)	(None, 18, 18, 512)	2359808
block4_conv4 (Conv2D)	(None, 18, 18, 512)	2359808
block4_pool (MaxPooling2D)	(None, 9, 9, 512)	0
block5_conv1 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv2 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv3 (Conv2D)	(None, 9, 9, 512)	2359808
block5_conv4 (Conv2D)	(None, 9, 9, 512)	2359808
block5_pool (MaxPooling2D)	(None, 4, 4, 512)	0
flatten (Flatten)	(None, 8192)	0
fc1 (Dense)	(None, 128)	1048704
dropout (Dropout)	(None, 128)	0
fc2 (Dense)	(None, 128)	16512
dropout_1 (Dropout)	(None, 128)	0
output (Dense)	(None, 2)	258

Total params: 21,089,858
Trainable params: 21,089,858
Non-trainable params: 0

<keras.engine.functional.Functional at 0x7f9714e9d480>

Fuente: Elaboración propia

A continuación se detalla las tareas realizadas en la preparación y aplicación de este modelo:

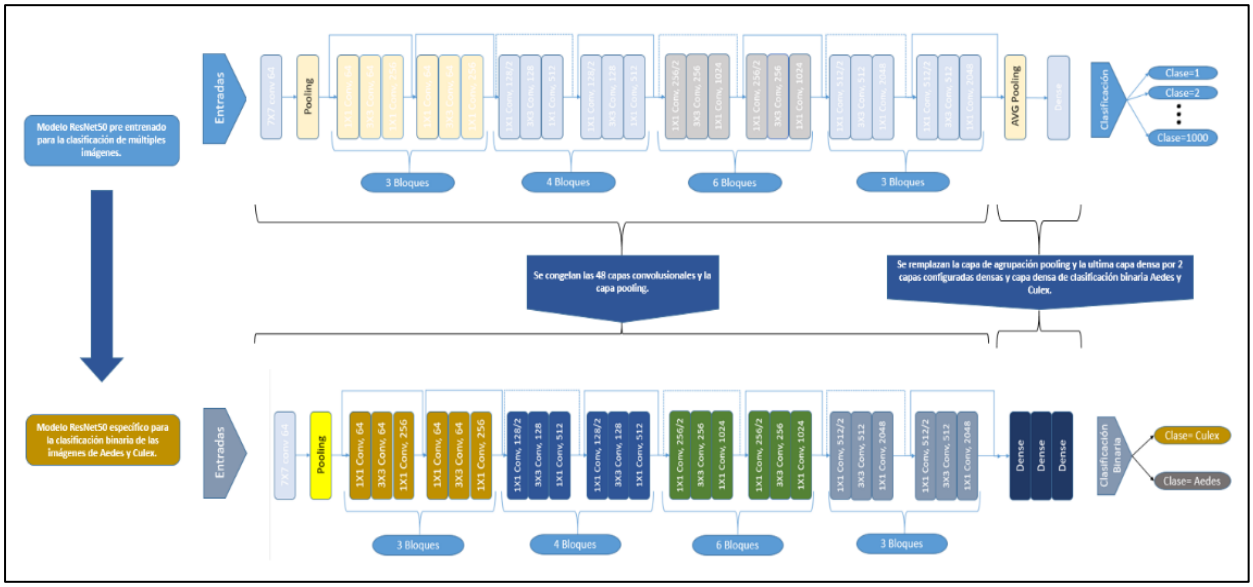
- Abrir el entorno de trabajo en Google Colab para ejecutar los bloques de código en Python.
- Importar la librería “OS” para cargar las imágenes almacenadas en drive.
- Se realiza la selección de los mejores hiperparámetros en la cual se importa de la librería de “sklearn.model_selection” con su módulo “RandomizedSearchCV”.
- Se carga en 4 variables temporales de tipo “lista” las etiquetas separadas de Aedes y Culex por Train y Test.
- Se carga en 4 variables temporales de tipo “lista” los valores de las imágenes separadas de Aedes y Culex por Train y Test.
- Se convierte a valores de 0 y 1, las 4 variables temporal de las etiquetas de Culex y Aedes. Posteriormente se guarda el total de valores 0 en una sola variable lista y lo mismo para los valores 1.
- Se convierte las variables de tipo lista que almacenan los valores de las imágenes en variables de tipo “arrays”.
- Se agregan en 4 variables los valores de hiperparámetros para el tuneo:
  - Learning Rate = [0.01,0.001,0.0001]
  - Dropout = [0.2, 0.3, 0.5]
  - Batch Size = [8, 16, 32]
  - Épocas = [60, 80, 120]
- Se implementa la carga de los hiperparámetros en la instancia de “RandomizedSearchCV” configuando adicionalmente un Cross-Validation de 10.

- Se guarda el modelo entrenado con sus mejores hiperparámetros.

3.2.3.3. **Modelo ResNet50.** La estructura base del modelo ResNet50 está configurado por defecto para la clasificación de 1000 clases de imágenes que corresponden al entrenamiento de la base de ImageNet que contiene un aproximado de 14 millones de imágenes y que tiene la particularidad de utilizar bloques residuales para mejorar el problema de la gradiente. En la que cada peso de esa red de 48 capas profundas contiene la información de caracteres de identificación de distintos tipos de figuras y rasgos. De esta manera para esta investigación se implementará el aprendizaje por transferencia cargando los pesos pre entrenados, congelando las 48 capas convolusionales, remplazando las dos primeras capas densas y configurando la última capa de salida para obtener la clasificación binaria de las etiquetas Aedes y Culex.

Figura 52

Implementación del modelo ResNet50 por Transferer Learning.



Fuente: Elaboración Propia

Debido a la cantidad de números de capas que contiene el modelo de resnet50 se comparte el siguiente link para la visualización de la estructura de este modelo implementado.

A continuación se detalla las tareas realizadas en la preparación y aplicación de este modelo:

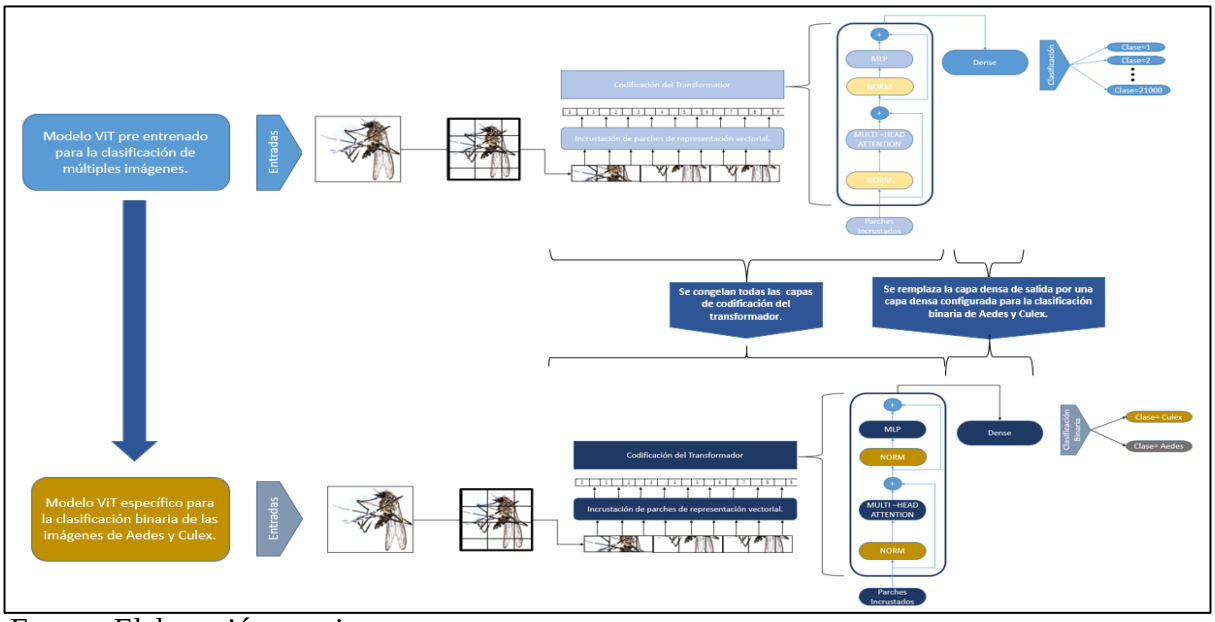
- Abrir el entorno de trabajo en Google Colab para ejecutar los bloques de código en Python.
- Importar la librería “OS” para cargar las imágenes almacenadas en drive.
- Se realiza la selección de los mejores hiperparámetros en la cual se importa de la librería de “sklearn.model_selection” con su módulo “RandomizedSearchCV”.
- Se carga en 4 variables temporales de tipo “lista” las etiquetas separadas de Aedes y Culex por Train y Test.
- Se carga en 4 variables temporales de tipo “lista” los valores de las imágenes separadas de Aedes y Culex por Train y Test.
- Se convierte a valores de 0 y 1, las 4 variables temporal de las etiquetas de Culex y Aedes. Posteriormente se guarda el total de valores 0 en una sola variable lista y lo mismo para los valores 1.
- Se convierte las variables de tipo lista que almacenan los valores de las imágenes en variables de tipo “arrays”.
- Se agregan en 4 variables los valores de hiperparámetros para el tuneo:
  - Learning Rate = [0.01,0.001,0.0001]
  - Dropout = [0.2, 0.3, 0.5]
  - Batch Size = [8, 16, 32]
  - Épocas = [60, 80, 120]

- Se implementa la carga de los hiperparámetros en la instancia de "RandomizedSearchCV" configuando adicionalmente un Cross-Validation de 10.
- Se guarda el modelo entrenado con sus mejores hiperparámetros.

3.2.3.4.Modelo Visión Transformers. La estructura base del modelo Vision Transformers está configurado por defecto para la clasificación de 21000 clases de imágenes que corresponden al entrenamiento de la base de ImageNet que contiene un aproximado de 14 millones de imágenes. En la que cada peso de esa red de codificadores y capas de atención contiene la información de caracteres de identificación de distintos tipos de figuras y rasgos. De esta manera para esta investigación se implementará el aprendizaje por transferencia cargando los pesos pre entrenados, congelando toda la estructura del codificador, remplazando la última capa densa por una capa configurada para la salida de dos tipo de clases (Aedes y Culex).

Figura 53

Implementación del modelo Vision Transformers por Transferer Learning.



Fuente: Elaboración propia

Debido a la cantidad de capas que contiene el modelo de ViT se comparte el siguiente link para la visualización de la estructura de este modelo implementado.

A continuación se detalla las tareas realizadas en la preparación y aplicación de este modelo:

- Abrir el entorno de trabajo en Google Colab para ejecutar los bloques de código en Python.
- Importar la librería “OS” para cargar las imágenes almacenadas en drive.
- Se realiza la selección de los mejores hiperparámetros en la cual se importa de la librería de “sklearn.model_selection” con su módulo “RandomizedSearchCV”.
- Se carga en 4 variables temporales de tipo “lista” las etiquetas separadas de Aedes y Culex por Train y Test.
- Se carga en 4 variables temporales de tipo “lista” los valores de las imágenes separadas de Aedes y Culex por Train y Test.
- Se convierte a valores de 0 y 1, las 4 variables temporal de las etiquetas de Culex y Aedes. Posteriormente se guarda el total de valores 0 en una sola variable lista y lo mismo para los valores 1.
- Se convierte las variables de tipo lista que almacenan los valores de las imágenes en variables de tipo “arrays”.
- Se agregan en 4 variables los valores de hiperparámetros para el tuneo:
  - Learning Rate = [0.01,0.001,0.0001]
  - Dropout = [0.2, 0.3, 0.5]
  - Batch Size = [8, 16, 32]
  - Épocas = [60, 80, 120]

- Se implementa la carga de los hiperparámetros en la instancia de "RandomizedSearchCV" configuando adicionalmente un Cross-Validation de 10.
- Se guarda el modelo entrenado con sus mejores hiperparámetros.

Luego de realizar la implementación de cada modelo se procederá a configurar las capas de pesos de extracción de características en cada modelo para evaluar los vectores características que utiliza cada modelo y reflejarlos mediante el módulo Grid-CAM visualizar las regiones de interés más importantes en la clasificación de estos tipos de imágenes.

3.3.Metodología para la medición de resultados de la implementación

Para la medición de resultados en cada modelo de aprendizaje profundo se utilizará los siguientes indicadores de rendimiento:

3.3.1.Accuracy

Consiste en evaluar el porcentaje de predicciones correctas realizadas por el modelo y que se obtiene con la fórmula siguiente:

$$\mathbf{Accuracy} = \frac{\mathbf{TP} + \mathbf{TN}}{\mathbf{TP} + \mathbf{TN} + \mathbf{FP} + \mathbf{FN}}$$

Donde

TP = *Verdaderos positivos,* TN = *Verdaderos negativos,* FP = *Falsos positivos y* FN = *Falsos negativos*

3.3.2.Recall

Muestra el porcentaje de predicciones que son acertadas correctamente de una determinada clase y se calcula de la siguiente manera:

$$\mathbf{Recall} = \frac{\mathbf{TP}}{\mathbf{TP} + \mathbf{FN}}$$

Donde

TP = *Verdaderos positivos y* FN = *Falsos negativos*

3.3.3.Precisión

Este indicador indica el porcentaje de valores acertados que son positivos con respecto al total de valores positivos y se calcula de la siguiente manera:

$$\mathbf{Presicion} = \frac{\mathbf{TP}}{\mathbf{TP} + \mathbf{FP}}$$

Donde

TP = *Verdaderos positivos y* FP = *Falsos positivos*

3.3.4.F1-Score

Este indicador que combina la precisión y el recall, mostrando la media armónica de ambos indicadores.

$$\mathbf{2} * \frac{\boldsymbol{Presicion} * \boldsymbol{Recall}}{\boldsymbol{Presicion} + \boldsymbol{Recall}}$$

3.3.5.Especificidad

Esta métrica refleja el porcentaje de valores acertados que son negativos con respecto al total de valores negativos y se calcula de la siguiente manera:

$$\textbf{Especificidad} = \frac{\textbf{TN}}{\textbf{TN} + \textbf{FN}}$$

Donde

TN = *Verdaderos negativos y* FN = *Falsos negativos*

3.3.6.AUC

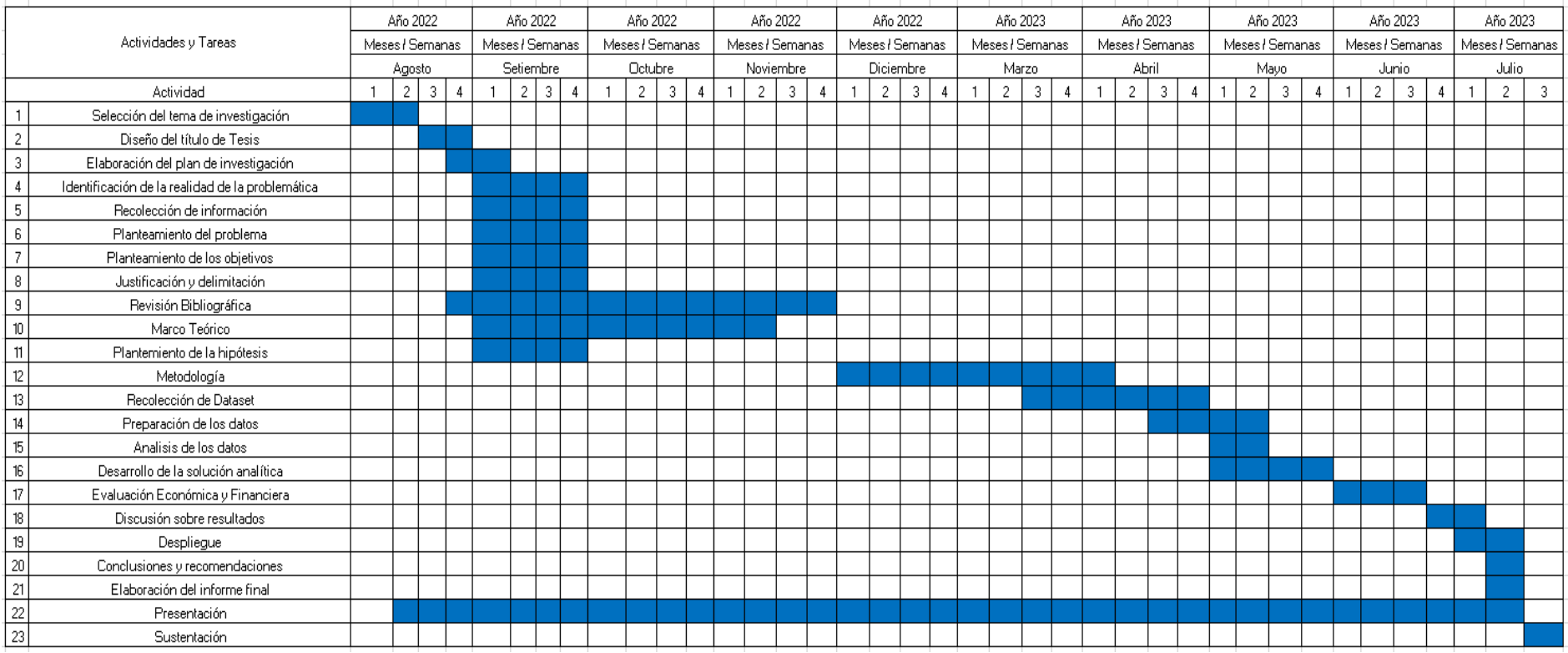
Este indicador muestra una curva gráfica que refleja el rendimiento general de un modelo de clasificación binaria mediante el nivel de clasificación de cada clase. Sus valores se encuentran entre 0 y 1. Siendo un modelo con altas características de clasificación si tiene AUC mayor a 0.85.

3.4.Cronograma de actividades y presupuesto

3.4.1.Cronograma

Figura 54

Descripción de las actividades del presente proyecto de investigación



Actividades y Tareas		Año 2022				Año 2022				Año 2022				Año 2022				Año 2022				Año 2023				Año 2023				Año 2023				Año 2023				Año 2023		
		Meses / Semanas				Meses / Semanas				Meses / Semanas				Meses / Semanas				Meses / Semanas				Meses / Semanas				Meses / Semanas				Meses / Semanas				Meses / Semanas				Meses / Semanas		
		Agosto				Setiembre				Octubre				Noviembre				Diciembre				Marzo				Abril				Mayo				Junio				Julio		
Actividad		1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3
1	Selección del tema de investigación																																							
2	Diseño del título de Tesis																																							
3	Elaboración del plan de investigación																																							
4	Identificación de la realidad de la problemática																																							
5	Recolección de información																																							
6	Planteamiento del problema																																							
7	Planteamiento de los objetivos																																							
8	Justificación y delimitación																																							
9	Revisión Bibliográfica																																							
10	Marco Teórico																																							
11	Planteamiento de la hipótesis																																							
12	Metodología																																							
13	Recolección de Dataset																																							
14	Preparación de los datos																																							
15	Analisis de los datos																																							
16	Desarrollo de la solución analítica																																							
17	Evaluación Económica y Financiera																																							
18	Discusión sobre resultados																																							
19	Despliegue																																							
20	Conclusiones y recomendaciones																																							
21	Elaboración del informe final																																							
22	Presentación																																							
23	Sustentación																																							

Fuente: Elaboración propia

3.4.2.Presupuesto

En la presente investigación se dispondrá de montos de inversión por cada recurso que se necesita para el desarrollo de este proyecto, estos montos están clasificados por tipo y categorías.

Tabla 13

Información de presupuesto de la investigación

Tipo	Categoría	Recurso	Descripción	Fuente Financiadora	Monto
Recursos disponibles	Equipo	Laptop	Este quipo servirá para construir y desarrollar los modelos de inteligencia artificial	Propio	S/ 4800
	Software	Anaconda	Software de Inteligencia Artificial	Propio	-
	Nube	Google Colab	Plataforma Online de Inteligencia Artificial	Propio	-
Recursos necesarios	Equipo	GPU	Adquisición de recursos computacionales de GPU y TPU	Compra de servicio en Google	S/ 600

Fuente: Elaboración propia

Capítulo IV: Desarrollo del Experimento

En esta fase del proyecto se detallará la realización de todos los pasos explicados en la metodología de este experimento, donde se mostrará la obtención del dataset, el pre procesamiento, la construcción de los modelos de inteligencia artificial y sus resultados.

4.1. Obtención del Dataset

En base a lo explicado en la parte de la metodología, para este proyecto se realizó la obtención de 5 datasets de imágenes de los insectos de tipo culícidos de la familia Aedes y Culex.

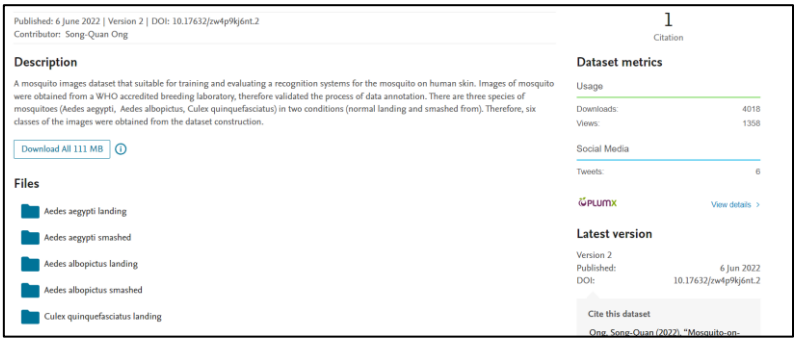
4.1.1. Obtención del primer dataset

Para la obtención de este primer dataset, se consideró las imágenes trabajadas en el laboratorio de la Universidad Sabah de Malasia, en la cual se encuentra acreditada por la OMS. En este laboratorio se trabajaron las imágenes de los insectos vivos y aplastados de las especies Aedes Aegypty, Aedes Albopictus y Culex Quinquefasciatus con el objetivo de reconocer y obtener una clasificación automática de cada especie en la posición viva y aplastada en la piel humana. De todas estas imágenes, para este proyecto solo se consideró la toma de las imágenes de los insectos vivos mostrando toda la morfología del cuerpo del insecto.

Luego de identificar las imágenes de las especies vivas de Aedes y Culex, se procedió a descargar solo las carpetas específicas que se encuentran en el repositorio oficial del laboratorio de esta universidad.

Figura 55

Repositorio Oficial de descarga del primer dataset



Fuente: Universidad Sabah de Malasia

En este repositorio solo se consideró 239 imágenes de Aedes Aegypti, 239 imágenes de Aedes Albopictus y 246 imágenes de Culex Quinquefasciatus. En resumen se tiene 478 imágenes de tipo Aedes y 246 imágenes de tipo Culex.

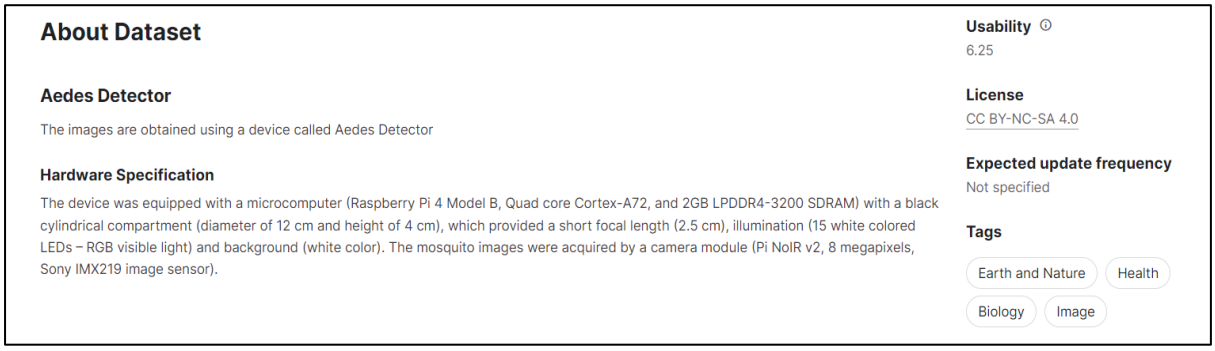
4.1.2. Obtención del segundo dataset

Para la obtención del segundo dataset, se consideró las imágenes trabajadas en el laboratorio de la Universidad KDU Penang de Malasia. En este laboratorio se trabajaron las imágenes de los insectos de las especies Aedes Aegypti y Aedes Albopictus, con el objetivo de clasificar por edades a estos dos tipos de especies en base a sus niveles de escamas mediante inteligencia Artificial. De todas estas imágenes, para este proyecto solo se consideró la toma de las imágenes de los insectos de Aedes mostrando toda la morfología completa del cuerpo del insecto.

Luego de identificar las imágenes de las especies vivas de Aedes, se procedió a descargar solo sus carpetas correspondientes que se encuentran en el repositorio de Kaggle asociada a la universidad.

Figura 56

Repositorio Oficial de descarga del segundo dataset



Fuente: Universidad de KDU Penang de Malasia

En este repositorio solo se consideró 603 imágenes de Aedes Aegypti y 600 imágenes de Aedes Albopictus. En resumen se tiene un total de 1203 imágenes de tipo Aedes.

4.1.3. Obtención del tercer dataset

Para la obtención del tercer dataset, se consideró las imágenes trabajadas por la Universidad de Incheon de Corea del Sur. En esa investigación de inteligencia artificial realizaron la clasificación de 8 especies, del cual para esta investigación se seleccionaron solo las especies de Aedes y Culex, mostrando toda la morfología completa del insecto.

Luego de identificar las imágenes de las especies de Aedes y Culex, se procedió a descargar solo sus carpetas correspondientes que se encuentran en el repositorio de Github asociada a la universidad.

Figura 57

Repositorio Oficial de descarga del tercer dataset



Fuente: Universidad de Incheon de Coreo del Sur

En este repositorio solo se consideró 600 imágenes de Aedes Albopictus, 600 imágenes de Culex Pippiens, 594 imágenes de Culex Tritaeniorhynchus y 199 imágenes de Culex Inatomii. En resumen se tiene 600 imágenes de tipo Aedes y 1393 imágenes de tipo Culex.

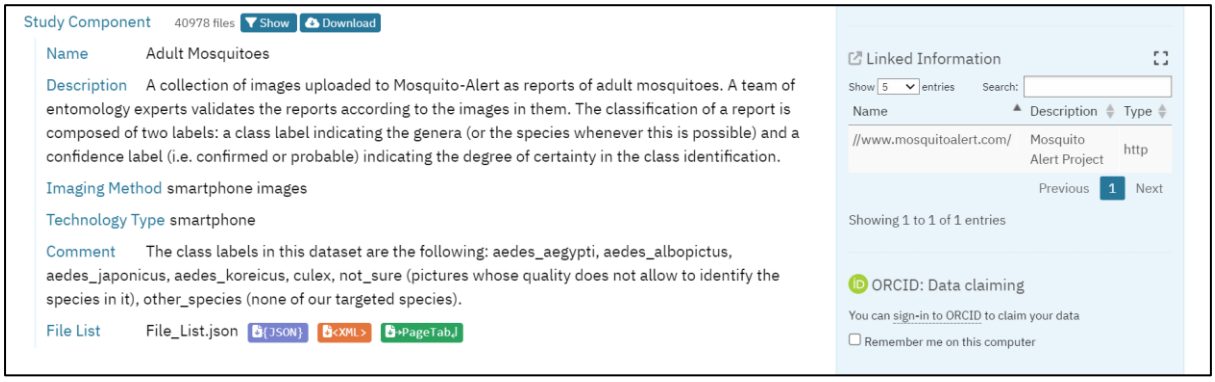
4.1.4. Obtención del cuarto dataset

Para la obtención del cuarto dataset, se consideró las imágenes del observatorio ciudadano en coordinación con diferentes instituciones públicas de investigación, que en su base original tienen miles de imágenes de diferentes especies de insectos. En la cual para esta investigación se consideró solo las imágenes de las especie Culex Pipiens.

Luego de identificar las imágenes de las especie de Culex, se procedió a descargar solo sus carpetas correspondientes que se encuentran en el repositorio de Github asociada a la universidad.

Figura 58

Repositorio Oficial de descarga del cuarto dataset



Fuente: Observatorio Ciudadano Internacional

En este repositorio solo se consideró 163 imágenes de Culex Pipiens, donde cada imagen muestra la morfología del cuerpo completo del insecto.

4.1.5. Obtención del quinto dataset

Para obtener el quinto dataset, se realizó la técnica de web scraping donde se extrajeron las imágenes de Aedes y Culex de la página web oficial de la Universidad del Estado Libre del Departamento de Zoología y Entomología de Sudáfrica con la asociación de la organización International Barcode of Life Project.

Primero, en el entorno de trabajo de Jupyter Notebook se realizó la importación de las librerías necesarias para el trabajo de web scraping.

Figura 59

Importación de librerías para Web Scraping

```
[ ]  import time
     import os
     import io
     import PIL.Image as Image
     import cv2
     from bs4 import BeautifulSoup
     import re
     import requests
```

Fuente: Elaboración Propia

Como se describe en la imagen, principalmente se utilizó la librería “time” para medir los tiempos entre cada línea de código al momento de extraer cada imagen de la página. La librería “os” para acceder a las carpetas donde se guardaran las imágenes en google drive. También se utilizó la librería “cv2” para el tratamiento de la imágenes .Adicionalmente, las librerías más importantes en esta etapa fueron la de “request” que extrajo todo el “html” por página para obtener las urls de imágenes seleccionadas de las especies de Aedes y Culex. Así como también la librería de “beautifulsoup” que permitió realizar el maquetado y la búsqueda por cada línea de “html” donde se ubique cada etiqueta “tag” que contenga la url de cada imagen.

El proceso de web scraping se realizó por cada especie de Aedes y Culex, que al final de su debido procedimiento se almacenaron en una carpeta en drive.

4.1.5.1. Extracción de imágenes del primer grupo de Culex. Como primer paso se identificó la url que contiene la dirección y paginado de la imágenes de tipo Culex Pipiens. Con la librería request se extrajo todo el “html” de esta página en cuestión. Luego, se guardó el “html” en un archivo “.txt” en el google drive y posteriormente leerlo mediante “beautifulsoup” y utilizar sus propiedades de esta librería para estructurar la información y leer cada el tag objetivo.

Figura 60

Identificación y registro de url de la página objetivo del primer grupo de culex

```
[ ] url = "https://www.boldsystems.org/index.php/Public_BarcodeCluster?clusteruri=BOLD:AAA4751"

    p12=requests.get(url)

[ ] driver.get('https://www.boldsystems.org/index.php/Public_RecordView?processid=GBCC045-12')

[ ] f = open ('C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/DataSet/HTML.txt','r')
    mensaje = f.read()

[ ] s=BeautifulSoup(mensaje, 'lxml')

[ ] secciones = s.find('tbody',attrs={'class':'scrollContent'}).find_all('td')
```

Fuente: Elaboración propia

En segundo paso, luego de obtener todo el html estructurado se procede a encontrar los códigos numéricos asociados a las etiquetas que textualmente están como “Culex” y se guardan en una variable temporal de tipo lista.

Figura 60

Identificación de los códigos asociados a la clase Culex del primer grupo de esta especie.

```
lista = []
for i in secciones:
    if ('Culex pipiens' in (i.img.get('title'))):
        lista.append(i.img.get('title'))

[ ] lista

['KCOLE2517-13 {Culex pipiens}',
 'KCOLE2543-13 {Culex pipiens}',
 'KCOLE2545-13 {Culex pipiens}',
 'MPSAM021-21 {Culex pipiens}',
 'KCOLE2549-13 {Culex pipiens}',
 'GDIP4675-22 {Culex pipiens}',
 'KCOLE2542-13 {Culex pipiens}',
 'KCOLE2516-13 {Culex pipiens}',
 'KCOLE2554-13 {Culex pipiens}',
 'KCOLE2515-13 {Culex pipiens}',
 'KCOLE2544-13 {Culex pipiens}',
 'BNLL274-21 {Culex pipiens}',
 'KCOLE2552-13 {Culex pipiens}',
 'MPSAM034-21 {Culex pipiens}',
 'MOAV077-15 {Culex pipiens}',
 'LEASV1590-19 {Culex pipiens}',
 'CROCU245-21 {Culex pipiens}',
 'CROCU120-21 {Culex pipiens}',
 'KCOLE2550-13 {Culex pipiens}',
 'KCOLE2546-13 {Culex pipiens}',
 'KCOLE2551-13 {Culex pipiens}',
 'PAVEA161-22 {Culex pipiens}',
 'PAVEA154-22 {Culex pipiens}',
```

Fuente: Elaboración propia

Como tercer paso, luego de obtener todos los códigos números asociados a la etiqueta “Culex”, se procede a agregarlos a la url principal de la página que contiene las

especificaciones de cada insecto y posterior a ello obtener su correspondiente url de cada imagen para luego guardar en una carpeta especifica en drive.

Figura 61

Agregación de códigos a la url de la página web objetivo



```
k = 0
for i in lista_nueva:
    url  = 'https://www.boldsystems.org/index.php/Public_RecordView?processid=' + i
    try:
        page = requests.get(url)
        s_page = BeautifulSoup(page.text,'lxml')
        imagen_url = s_page.find('img', attrs={'width':'100%'}).get('src')
        imagen_url = 'https:' + imagen_url
        image = requests.get(imagen_url)
        imageh = Image.open(io.BytesIO(image.content))
        imageh = imageh.convert('RGB')
        imageh.save("C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/DataSet/BD5_U4/"+str(k)+".jpg")
        k=k+1
    except Exception as e:
        print('Registros vacios:')
        print(i)
        print(e)
        print('\n')
```

Fuente: Elaboración propia

4.1.5.1. Extracción de imágenes del segundo grupo de Culex. Como primer paso se identificó la url que contiene la dirección y paginado de la imágenes de tipo Culex Tritaeniorhynchus. Posterior a ello mediante la librería request se extrajo todo el html de esta página en cuestión. Luego, se guardó el “html” en un archivo “.txt” en el google drive y posteriormente leerlo mediante “beautifulsoup” y utilizar sus funciones de esta librería para estructurar la información y leer cada el “tag” objetivo.

Figura 62

Identificación y registro de url de la página objetivo del segundo grupo de Culex.



```
[ ] url = "https://www.boldsystems.org/index.php/Public_BarcodeCluster?clusteruri=BOLD:AAA4751"

p12=requests.get(url)

[ ] driver.get('https://www.boldsystems.org/index.php/Public_RecordView?processid=GBCC045-12')

[ ] f = open ('C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/DataSet/HTML.txt','r')
    mensaje = f.read()

[ ] s=BeautifulSoup(mensaje, 'lxml')

[ ] secciones = s.find('tbody',attrs={'class':'scrollContent'}).find_all('td')
```

Fuente: Elaboración propia

En segundo paso, luego de obtener todo el html estructurado se procede a encontrar los códigos numéricos asociados a las etiquetas que textualmente están como “Culex Tritaeniorhynchus” y se guardan en una variable temporal de tipo lista.

Figura 63

Identificación de los códigos asociados a la clase Culex del segundo grupo de esta especie.



```
[ ] lista = []
    for i in secciones:
        if ('Culex tritaeniorhynchus' in (i.img.get('title'))):
            lista.append(i.img.get('title'))

[ ] lista

['MOSPK189-22 {Culex tritaeniorhynchus}',
 'MOSPK132-22 {Culex tritaeniorhynchus}',
 'MOSPK179-22 {Culex tritaeniorhynchus}',
 'MOSPK117-22 {Culex tritaeniorhynchus}',
 'MOSPK128-22 {Culex tritaeniorhynchus}',
 'MOSPK125-22 {Culex tritaeniorhynchus}',
 'MOSPK141-22 {Culex tritaeniorhynchus}',
 'MOSPK188-22 {Culex tritaeniorhynchus}',
 'MOSPK120-22 {Culex tritaeniorhynchus}',
 'MOSPK137-22 {Culex tritaeniorhynchus}',
 'MOSPK118-22 {Culex tritaeniorhynchus}',
 'MOSPK122-22 {Culex tritaeniorhynchus}',
 'MOSPK140-22 {Culex tritaeniorhynchus}',
 'MOSPK121-22 {Culex tritaeniorhynchus}',
 'MOSPK129-22 {Culex tritaeniorhynchus}',
 'MOSPK123-22 {Culex tritaeniorhynchus}',
 'MOSPK056-19 {Culex tritaeniorhynchus}',
 'MOSPK072-19 {Culex tritaeniorhynchus}',
 'MOSPK043-19 {Culex tritaeniorhynchus}',
 'MOSPK067-19 {Culex tritaeniorhynchus}',
 'MOSPK037-19 {Culex tritaeniorhynchus}',
```

Fuente: Elaboración propia

Como tercer paso, luego de obtener todos los códigos números asociados a la etiqueta “Culex Tritaeniorhynchus”, se procede a agregarlos a la url principal de la página que

contiene las especificaciones de cada insecto y posterior a ello obtener su correspondiente url de cada imagen para luego guardar en una carpeta especifica en drive.

Figura 64

Agregación de códigos a la url de la página web objetivo del segundo grupo de Culex.

```
k = 0
for i in lista_nueva:
    url  = 'https://www.boldsystems.org/index.php/Public_RecordView?processid=' + i
    try:
        page = requests.get(url)
        s_page = BeautifulSoup(page.text,'lxml')
        imagen_url = s_page.find('img', attrs={'width':'100%'}).get('src')
        imagen_url = 'https:' + imagen_url
        image = requests.get(imagen_url)
        imageh = Image.open(io.BytesIO(image.content))
        imageh = imageh.convert('RGB')
        imageh.save("C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/OFICIAL_DATASETS/BD6/Culex/Culex Tritaeniorhynchus/"+str(k)+".jpg")
        k=k+1
    except Exception as e:
        print('Registros vacios:')
        print(i)
        print(e)
        print('\n')
```

Fuente: Elaboración propia

4.1.5.1. Extracción de imágenes del tercer grupo de Culex. Como primer paso se identificó la url que contiene la dirección y paginado de la imágenes de tipo Culex Quinquefasciatus. Posterior a ello mediante la librería request se extrajo todo el html de esta página en cuestión. Luego, se guardó el html en un archivo “.txt” en el google drive y posteriormente leerlo mediante “beautifulsoup” y utilizar sus propiedades de esta librería para estructurar la información y leer cada el “tag” objetivo.

Figura 65

Identificación y registro de url de la página objetivo del tercer grupo de Culex.



```
[ ] url = "https://www.boldsystems.org/index.php/Public_BarcodeCluster?clusteruri=BOLD:AAA4751"

    p12=requests.get(url)

[ ] driver.get('https://www.boldsystems.org/index.php/Public_RecordView?processid=GBCC045-12')

[ ] f = open ('C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/DataSet/HTML.txt','r')
    mensaje = f.read()

[ ] s=BeautifulSoup(mensaje, 'lxml')

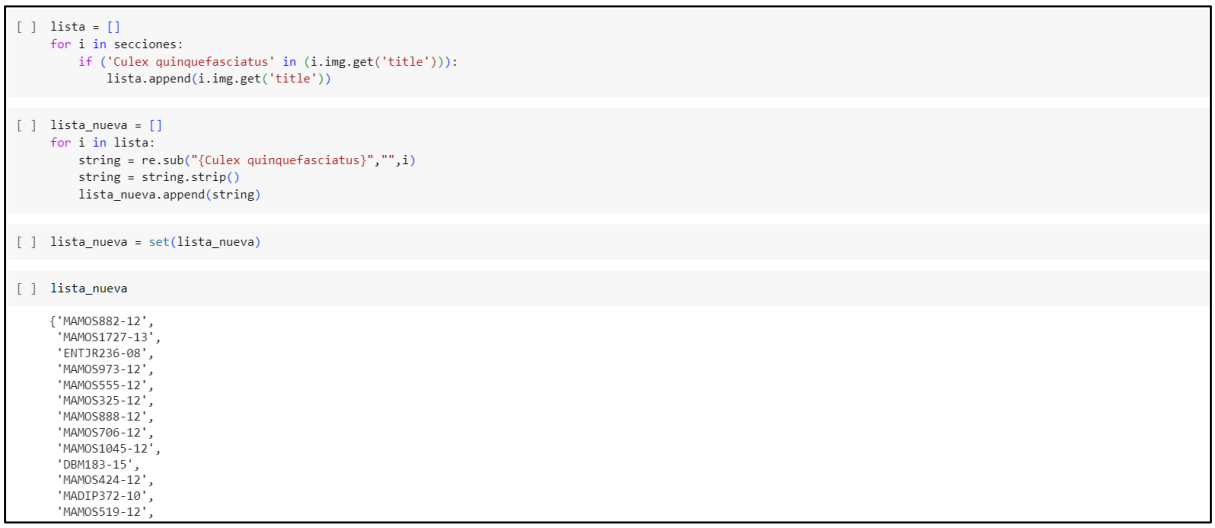
[ ] secciones = s.find('tbody',attrs={'class':'scrollContent'}).find_all('td')
```

Fuente: Elaboración propia

En segundo paso, luego de obtener todo el html estructurado se procede a encontrar los códigos numéricos asociados a las etiquetas que textualmente están como “Culex Quinquefasciatus” y se guardan en una variable temporal de tipo lista.

Figura 66

Identificación de los códigos asociados a la clase Culex del tercer grupo de esta especie.



```
[ ] lista = []
    for i in secciones:
        if ('Culex quinquefasciatus' in (i.img.get('title'))):
            lista.append(i.img.get('title'))

[ ] lista_nueva = []
    for i in lista:
        string = re.sub("{Culex quinquefasciatus}","",i)
        string = string.strip()
        lista_nueva.append(string)

[ ] lista_nueva = set(lista_nueva)

[ ] lista_nueva

    {'MAMOS882-12',
     'MAMOS1727-13',
     'ENTJR236-08',
     'MAMOS973-12',
     'MAMOS555-12',
     'MAMOS325-12',
     'MAMOS888-12',
     'MAMOS706-12',
     'MAMOS1045-12',
     'DBM183-15',
     'MAMOS424-12',
     'MADIP372-10',
     'MAMOS519-12',
```

Como tercer paso, luego de obtener todos los códigos números asociados a la etiqueta “Culex Quinquefasciatus”, se procede a agregarlos a la url principal de la página que contiene las especificaciones de cada insecto y posterior a ello obtener su correspondiente url de cada imagen para luego guardar en una carpeta especifica en drive.

Figura 67

Agregación de códigos a la url de la página web objetivo del tercer grupo de Culex.

```
[ ]  k = 0
     for i in lista_nueva:
         url  = 'https://www.boldsystems.org/index.php/Public_RecordView?processid=' + i
         try:
             page = requests.get(url)
             s_page = BeautifulSoup(page.text,'lxml')
             imagen_url = s_page.find('img', attrs={'width':'100%'}).get('src')
             imagen_url = 'https:' + imagen_url
             image = requests.get(imagen_url)
             imageh = Image.open(io.BytesIO(image.content))
             imageh = imageh.convert('RGB')
             imageh.save("C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/OFICIAL_DATASETS/BD6/Culex/Culex Quinquefasciatus/"+str(k)+".jpg")
             k=k+1
         except Exception as e:
             print('Registros vacios:')
             print(i)
             print(e)
             print('\n')
```

Fuente: Elaboración propia

4.1.5.1. Extracción de imágenes del primer grupo de Aedes. Como primer paso se identificó la url que contiene la dirección y paginado de la imágenes de tipo Aedes Aegypti. Posterior a ello mediante la librería request se extrajo todo el html de esta página en cuestión. Luego, se guardó el html en un archivo “.txt” en el google drive y posteriormente leerlo mediante “beautifulsoup” y utilizar sus propiedades de esta librería para estructurar la información y leer cada el tag objetivo.

Figura 68

Identificación y registro de url de la página objetivo del primer grupo de Aedes.

```
url_a_aegypti = "https://www.boldsystems.org/index.php/Public_BarcodeCluster?clusteruri=BOLD:AEI9358"

[ ]  page_a_aegypti = requests.get(url_a_aegypti)

[ ]  s_a_aegypti = BeautifulSoup(page_a_aegypti.text, 'lxml')

[ ]  secciones_a_aegypti = s_a_aegypti.find('tbody',attrs={'class':'scrollContent'}).find_all('td')

[ ]  secciones_a_aegypti

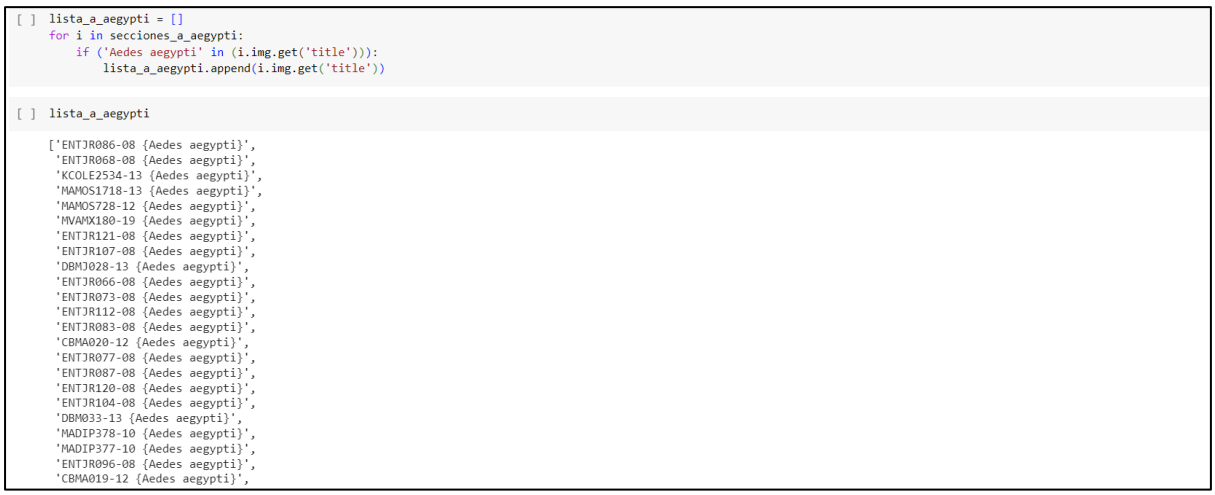
[<td class="imagegallery_imagecell" style="background-color:#EFD279"><span style="top: 0px; font-size: 0.9em; font-family: Arial, Helvetica, sans-serif;">Lateral</span><br/>
 <img alt="" onclick='previewImage("Lateral","//www.boldsystems.org/pics/ENTJR/tmen009-004%2B1203607906.JPG","ENTJR086-08","Aedes aegypti","CreativeCommons - Attribution","CBG Photography
Group","(2008)","Centre for Biodiversity Genomics", "/libhtml/icons/creativecommons/by_80x15.png", "http://creativecommons.org/licenses/by/3.0/",261715)'
src="//www.boldsystems.org/pics/_w70/ENTJR/tmen009-004%2B1203607906.JPG" style="height: 60px; width: 60px;" title="ENTJR086-08 (Aedes aegypti)"/>
 </td>,
 <td class="imagegallery_imagecell" style="background-color:#EFD279"><br/>
 <img alt="" onclick='previewImage("Lateral","//www.boldsystems.org/pics/ENTJR/tmen007-006%2B1203607900.JPG","ENTJR068-08","Aedes aegypti","CreativeCommons - Attribution","CBG Photography
Group","(2008)","Centre for Biodiversity Genomics", "/libhtml/icons/creativecommons/by_80x15.png", "http://creativecommons.org/licenses/by/3.0/",261697)'
src="//www.boldsystems.org/pics/_w70/ENTJR/tmen007-006%2B1203607900.JPG" style="height: 60px; width: 60px;" title="ENTJR068-08 (Aedes aegypti)"/>
 </td>,
 <td class="imagegallery_imagecell" style="background-color:#EFD279"><br/>
 <img alt="" onclick='previewImage("Lateral","//www.boldsystems.org/pics/KCOLE/D05_DSC_4133%2B1366973580.JPG","KCOLE2534-13","Aedes aegypti","Copyright","Yvonne U Ajamma","(2012)","icipe",
"", "",2179455)' src="//www.boldsystems.org/pics/_w70/KCOLE/D05_DSC_4133%2B1366973580.JPG" style="height: 60px; width: 60px;" title="KCOLE2534-13 (Aedes aegypti)"/>
 </td>
```

Fuente: Elaboración propia

En segundo paso, luego de obtener todo el html estructurado se procede a encontrar los códigos numéricos asociados a las etiquetas que textualmente están como “Aedes Aegypti” y se guardan en una variable temporal de tipo lista.

Figura 69

Identificación de los códigos asociados a la clase Aedes al primer grupo de esta especie.



```
[ ] lista_a_aegypti = []
    for i in secciones_a_aegypti:
        if ('Aedes aegypti' in (i.img.get('title'))):
            lista_a_aegypti.append(i.img.get('title'))

[ ] lista_a_aegypti

['ENTJR086-08 {Aedes aegypti}',
 'ENTJR068-08 {Aedes aegypti}',
 'KCOLE2534-13 {Aedes aegypti}',
 'MAMOS1718-13 {Aedes aegypti}',
 'MAMOS728-12 {Aedes aegypti}',
 'MVAMX180-19 {Aedes aegypti}',
 'ENTJR121-08 {Aedes aegypti}',
 'ENTJR107-08 {Aedes aegypti}',
 'DBMJ028-13 {Aedes aegypti}',
 'ENTJR066-08 {Aedes aegypti}',
 'ENTJR073-08 {Aedes aegypti}',
 'ENTJR112-08 {Aedes aegypti}',
 'ENTJR083-08 {Aedes aegypti}',
 'CBMA020-12 {Aedes aegypti}',
 'ENTJR077-08 {Aedes aegypti}',
 'ENTJR087-08 {Aedes aegypti}',
 'ENTJR120-08 {Aedes aegypti}',
 'ENTJR104-08 {Aedes aegypti}',
 'DBM033-13 {Aedes aegypti}',
 'MADIP378-10 {Aedes aegypti}',
 'MADIP377-10 {Aedes aegypti}',
 'ENTJR096-08 {Aedes aegypti}',
 'CBMA019-12 {Aedes aegypti}',
```

Fuente: Elaboración propia

Como tercer paso, luego de obtener todos los códigos números asociados a la etiqueta “Aedes Aegypti”, se procede a agregarlos a la url principal de la página que contiene las especificaciones de cada insecto y posterior a ello obtener su correspondiente url de cada imagen para luego guardar en una carpeta especifica en drive.

Figura 70

Agregación de códigos a la url de la página web objetivo del primer grupo de Aedes.

```
[ ]  k = 0
     for i in lista_nueva_a_aegypti:
         url  = 'https://www.boldsystems.org/index.php/Public_RecordView?processid=' + i
         try:
             page = requests.get(url)
             s_page = BeautifulSoup(page.text,'lxml')
             imagen_url = s_page.find('img', attrs={'width':'100%'}).get('src')
             imagen_url = 'https:' + imagen_url
             image = requests.get(imagen_url)
             imageh = Image.open(io.BytesIO(image.content))
             imageh = imageh.convert('RGB')
             imageh.save("C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/OFICIAL_DATASETS/BD6/Culex/Aedes Aegypti/"+str(k)+".jpg")
             k=k+1
         except Exception as e:
             print('Registros vacios:')
             print(i)
             print(e)
             print('\n')
```

Fuente: Elaboración propia

4.1.5.1. Extracción de imágenes del segundo grupo de Aedes. Como primer paso se identificó la url que contiene la dirección y paginado de la imágenes de tipo Aedes Albopictus. Posterior a ello mediante la librería request se extrajo todo el html de esta página en cuestión. Luego, se guardó el html en un archivo txt en el google drive y posteriormente leerlo mediante beautifulsoup y utilizar sus propiedades de esta librería para estructurar la información y leer cada el tag objetivo.

Figura 71

Identificación y registro de url de la página objetivo del segundo grupo de Aedes.

```
url_a_albopictus = "https://www.boldsystems.org/index.php/Public_BarcodeCluster?clusteruri=BOLD:AAA5870"

[ ] page_a_albopictus = requests.get(url_a_albopictus)

[ ] s_a_albopictus = BeautifulSoup(page_a_albopictus.text, 'lxml')

[ ] secciones_a_albopictus = s_a_albopictus.find('tbody',attrs={'class':'scrollContent'}).find_all('td')

[ ] secciones_a_albopictus

[<td class="imagegallery_imagecell" style="background-color:#EFD279"><span style="top: 0px; font-size: 0.9em; font-family: Arial, Helvetica, sans-serif;">Lateral</span><br/>
 <img alt="" onclick='previewImage("Lateral","//www.boldsystems.org/pics/CROCU/Aedes_albopictus_AK-M14b%2B1528991818.JPG","CROCU302-21","Aedes albopictus","CreativeCommons - Attribution Non-Commercial Share-Alike","Natasa Busic","(2020)","Josip Juraj Strossmayer University of Osijek", "/libhtml/icons/creativecommons/by-nc-sa_80x15.png", "http://creativecommons.org/licenses/by-nc-sa/3.0/",5820441)' src="//www.boldsystems.org/pics/_w70/CROCU/Aedes_albopictus_AK-M14b%2B1528991818.JPG" style="height: 60px; width: 60px;" title="CROCU302-21 {Aedes albopictus}"/>
 </td>,
 <td class="imagegallery_imagecell" style="background-color:#EFD279"><br/>
 <img alt="" onclick='previewImage("Lateral","//www.boldsystems.org/pics/CROCU/Aedes_albopictus_NG-230%2B1597851424.jpg","CROCU225-21","Aedes albopictus","CreativeCommons - Attribution Non-Commercial Share-Alike","Natasa Busic","(2020)","Josip Juraj Strossmayer University of Osijek", "/libhtml/icons/creativecommons/by-nc-sa_80x15.png", "http://creativecommons.org/licenses/by-nc-sa/3.0/",5820303)' src="//www.boldsystems.org/pics/_w70/CROCU/Aedes_albopictus_NG-230%2B1597851424.jpg" style="height: 60px; width: 60px;" title="CROCU225-21 {Aedes albopictus}"/>
 </td>,
 <td class="imagegallery_imagecell" style="background-color:#EFD279"><br/>
 <img alt="" onclick='previewImage("Lateral","//www.boldsystems.org/pics/CROCU/Aedes_albopictus_NG-92c%2B1573656476.jpg","CROCU126-21","Aedes albopictus","CreativeCommons - Attribution Non-Commercial Share-Alike","Natasa Busic","(2020)","Josip Juraj Strossmayer University of Osijek", "/libhtml/icons/creativecommons/by-nc-sa_80x15.png", "http://creativecommons.org/licenses/by-nc-sa/3.0/",5820150)' src="//www.boldsystems.org/pics/_w70/CROCU/Aedes_albopictus_NG-92c%2B1573656476.jpg" style="height: 60px; width: 60px;" title="CROCU126-21 {Aedes albopictus}"/>
```

Fuente: Elaboración propia

En segundo paso, luego de obtener todo el html estructurado se procede a encontrar los códigos numéricos asociados a las etiquetas que textualmente están como “Aedes Albopictus” y se guardan en una variable temporal de tipo lista agregando también la eliminación de etiquetas duplicadas.

Figura 72

Identificación de los códigos asociados a la clase Aedes al segundo grupo de esta especie.

```
[ ]  lista_a_albopictus = []
     for i in secciones_a_albopictus:
         if ('Aedes albopictus' in (i.img.get('title'))):
             lista_a_albopictus.append(i.img.get('title'))

[ ]  len(lista_a_albopictus)

     280

[ ]  lista_nueva_a_albopictus = []
     for i in lista_a_albopictus:
         string = re.sub("{Aedes albopictus}","",i)
         string = string.strip()
         lista_nueva_a_albopictus.append(string)

[ ]  len(lista_nueva_a_albopictus)

     280

[ ]  lista_nueva_a_albopictus = set(lista_nueva_a_albopictus)

[ ]  len(lista_nueva_a_albopictus)

     254
```

Fuente: Elaboración propia

Como tercer paso, luego de obtener todos los códigos números asociados a la etiqueta “Aedes Aegypti”, se procede a agregarlos a la url principal de la página que contiene las especificaciones de cada insecto y posterior a ello obtener su correspondiente url de cada imagen para luego guardar en una carpeta especifica en drive.

Figura 73

Agregación de códigos a la url de la página web objetivo del segundo grupo de Aedes.

```
k = 0
for i in lista_nueva_a_albopictus:
    url  = 'https://www.boldsystems.org/index.php/Public_RecordView?processid=' + i
    try:
        page = requests.get(url)
        s_page = BeautifulSoup(page.text,'lxml')
        imagen_url = s_page.find('img', attrs={'width':'100%'}).get('src')
        imagen_url = 'https:' + imagen_url
        image = requests.get(imagen_url)
        imageh = Image.open(io.BytesIO(image.content))
        imageh = imageh.convert('RGB')
        imageh.save("C:/Users/cuent/Desktop/Cursos/2022-2/Tesis 2/OFICIAL_DATASETS/BD6/Culex/Aedes Albopictus/"+str(k)+".jpg")
        k=k+1
    except Exception as e:
        print('Registros vacios:')
        print(i)
        print(e)
        print('\n')
```

Fuente: Elaboración propia

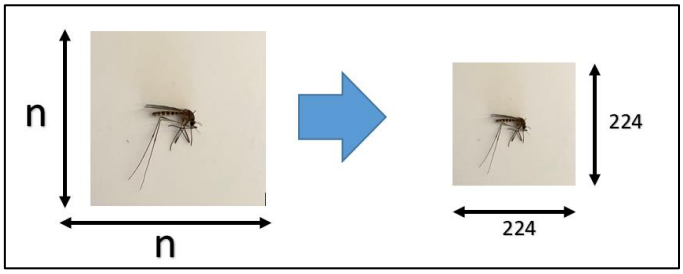
4.2. Pre procesamiento

4.2.1. Redimensionamiento

Luego de realizar la obtención de las 5 base de datos, se precedió a analizar el tamaño de las imágenes de cada conjunto de datos y se observó que tenían distintos tamaños. Por ende para mantener una sola dimensión en todo el conjunto de datos totales y por la propia necesidad de tener una sola dimensión en la clasificación binaria mediante los modelos de inteligencia artificial, se realizó en el cambio de dimensión a 224x224 en cada imagen del dataset.

Figura 74

Especificación de cambio de dimensión a todo el dataset.



Fuente: Elaboración propia

En este proceso se utilizaron las librerías de “os” para obtener la ruta, la navegación hacia archivo de cada imagen en google drive. También se utilizó la librería de “google.colab” para la conexión a google drive. Por último se utilizó la librería “PIL” para la conversión de la imágenes.

Figura 75

Importación de librerías de cambio de dimensión



Fuente: Elaboración propia

Luego se estableció la conexión con google drive donde se guardaron las imágenes totales del dataset de esta investigación.

Figura 76

Importación de librería de conexión a google drive para el redimensionamiento.



Fuente: Elaboración propia

Con la conexión ya establecida hacia los directorios objetivos, se procedió a guardar las rutas y mediante un bucle “for” se procedió a guardar cada imagen en una variable temporal. En la cual mediante la librería “PIL” utilizando su componente “Imagen” y con su funcionalidad “resize” se convirtieron cada imagen del dataset a una dimensión de 224x224. Luego cada imagen redimensionada se procedió a guardarla en una carpeta distinta.

Figura 77

Código donde se guarda las imágenes redimensionadas a 224x224

```
eti_base_culex =sorted(os.listdir('/content/drive/MyDrive/2023-1/DATA/DATA_GENERAL/BD_Original/Culex'))
eti_base_aedes=sorted(os.listdir('/content/drive/MyDrive/2023-1/DATA/DATA_GENERAL/BD_Original/Aedes'))

[ ] #Culex
    k = 0
    for i in eti_base_culex:
      im = Image.open('/content/drive/MyDrive/2023-1/DATA/DATA_GENERAL/BD_Original/Culex/'+ i)
      im = im.convert('RGB')
      im = im.resize((224,224))
      im.save('/content/drive/MyDrive/2023-1/DATA/DATA_GENERAL/BD_Original_RE/Culex/' + str(k)+'.jpg', quality=95)
      k = k+1

[ ] #Aedes
    k=0
    for i in eti_base_aedes:
      im = Image.open('/content/drive/MyDrive/2023-1/DATA/DATA_GENERAL/BD_Original/Aedes/'+ i)
      im = im.convert('RGB')
      im = im.resize((224,224))
      im.save('/content/drive/MyDrive/2023-1/DATA/DATA_GENERAL/BD_Original_RE/Aedes/' + str(k)+'.jpg', quality=95)
      k = k+1
```

Fuente: Elaboración propia

4.2.2. Balanceo de datos

Esta técnica de pre procesamiento se implementó en los datos debido a la diferencia en cantidades entre las dos clases de tipo de insectos. La clase mayoritaria es el de tipo Aedes con una cantidad de 2432 y una clase minoritaria del tipo Culex de 2081 imágenes. Debido a esto, se procedió a nivelar la clase minoritaria mediante la técnica SMOTE.

Como primer paso se realizó la importación de las librerías a utilizar, donde se consideró el “cv2” para el tratamiento de la imágenes, la librería “os” para navegar entre carpetas en google drive, la librería “pandas” para estructurar la información de la imágenes, la librería “matplotlib” para la visualización de las etiquetas, la librería “imblearn” para la generación de datos sintéticos y la librería de “numpy” para manejar los arrays de cada imagen.

Figura 78

Importación de librerías para el balanceo de datos



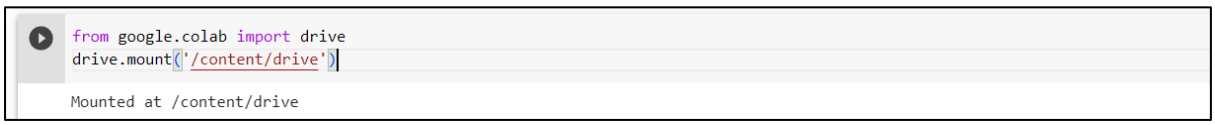
```
import cv2
import os
import pandas as pd
import matplotlib.pyplot as plt
from imblearn.over_sampling import SMOTE
import numpy as np
```

Fuente: Elaboración propia

Luego se estableció la conexión con las capetas de las imágenes en google drive para conectarse al directorio de las imágenes redimencionadas.

Figura 79

Importación de librería de conexión a google drive para el balanceo de datos



```
from google.colab import drive
drive.mount('/content/drive')
```

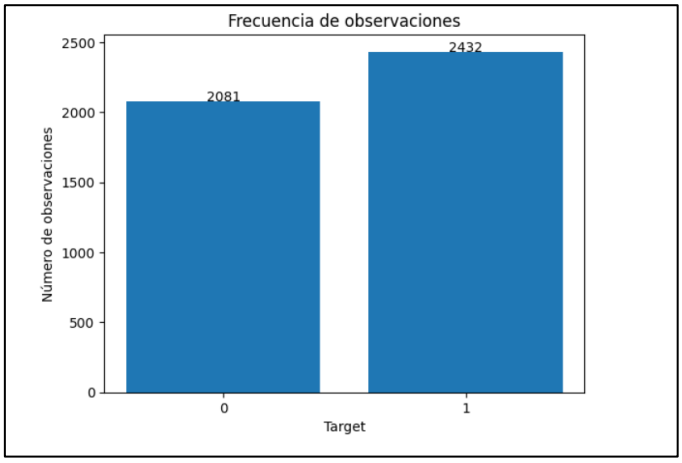
```
Mounted at /content/drive
```

Fuente: Elaboración propia

Luego se cargó la ruta de cada tipo de clase de insecto a una variable temporal y se transformó de etiquetas texto “Culex” y “Aedes” a valores numéricos de 0 y 1 correspondientemente.

Figura 80

Número de imágenes por clase de insecto



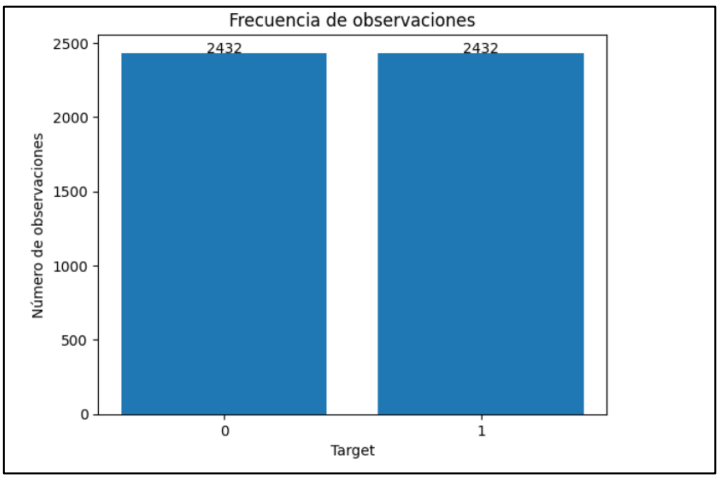
Fuente: Elaboración propia

Posteriormente cada imagen del total del dataset, se guardaron en una variable de tipo lista para luego cargarlas al método “smote” juntamente con sus etiquetas correspondientes para la generación de datos sintéticos.

Luego de generar las imágenes sintéticas, se procedió a contabilizar el total de imágenes por cada clase.

Figura 81

Número de imágenes por clase de insecto después del balanceo de datos



Fuente: Elaboración de datos

Por ultimo las imágenes cargadas en memoria, se procedió a guardarlas en una carpeta en específico en google drive.

Figura 81

Ejemplo de imágenes generadas sintéticamente de la clase minoritaria.



Fuente: Elaboración propia

4.2.3. Aumento de datos

En esta etapa de pre procesamiento se realizó el aumento de datos con el objetivo de mejorar los resultados a nivel de precisión del modelo, para ello se importó la librería de "keras.preprocessing.image" y se utilizó de esa librería la función "ImageDataGenerator" para agregar 4 tipos de variaciones por cada imagen. Estas variaciones fueron:

- Share_range: Genera imágenes con angulos de corte.
- Zoom_range: Genera imágenes con acercamiento.
- Horizontal_flip: Genera imágenes invirtiendo el eje a nivel horizontal
- Vertical_flip: Genera imágenes invirtiendo el eje a nivel vertical

Cabe resaltar que estas variaciones por cada imagen solo se implementaron en el conjunto de entrenamiento y no el de prueba, debido a que el conjunto de prueba necesita entrar de manera directa al modelo y medir su nivel de clasificación. Así mismo estas imágenes aumentadas serán generadas solo en tiempo de ejecución al momento de entrenar el

modelo, para la evaluación luego de implementar la técnica de validación cruzada solo se tomará en cuenta las imágenes originales.

Figura 82

Código de aumento de datos

```
entrenamiento_datagen = ImageDataGenerator(
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    vertical_flip=True,
    validation_split=0.15,
    preprocessing_function=preprocess_input)

test_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)
```

Fuente: Elaboración propia

4.2.4. Normalización de datos

Por último se realizó la normalización de las imágenes, con el objetivo de convertir los valores de 0 a 255 en valores de 0 y 1 para que reciben correctamente los modelos de inteligencia artificial que se implementaran. La normalización de aplica también dentro de la función de “ImageDataGenerator” para estructurar las imágenes y pasarlas a los modelos.

Figura 83

Código de agregación de normalización a la función “ImageDataGenerator”

```
[ ] entrenamiento_datagen = ImageDataGenerator(
        rescale=1. / 255,
        shear_range=0.2,
        zoom_range=0.2,
        horizontal_flip=True,
        vertical_flip=True,
        validation_split=0.15,
        preprocessing_function=preprocess_input)

    test_datagen = ImageDataGenerator(rescale=1. / 255,preprocessing_function=preprocess_input)
```

Fuente: Elaboración propia

4.3. Metodología

En esta etapa del experimento se realizó la construcción de los modelos de inteligencia artificial que son el VGG16, VGG19, ResNet50 y Vision Transformers. Así como también el debido entrenamiento y muestra de resultados por cada modelo.

4.3.1. Construcción del modelo VGG16

Como primera etapa se detallará como se realizó la construcción del modelo VGG16, en la cual como primera instancia se realizó la búsqueda de hiperparámetros en la que se entrenó el modelo VGG16 aplicando la técnica de validación cruzada con un k = 5.

4.3.1.1. Implementación del modelo. En base a lo explicado en el capítulo de metodología se inició con la construcción del modelo VGG16, que es una variación del modelo base de red neuronal convolucional en la que su estructura de 16 capas profundas contiene los pesos características de clasificación de múltiple clases con respecto a la base datos de “ImageNet”. Debido a esto se procedió a construir el modelo VGG16 mediante “Trasferir Learning”.

Figura 84

Código de implementación del modelo vgg16 mediante aprendizaje por transferencia

```
[ ]  def modelo_vgg16(tasa_aprendizaje, dropout, fine_tune):

        base_convolucion = VGG16(include_top=False,
                         weights='imagenet',
                         input_shape=(224, 224, 3))

        if fine_tune > 0:
            for layer in base_convolucion.layers[:-fine_tune]:
                layer.trainable = False
        else:
            for layer in base_convolucion.layers:
                layer.trainable = False

        modelo_inter = base_convolucion.output
        modelo_inter = Flatten(name="flatten")(modelo_inter)
        modelo_inter = Dense(4096, activation='relu',kernel_regularizer=l2(0.001))(modelo_inter)
        modelo_inter = Dense(1072, activation='relu',kernel_regularizer=l2(0.001))(modelo_inter)
        modelo_inter = Dropout(dropout)(modelo_inter)
        capa_salida = Dense(2, activation='softmax')(modelo_inter)

        modelo_implement = Model(inputs=base_convolucion.input, outputs=capa_salida)

        modelo_implement.compile(optimizer=tasa_aprendizaje,
                      loss='categorical_crossentropy',
                      metrics=['accuracy'])

        return modelo_implement
```

Fuente: Elaboración propia

En la construcción de este modelo por aprendizaje por transferencia se agregó una capa “flatten” estructurar la salida de la base convolucional y se agregaron dos capas “densas” de 4096 y 1072 con una función “ReLU”. En estas capas densas se aplicó la técnica de regularización, que permite prevenir el sobre ajuste en el modelo. Es así que se agregó un regularizador “L2” con un valor de 0.001 a cada capa densa. Además, para mejorar el rendimiento del modelo también se aplicó la técnica de ajuste fine para establecer la cantidad de las últimas capas a ser entrenables. Adicionalmente, se agregó una capa densa que permitirá la salida de los dos tipos de clases a clasificar con una función de activación “softmax”.

Posteriormente antes de realizar el propio entrenamiento del modelo se realizó el proceso de búsqueda de mejores hiperparámetros en la que se importó la librería de “sklearn” donde se utilizó su componente “RandomizedSearchCV” que hará la búsqueda de los mejores hiperparámetros en la que se seleccionaron la tasa de aprendizaje, dropout, batch size y el número de épocas.

Figura 85

Código de hiperparámetros en variables temporales

```
learnRate = [0.01,0.001,0.0001]
dropout = [0.2, 0.3, 0.5]
batch_size = [8, 16, 32]
epochs = [60, 80, 120]

grid = dict(
  learnRate=learnRate,
  dropout=dropout,
  batch_size=batch_size,
  epochs=epochs
)
```

Fuente: Elaboración propia

Luego mediante el componente “KerasClassifier” se procedió a cargar el modelo de vgg16 y transformarlo en una estructura de modelo que pueda ser admitida en “RandomizedSearchCV”.

Figura 86

Código de instancia de la función “KerasClassifier”

```
model = KerasClassifier(build_fn=tuneo_model_vgg16, verbose=0)
```

Fuente: Elaboración propia

Por consiguiente se instanció la clase de “RandomizedSearchCV” y estableciendo la técnica de validación cruzada con un valor de 5 y midiendo el rendimiento mediante el indicador de “accuracy”.

Figura 87

Código de instancia de la función “RandomizedSearchCV”

```
searcher = RandomizedSearchCV(estimator=model, n_jobs=1, cv=5, param_distributions=grid, scoring="accuracy")
```

Fuente: Elaboración propia

Por último, se activa la GPU de Google Colab para que inicie el entrenamiento del modelo y búsqueda de los mejores hiperparámetros para el modelo de VGG16.

Figura 88

Código del entrenamiento del modelo en la búsqueda de mejores hiperparámetros

```
with tf.device('/device:GPU:0'):
  searchResults = searcher.fit(base_imagen_entrenamiento_total, etiquetas_entrenamiento_total)
```

Fuente: Elaboración propia

Luego del entrenamiento que duró aproximadamente 23 horas se obtuvo los mejores hiperparámetros para el modelo VGG16.

Figura 89

Código de impresión de mejores hiperparámetros del modelo VGG16

```
print("Best Score: ",
      searchResults.best_score_,
      "and Best Params: ",
      searchResults.best_params_)
```

```
Best Score:  0.9728846732911774 and Best Params:  {'learnRate': 0.0001, 'epochs': 80, 'dropout': 0.3, 'batch_size': 32}
```

Fuente: Elaboración propia

Con los valores obtenidos de los hiperparámetros se procedió a construir el modelo VGG16 con una validación cruzada de 5 y un valor de ajuste fino de 1. Estos valores se ingresaron de manera directa al implementar el modelo.

Figura 90

Código de construcción del modelo VGG16 con los hiperparámetros definidos

```
def modelo_vgg16(tasa_aprendizaje= 0.0001, fine_tune=1):

    base_convolucion = VGG16(include_top=False,
                     weights='imagenet',
                     input_shape=(150, 150, 3))

    if fine_tune > 0:
        for layer in base_convolucion.layers[:-fine_tune]:
            layer.trainable = False
    else:
        for layer in base_convolucion.layers:
            layer.trainable = False

    modelo_inter = base_convolucion.output
    modelo_inter = Flatten(name="flatten")(modelo_inter)
    modelo_inter = Dense(4096, activation='relu')(modelo_inter)
    modelo_inter = Dense(1072, activation='relu')(modelo_inter)
    modelo_inter = Dropout(0.3)(modelo_inter)
    capa_salida = Dense(2, activation='softmax')(modelo_inter)

    modelo_implement = Model(inputs=base_convolucion.input, outputs=capa_salida)

    modelo_implement.compile(optimizer=tasa_aprendizaje,
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

    return modelo_implement
```

Fuente: Elaboración propia

Luego se construyó el código para implementar la validación cruzada de particiones 5 con el modelo de VGG16.

Figura 91

Código de construcción de la validación cruzada y el entrenamiento del modelo VGG16.



```
# Dividir el conjunto de datos en k subconjuntos utilizando KFold
accuracy_list = []
y_real = []
y_predecido = []

kf = KFold(n_splits=5)
for fold, (train_index, val_index) in enumerate(kf.split(os.listdir(os.path.join(directorio_data, 'Aedes')))):


  # Crear directorios temporales para los datos de entrenamiento y validación
  train_dir = os.path.join(directorio_temporal, f'fold_{fold}', 'train')
  val_dir = os.path.join(directorio_temporal, f'fold_{fold}', 'val')
  os.makedirs(train_dir)
  os.makedirs(val_dir)


  with tf.device('/device:GPU:0'):
    vgg_ft_history = modelo_completo_vgg16.fit(entrenamiento_generador,
                                  batch_size=batch_size,
                                  epochs=epocas,
                                  validation_data=validacion_generador,
                                  steps_per_epoch=pasos,
                                  validation_steps=validation_steps,
                                  verbose=1)


  vgg16_predict = modelo_completo_vgg16.predict(test_generador)
  vgg16_predict_classes = np.argmax(vgg16_predict, axis=1)
  true_classes = test_generador.classes
  vgg_acc_ft = accuracy_score(true_classes, vgg16_predict_classes)
  y_real.append(true_classes)
  y_predecido.append(vgg16_predict_classes)
  accuracy_list.append(vgg_acc_ft)
  shutil.rmtree(os.path.join(directorio_temporal, f'fold_{fold}'))
  time.sleep(20)
  print(vgg_acc_ft)
```

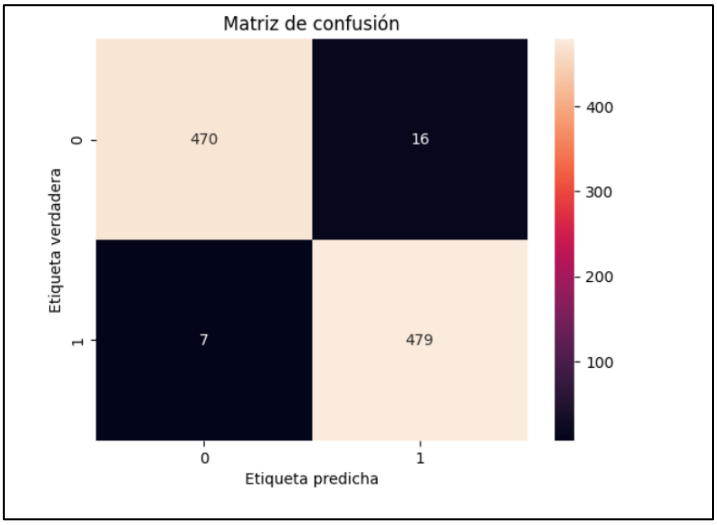
Fuente: Elaboración propia

4.3.2.2. Resultados del modelo. Al término del entrenamiento del modelo VGG16 con una validación cruzada de K=5, se obtuvo 5 indicadores de “accurracy” por cada iteración en la cual se aplicó el promedio para obtener el accuraccy final del modelo que fue de 97.59%.

Así mismo de cada iteración de validación cruzada también se obtuvo 5 matrices de confusión en la cual se sumaron estas matrices y se obtuvieron su promedio para obtener la matriz de confusión del modelo.

Figura 91

Matriz de confusión del modelo VGG16

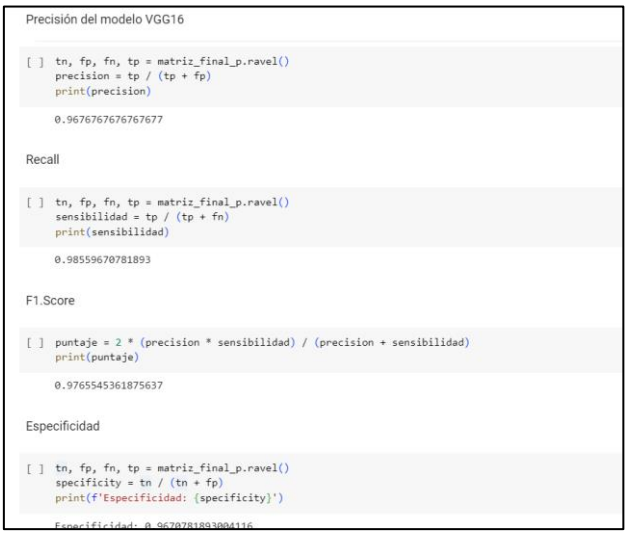


Fuente: Elaboración propia

A partir de esta matriz de se obtuvieron los valores de los otros 4 indicadores de evaluación del modelo. De precisión se obtuvo un 96.7%, de Recall se obtuvo un 98.5%, F1-Score se obtuvo 97.6% y un especificidad de 96.7%.

Figura 91

Resultados de los indicadores de rendimiento del modelo VGG16

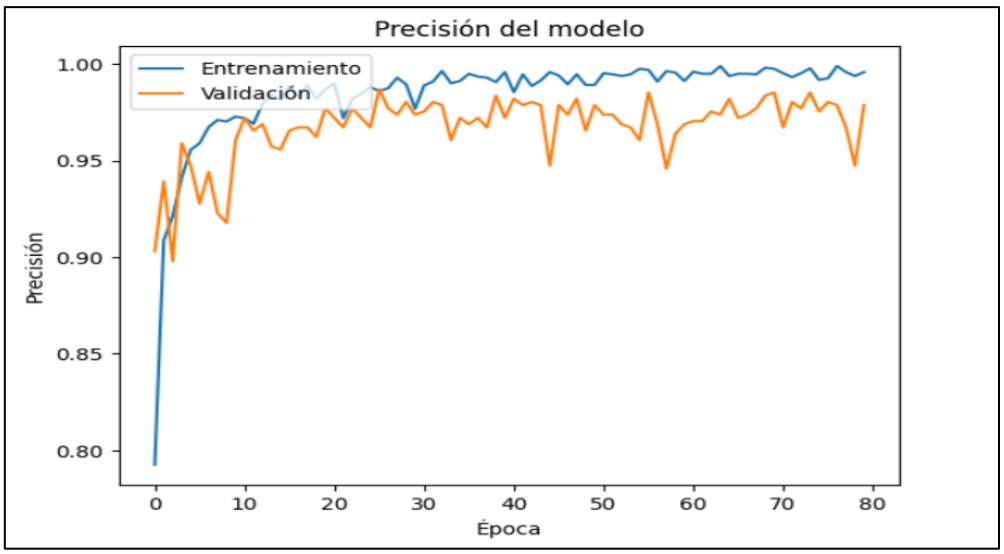


Fuente: Elaboración propia

En lo que respecta a la medición del “Accuracy” del entrenamiento y prueba en cada época. Así como también la perdida correspondiente en cada época, se presenta en las siguientes dos figuras.

Figura 92

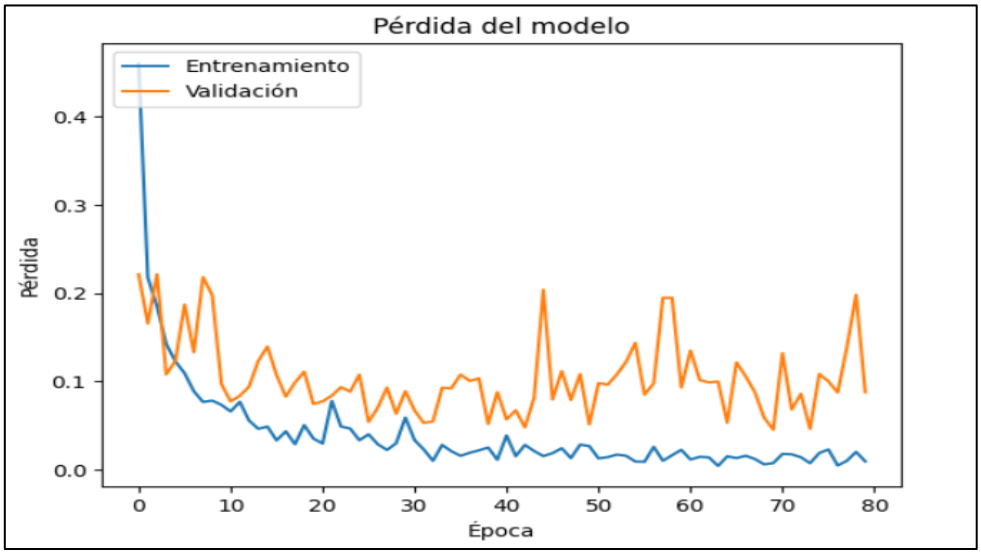
Gráfica de Accuracy por época del modelo VGG16



Fuente: Elaboración propia

Figura 93

Gráfica de precisión por época del modelo VGG16



Fuente: Elaboración propia

4.3.2. Construcción del modelo VGG19

Como primera etapa se detallará como se realizó la construcción del modelo VGG19, en la cual como primera instancia se realizó la búsqueda de hiperparámetros en la que se entrenó el modelo VGG19 aplicando la técnica de validación cruzada con un k = 5.

4.3.1.1. Implementación del modelo. En base a lo explicado en el capítulo de metodología se inició con la construcción del modelo VGG19, que es una variación del modelo base de red neuronal convolucional en la que su estructura de 19 capas profundas contiene los pesos características de clasificación de múltiple clases con respecto a la base datos de “ImageNet”. Debido a esto se procedió a construir el modelo VGG19 mediante “Trasferir Learning”.

Figura 94

Código de implementación del modelo VGG19 mediante aprendizaje por transferencia

```
[ ]  def tuneo_model_vgg19(dropout, learnRate,fine_tune):

        ase_convolucion = VGG19(include_top=False,
                          weights='imagenet',
                          input_tensor=image_input)

      ultima_capa = base_convolucion.layers[-1].output
      x= Flatten(name='flatten')(ultima_capa)
      x = Dense(128, activation='relu', name='fc1',kernel_regularizer=l2(0.001))(x)
      x=Dropout(dropout)(x)
      x = Dense(128, activation='relu', name='fc2',kernel_regularizer=l2(0.001))(x)
      x=Dropout(dropout)(x)
      salida = Dense(2, activation='softmax', name='output')(x)
      modelo_implement = Model(base_convolucion.input, salida)
      modelo_implement.summary()

      for layer in modelo_implement.layers[:-fine_tune]:
          layer.trainable = False

      modelo_implement.compile(loss='categorical_crossentropy',
                               optimizer=learnRate,
                               metrics=['accuracy'])


        return model
```

Fuente: Elaboración propia

En la construcción de este modelo por aprendizaje por transferencia se agregaron dos capas densas de 128 con una función de activación “ReLU” y una capa “dropout”. En estas capas densas se aplicó la técnica de regularización, que permite prevenir el sobre ajuste en el modelo. Es así que se agregó un regularizador “L2” con un valor de 0.001 a cada capa densa. Además, para mejorar el rendimiento del modelo también se aplicó la técnica de ajuste fine para establecer la cantidad de las últimas capas a ser entrenables. Por último se agregó una capa densa que permitirá la salida de los dos tipos de clases a clasificar.

Antes de realizar el propio entrenamiento del modelo se realizó el proceso de búsqueda de mejores hiperparámetros en la que se importó la librería de “sklearn” donde se utilizó su componente “RandomizedSearchCV” que hará la búsqueda de los mejores hiperparámetros en la que se seleccionaron la tasa de aprendizaje, dropout, batch size y el número de épocas.

Figura 94

Código de hiperparámetros en variables temporales del modelo VGG19

```
learnRate = [0.01,0.001,0.0001]
dropout = [0.2, 0.3, 0.5]
batch_size = [8, 16, 32]
epochs = [60, 80, 120]

grid = dict(
  learnRate=learnRate,
  dropout=dropout,
  batch_size=batch_size,
  epochs=epochs
)
```

Luego mediante el componente “KerasClassifier” se procedió a cargar el modelo de vgg19 y transformarlo en una estructura de modelo que pueda ser admitida en “RandomizedSearchCV”.

Figura 95

Código de instancia de la función “KerasClassifier” para el modelo VGG19

```
[ ]  model = KerasClassifier(build_fn=tuneo_model_vgg19, verbose=0)
```

Fuente: Elaboración propia

Por consiguiente se instanció la clase de “RandomizedSearchCV” y estableciendo la técnica de validación cruzada con un valor de 5 y midiendo el rendimiento mediante el indicador de “accuracy”.

Figura 96

Código de instancia de la función “RandomizedSearchCV” para el modelo VGG19

```
searcher = RandomizedSearchCV(estimator=model, n_jobs=1, cv=5, param_distributions=grid, scoring="accuracy")
```

Fuente: Elaboración propia

Por último, se activa la GPU de Google Colab para que inicie el entrenamiento del modelo y búsqueda de los mejores hiperparámetros para el modelo de VGG19.

Figura 97

Código del entrenamiento del modelo en la búsqueda de mejores hiperparámetros.

```
with tf.device('/device:GPU:0'):
  searchResults = searcher.fit(base_imagen_entrenamiento_total, etiquetas_entrenamiento_total)
```

Fuente: Elaboración propia

Luego del entrenamiento que duró aproximadamente 24 horas se obtuvo los mejores hiperparámetros para el modelo vgg19.

Figura 98

Código de impresión de mejores hiperparámetros del modelo VGG19

```
print("Best Score: ",
      searchResults.best_score_,
      "and Best Params: ",
      searchResults.best_params_)
```

```
Best Score:  0.97945846737535 and Best Params:  {'learnRate': 0.001, 'epochs': 80, 'dropout': 0.3, 'batch_size': 32}
```

Fuente: Elaboración propia

Con los valores obtenidos de los hiperparámetros se procedió a construir el modelo VGG19 con una validación cruzada de 5 y un valor de ajuste fino de 6. Estos valores se ingresaron de manera directa al implementar el modelo.

Figura 99

Código de construcción del modelo VGG19 con los hiperparámetros definidos

```
image_input = Input(shape=(224, 224, 3))
```

```
[ ]  def modelo_vgg19(tasa_aprendizaje):

       base_convolucion = VGG19(include_top=False,
                                weights='imagenet',
                                input_tensor=image_input)

       ultima_capa = base_convolucion.layers[-1].output
       x= Flatten(name='flatten')(ultima_capa)
       x = Dense(128, activation='relu', name='fc1')(x)
       x=Dropout(0.3)(x)
       x = Dense(128, activation='relu', name='fc2')(x)
       x=Dropout(0.3)(x)
       salida = Dense(2, activation='softmax', name='output')(x)
       modelo_implement = Model(base_convolucion.input, salida)
       modelo_implement.summary()

       for layer in modelo_implement.layers[:-6]:
           layer.trainable = False

       modelo_implement.compile(loss='categorical_crossentropy',
                                optimizer=tasa_aprendizaje,
                                metrics=['accuracy'])

       return modelo_implement
```

```
[ ]  tasa_aprendizaje = Adam(learning_rate=0.001)
     modelo_completo_vgg19 = modelo_vgg19(tasa_aprendizaje)
```

Fuente: Elaboración propia

Luego se construyó el código para implementar la validación cruzada de 5, utilizando la librería “KFold” en el modelo de VGG19.

Figura 100

Código de construcción de la validación cruzada con k=5 para el modelo VGG19

```
# Dividir el conjunto de datos en k subconjuntos utilizando KFold
accuracy_list = []
y_real = []
y_predecido = []


kf = KFold(n_splits=5)
for fold, (train_index, val_index) in enumerate(kf.split(os.listdir(os.path.join(directorio_data, 'Aedes')))):
```

Fuente: Elaboración propia

Posteriormente en el bucle “for” de la función “KFold” se agregó el entrenamiento del modelo en la que se guardó en cada iteración las matrices de confusión y el “accuracy” para luego realizar su correspondiente promedio.

Figura 101

Código del entrenamiento del modelo VGG19

```
with tf.device('/device:GPU:0'):
  vgg_ft_history = modelo_completo_vgg19.fit(entrenamiento_generador,
                                batch_size=batch_size,
                                epochs=epocas,
                                validation_data=validacion_generador,
                                steps_per_epoch=pasos,
                                validation_steps=validation_steps,
                                verbose=1)


vgg19_predict = modelo_completo_vgg19.predict(test_generador)
vgg19_predict_classes = np.argmax(vgg19_predict, axis=1)
true_classes = test_generador.classes
vgg_acc_ft = accuracy_score(true_classes, vgg19_predict_classes)
y_real.append(true_classes)
y_predecido.append(vgg19_predict_classes)
accuracy_list.append(vgg_acc_ft)
shutil.rmtree(os.path.join(directorio_temporal, f'fold_{fold}'))
time.sleep(20)
print(vgg_acc_ft)
```

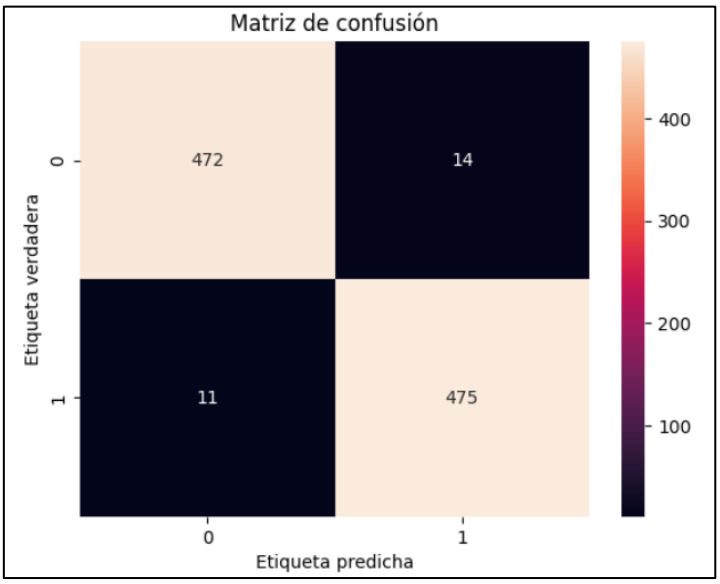
Fuente: Elaboración propio

4.3.2.2. Resultados del modelo. Al término del entrenamiento del modelo VGG19 con una validación cruzada de K=5, se obtuvo 5 indicadores de “accurracy” por cada iteración en la cual se aplicó el promedio para obtener el accuraccy final del modelo que fue de 97.74%.

Así mismo de cada iteración de validación cruzada también se obtuvo 5 matrices de confusión en la cual se sumaron estas matrices y se obtuvieron su promedio para obtener la matriz de confusión del modelo.

Figura 102

Matriz de confusión del modelo VGG19

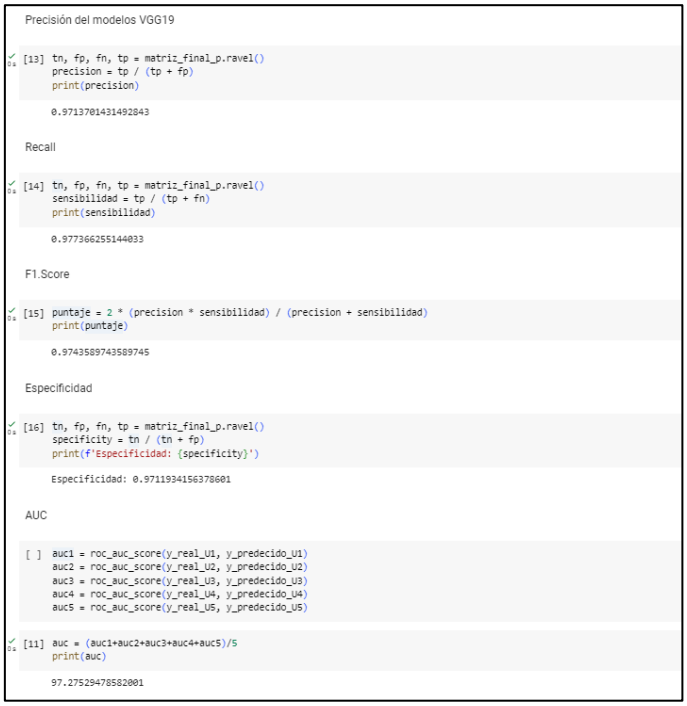


Fuente: Elaboración propia

A partir de esta matriz de se obtuvieron los valores de los otros 4 indicadores de evaluación del modelo. De precisión se obtuvo un 97.13%, de Recall se obtuvo un 97.73%, F1-Score se obtuvo 97.43%, especificidad de 97.11% y AUC de 97.27%.

Figura 103

Resultados de los indicadores de rendimiento del modelo VGG19



Precisión del modelos VGG19

```
[13] tn, fp, fn, tp = matriz_final_p.ravel()
     precision = tp / (tp + fp)
     print(precision)

     0.9713701431492843
```

Recall

```
[14] tn, fp, fn, tp = matriz_final_p.ravel()
     sensibilidad = tp / (tp + fn)
     print(sensibilidad)

     0.977366255144033
```

F1.Score

```
[15] puntaje = 2 * (precision * sensibilidad) / (precision + sensibilidad)
     print(puntaje)

     0.9743589743589745
```

Especificidad

```
[16] tn, fp, fn, tp = matriz_final_p.ravel()
     specificity = tn / (tn + fp)
     print(f'Especificidad: {specificity}')

     Especificidad: 0.9711934156378601
```

AUC

```
[ ]  auc1 = roc_auc_score(y_real_U1, y_predecido_U1)
     auc2 = roc_auc_score(y_real_U2, y_predecido_U2)
     auc3 = roc_auc_score(y_real_U3, y_predecido_U3)
     auc4 = roc_auc_score(y_real_U4, y_predecido_U4)
     auc5 = roc_auc_score(y_real_U5, y_predecido_U5)

[11] auc = (auc1+auc2+auc3+auc4+auc5)/5
     print(auc)

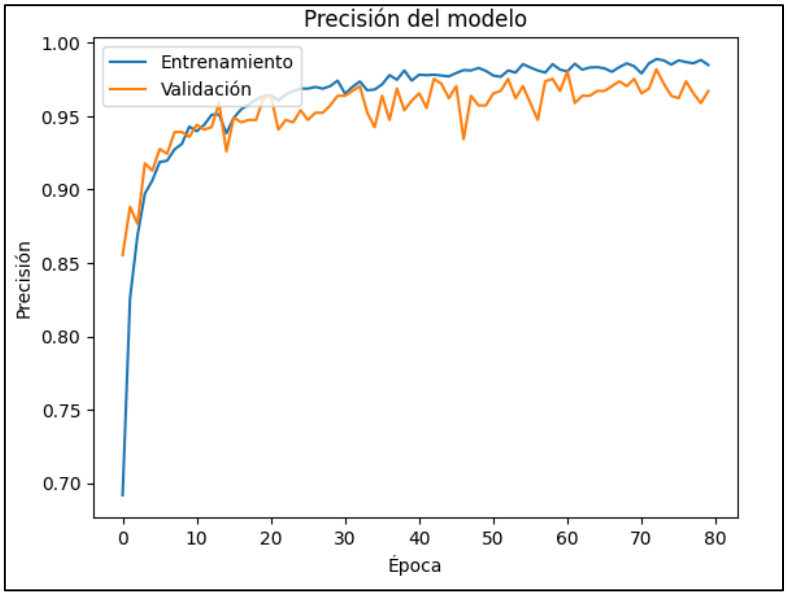
     97.27529478582001
```

Fuente: Elaboración propia

En lo que respecta a la medición del “Accuracy” del entrenamiento y prueba en cada época. Así como también la perdida correspondiente en cada época, se presenta en las siguientes dos imágenes.

Figura 104

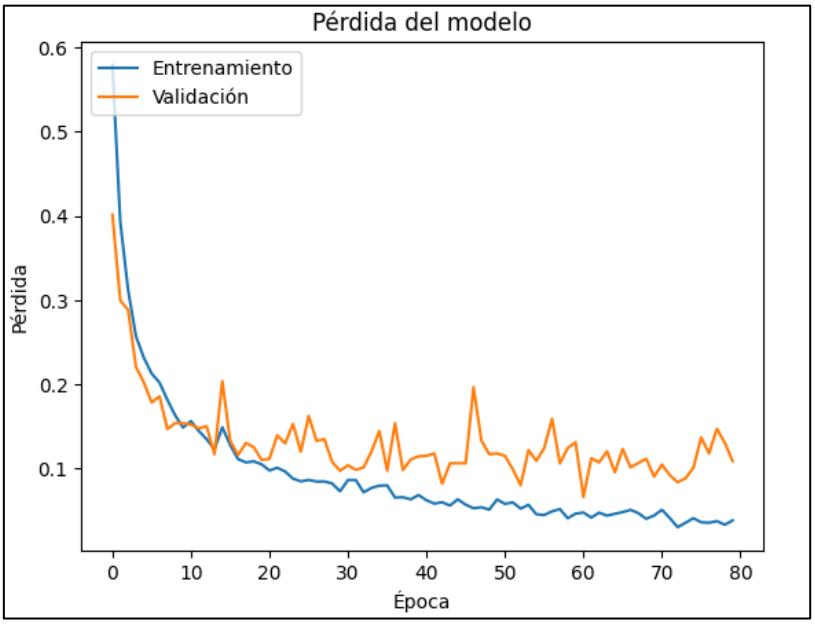
Gráfica de Accuracy por época del modelo VGG19



Fuente: Elaboración propia

Figura 105

Gráfica de la pérdida por época del modelo VGG19



Fuente: Elaboración propia

4.3.3. Construcción del modelo ResNet50

Como primera etapa se detallará como se realizó la construcción del modelo ResNet50, en la cual como primera instancia se realizó la búsqueda de hiperparámetros en la que se entrenó el modelo ResNet50 aplicando la técnica de validación cruzada con un k = 5.

4.3.1.1. Implementación del modelo. En base a lo explicado en el capítulo de metodología como tercer modelo se realizó la construcción del modelo ResNet50, que es una variación del modelo base de red neuronal convolucional en la que su estructura de 50 capas profundas contiene los pesos características de clasificación de múltiple clases con respecto a la base datos de “ImageNet”. Debido a esto se procedió a construir el modelo ResNet50 mediante “Trasferir Learning”.

Figura 106

Código de implementación del modelo ResNet50 mediante aprendizaje por transferencia.

```
[ ]  def modelo_ResNet50(tasa_aprendizaje,dropout,fine_tune):
       m_Resnet50 = ResNet50(input_tensor=image_input,
                             include_top=False,
                             weights='imagenet')


       ultima_capa = m_Resnet50.layers[-1].output

       capa_g= Flatten(name='flatten')(ultima_capa)
       capa_g = Dense(4096, activation='relu', name='fc1',kernel_regularizer=l2(0.01))(capa_g)
       capa_g = Dense(1072, activation='relu', name='fc2',kernel_regularizer=l2(0.01))(capa_g)
       capa_g=Dropout(dropout)(capa_g)
       salida = Dense(2, activation='softmax', name='output')(capa_g)
       modelo_final = Model(image_input, salida)
       modelo_final.summary()


       for layer in modelo_final.layers[:-fine_tune]:
         layer.trainable = False


       modelo_final.compile(loss='categorical_crossentropy',optimizer=tasa_aprendizaje,metrics=['accuracy'])

       return modelo_final
```

Fuente: Elaboración propia

En la construcción de este modelo por aprendizaje por transferencia se agregó una capa “flatten” estructurar la salida de la base convolucional y se agregaron dos capas

“densas” de 4096 y 1072 con una función “ReLU”. En estas capas densas se aplicó la técnica de regularización, que permite prevenir el sobre ajuste en el modelo. Es así que se agregó un regularizador “L2” con un valor de 0.01 a cada capa densa. Adicionalmente, se agregó una capa “dropout” para controlar también el sobre ajuste. Además, para mejorar el rendimiento del modelo también se aplicó la técnica de ajuste fine para establecer la cantidad de las últimas capas a ser entrenables. Por último se agregó una capa densa que permitirá la salida de los dos tipos de clases a clasificar.

Posteriormente antes de realizar el propio entrenamiento del modelo se realizó el proceso de búsqueda de mejores hiperparámetros en la que se importó la librería de “sklearn” donde se utilizó su componente “RandomizedSearchCV” que hará la búsqueda de los mejores hiperparámetros en la que se seleccionaron la tasa de aprendizaje, dropout, batch size y el número de épocas.

Figura 107

Código de hiperparámetros en variables temporales del modelo ResNet50



```
learnRate = [0.01,0.001,0.0001]
dropout = [0.2, 0.3, 0.5]
batch_size = [8, 16, 32]
epochs = [60, 80, 120]

grid = dict(
  learnRate=learnRate,
  dropout=dropout,
  batch_size=batch_size,
  epochs=epochs
)
```

Fuente: Elaboración propia

Luego mediante el componente “KerasClassifier” se procedió a cargar el modelo de ResNet50 y transformarlo en una estructura de modelo que pueda ser admitida en “RandomizedSearchCV”.

Figura 108

Código de instancia de la función “KerasClassifier” para el modelo ResNet50

```
model = KerasClassifier(build_fn=tuneo_model_resnet50, verbose=0)
```

Fuente: Elaboración propia

Por consiguiente se instanció la clase de “RandomizedSearchCV” y se estableció una validación cruzada de 5 y se consideró el “Accuracy” como score para el rendimiento del modelo.

Figura 109

Código de instancia de la función “RandomizedSearchCV” para el modelo ResNet50

```
searcher = RandomizedSearchCV(estimator=model, n_jobs=1, cv=5, param_distributions=grid, scoring="accuracy")
```

Fuente: Elaboración propia

Por último, se activa la GPU de Google Colab para que inicie el entrenamiento del modelo y búsqueda de los mejores hiperparámetros para el modelo de ResNet50.

Figura 110

Código del entrenamiento del modelo en la búsqueda de mejores hiperparámetros ResNet50

```
with tf.device('/device:GPU:0'):
  searchResults = searcher.fit(base_imagen_entrenamiento_total, etiquetas_entrenamiento_total)
```

Fuente: Elaboración propia

Luego del entrenamiento que duró aproximadamente 27 horas se obtuvo los mejores hiperparámetros para el modelo ResNet50.

Figura 111

Código de impresión de mejores hiperparámetros del modelo ResNet50

```
[ ]  print("Best Score: ",
           searchResults.best_score_,
           "and Best Params: ",
           searchResults.best_params_)

     Best Score:  0.9784442351673432 and Best Params:  {'learnRate': 0.0001, 'epochs': 80, 'dropout': 0.2, 'batch_size': 32}
```

Fuente: Elaboración propia

Con los valores obtenidos de los hiperparámetros se procedió a construir el modelo ResNet50 con una validación cruzada de 5 y un valor de ajuste fino de 2. Estos valores se ingresaron de manera directa al implementar el modelo.

Figura 112

Código de construcción del modelo ResNet50 con los hiperparámetros definidos

```
[ ]  def modelo_ResNet50(tasa_aprendizaje=Adam(learning_rate=0.0001),fine_tune=2):
       m_Resnet50 = ResNet50(input_tensor=image_input,
                             include_top=False,
                             weights='imagenet')


       ultima_capa = m_Resnet50.layers[-1].output

       capa_g= Flatten(name='flatten')(ultima_capa)
       capa_g = Dense(4096, activation='relu', name='fc1',kernel_regularizer=l2(0.01))(capa_g)
       capa_g = Dense(1072, activation='relu', name='fc2',kernel_regularizer=l2(0.01))(capa_g)
       capa_g=Dropout(0.2)(capa_g)
       salida = Dense(2, activation='softmax', name='output')(capa_g)
       modelo_final = Model(image_input, salida)
       modelo_final.summary()


       for layer in modelo_final.layers[:-fine_tune]:
         layer.trainable = False

       modelo_final.compile(loss='categorical_crossentropy',optimizer=tasa_aprendizaje,metrics=['accuracy'])

       return modelo_final
```

Fuente: Elaboración propia

Luego se construyó el código para implementar la validación cruzada de 5, utilizando la librería “KFold” en el modelo de ResNet50.

Figura 113

Código de construcción de la validación cruzada con k=5 para el modelo ResNet50

```
# Dividir el conjunto de datos en k subconjuntos utilizando KFold
accuracy_list = []
y_real = []
y_predecido = []


kf = KFold(n_splits=5)
for fold, (train_index, val_index) in enumerate(kf.split(os.listdir(os.path.join(directorio_data, 'Aedes')))):
```

Fuente: Elaboración propia

Posteriormente en el bucle “for” de la función “KFold” se agregó el entrenamiento del modelo ResNet50 en la que se guardó en cada iteración las matrices de confusión y el “accuracy”. Para luego realizar su correspondiente promedio.

Figura 114

Código del entrenamiento del modelo ResNet50

```
with tf.device('/device:GPU:0'):
  vgg_ft_history = modelo_completo_ResNet50.fit(entrenamiento_generador,
                                batch_size=batch_size,
                                epochs=epocas,
                                validation_data=validacion_generador,
                                steps_per_epoch=pasos,
                                validation_steps=validation_steps,
                                verbose=1)


ResNet50_predict = modelo_completo_ResNet50.predict(test_generador)
ResNet50_predict_classes = np.argmax(ResNet50_predict, axis=1)
true_classes = test_generador.classes
vgg_acc_ft = accuracy_score(true_classes, ResNet50_predict_classes)
y_real.append(true_classes)
y_predecido.append(ResNet50_predict_classes)
accuracy_list.append(vgg_acc_ft)
shutil.rmtree(os.path.join(directorio_temporal, f'fold_{fold}'))
time.sleep(20)
print(vgg_acc_ft)
```

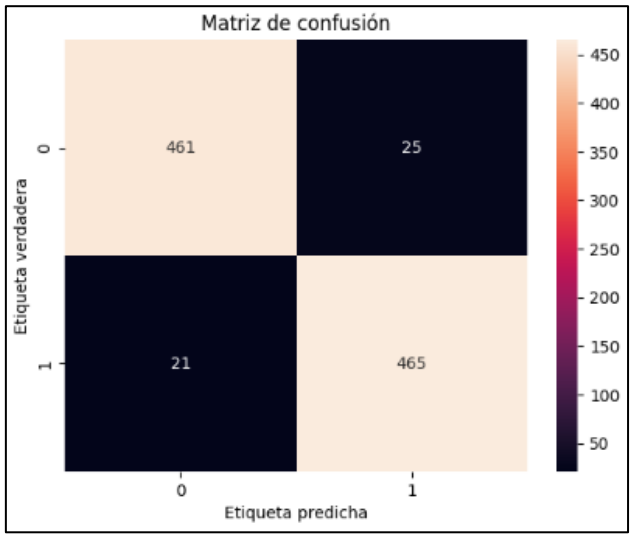
Fuente: Elaboración propia

4.3.2.2. Resultados del modelo. Al término del entrenamiento del modelo ResNet50 con una validación cruzada de K=5, se obtuvo 5 indicadores de “accurracy” por cada iteración en la cual se aplicó el promedio para obtener el accuraccy final del modelo que fue de 95.27%.

Así mismo de cada iteración de validación cruzada también se obtuvo 5 matrices de confusión en la cual se sumaron estas matrices y se obtuvieron su promedio para obtener la matriz de confusión del modelo.

Figura 115

Matriz de confusión del modelo ResNet50



Fuente: Elaboración propia

A partir de esta matriz de se obtuvieron los valores de los otros 4 indicadores de evaluación del modelo. De precisión se obtuvo un 94.89%, de Recall se obtuvo un 95.67%, F1-Score se obtuvo 95.28%, especificidad de 94.85% y AUC de 95.27%.

Figura 116

Resultados de los indicadores de rendimiento del modelo ResNet50

Precisión del modelo ResNet50

```
[ ]  tn, fp, fn, tp = matriz_final_p.ravel()
     precision = tp / (tp + fp)
     print(precision)

     0.9489795918367347
```

Recall

```
[ ]  tn, fp, fn, tp = matriz_final_p.ravel()
     sensibilidad = tp / (tp + fn)
     print(sensibilidad)

     0.9567901234567902
```

F1.Score

```
[ ]  puntaje = 2 * (precision * sensibilidad) / (precision + sensibilidad)
     print(puntaje)

     0.9528688524590165
```

Especificidad

```
[ ]  tn, fp, fn, tp = matriz_final_p.ravel()
     specificity = tn / (tn + fp)
     print(f'Especificidad: {specificity}')

     Especificidad: 0.948559670781893
```

AUC

```
[ ]  auc1 = roc_auc_score(y_real_U1, y_predecido_U1)
     auc2 = roc_auc_score(y_real_U2, y_predecido_U2)
     auc3 = roc_auc_score(y_real_U3, y_predecido_U3)
     auc4 = roc_auc_score(y_real_U4, y_predecido_U4)
     auc5 = roc_auc_score(y_real_U5, y_predecido_U5)

[ ]  auc = (auc1+auc2+auc3+auc4+auc5)/5
     print(auc)

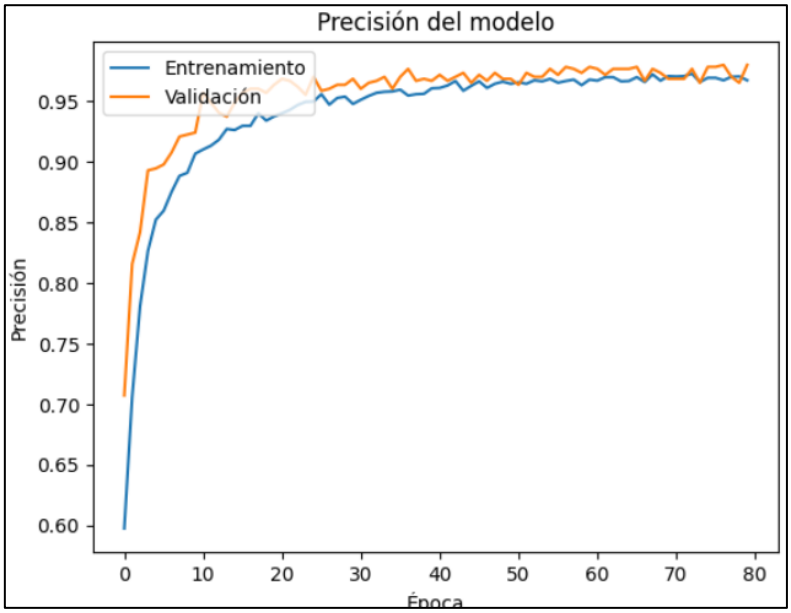
     0.9527496810065827
```

Fuente: Elaboración propia

Para la medición del Accuracy del entrenamiento y prueba en cada época. Así como también la pérdida correspondiente en cada época, se presenta en las siguientes gráficas.

Figura 117

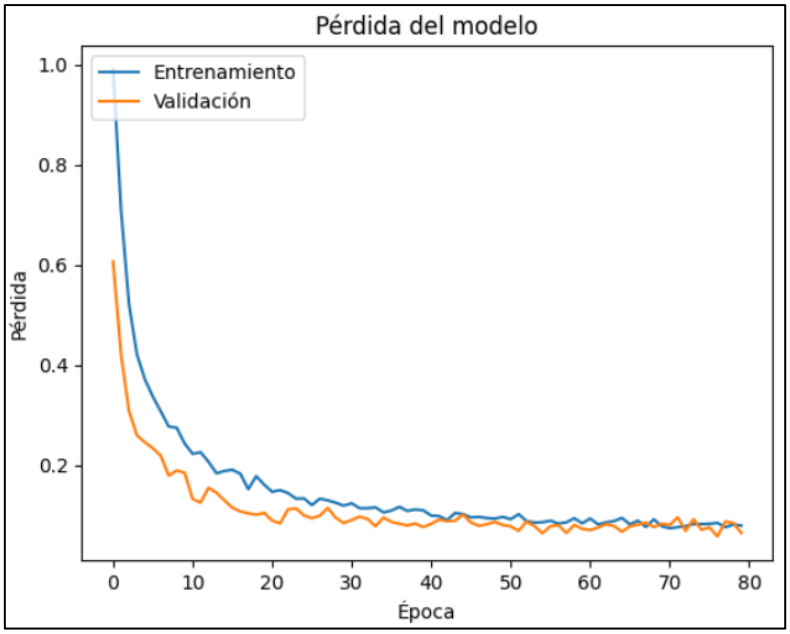
Gráfica de Accuracy por época del modelo ResNet50



Fuente: Elaboración Propia

Figura 118

Gráfica de la pérdida por época del modelo ResNet50



Fuente: Elaboración Propia

4.3.4. Construcción del modelo Vision Transformers

Como primera etapa se detallará como se realizó la construcción del modelo Vision Transformers, en la cual como primera instancia se realizó la búsqueda de hiperparámetros en la que se entrenó el modelo aplicando la técnica de validación cruzada con un k = 5.

4.3.1.1. Implementación del modelo. En base a lo explicado en el capítulo de metodología como cuarto modelo se realizó la construcción del modelo Vision Transformers, que tiene una estructura muy distinta a la de las variantes de las redes convolucionales. Sin embargo, esta red también se entrenó en base a un número mayor de datos de "ImageNet" en la que guarda también los pesos característicos del mencionado entrenamiento. Debido a esto se procedió a construir este modelo mediante "Trasferir Learning".

Figura 119

Código de definición del modelo de Vision Transformers mediante aprendizaje por transferencia.

```
[ ]  # Definición del modelo
     class ViT(nn.Module):
         def __init__(self):
             super(ViT, self).__init__()
             self.vit = timm.create_model('vit_base_patch16_224', pretrained=True)
             self.dropout = nn.Dropout(0.3)
             self.fc = nn.Linear(1000, 2)
         def forward(self, x):
             x = self.vit(x)
             x = self.fc(x)
             return x

[ ]  model = ViT()

[ ]  peso_bajada = 1e-4
     optimizer = Adam(model.parameters(), weight_decay=peso_bajada)
```

Fuente: Elaboración propia

En la construcción de este modelo por aprendizaje por transferencia se cargó los pesos pre entrenados configurados solo para imágenes de 224x224 mediante la librería de Pytorch.

Además se agregó una capa “dropout” para controlar el sobre ajuste. Adicionalmente se configuró también la técnica de regularización en el parámetro de optimizador del modelo con un valor de “1e-4” y por último se agregó una capa densa que configura la salida de las dos clases a clasificar.

Antes de realizar el propio entrenamiento del modelo se realizó el proceso de búsqueda de mejores hiperparámetros que corresponden a la taza de aprendizaje, dropout, batch size y el número de épocas.

Figura 120

Código de hiperparámetros en variables temporales del modelo Vision Transformers

```
learnRate = [0.01,0.001,0.0001]
dropout = [0.2, 0.3, 0.5]
batch_size = [8, 16, 32]
epochs = [60, 80, 120]

grid = dict(
  learnRate=learnRate,
  dropout=dropout,
  batch_size=batch_size,
  epochs=epochs
)
```

Fuente: Elaboración de propia

Luego se utilizó la librería “KFold” para aplicar la técnica de validación cruzada con un K=5 en la búsqueda de hiperparámetros.

Figura 121

Código de construcción de la validación cruzada con k=5 para el modelo Vision Transformers

```
kf = KFold(n_splits=5)
for fold, (train_index, val_index) in enumerate(kf.split(os.listdir(os.path.join(directorio_data, 'Aedes')))):
```

Fuente: Elaboración propia

Dentro de ese bucle se agregó los bucles de “for” por cada búsqueda de hiperparámetros considerando en cada iteración los 5 grupos por la validación cruzada.

Figura 122

Código de hiperparámetros en variables temporales del modelo Vision Transformers

```
[ ]  # Optimización de Hyperparametros y Cros-Validation
     accuracy_mean = []
     hyper_param = []
     for l in learnRate:
       for j in dropout:
         for h in batch_size:
           for e in epochs:
             accuracy_vit = []
               accuracy_vit.append(tuneo_model_ViT(data_entrenamiento,data_validacion,class_subset,l,j,h,e))
               eliminar_imagenes()
               time.sleep(30)
             accuracy_mean.append((sum(accuracy_vit)/len(accuracy_vit))*100)
             hyper_param.append([l,j,h,e])
```

Fuente: Elaboración propia

Luego del entrenamiento que duró aproximadamente 25 horas se obtuvo los mejores hiperparámetros para el modelo Vision Transformers.

Figura 123

Código de impresión de mejores hiperparámetros del modelo Vision Transformers

```
print(hyper_param)

{'lr': 0.0001, 'dropout': 0.2, 'batch_size': 8, 'epocas': 80}
```

Fuente: Elaboración propia

Con los valores obtenidos de los hiperparámetros se procedió a construir el modelo Vision Transformers. Estos valores se ingresaron de manera directa al implementar el modelo.

Figura 124

Código de entrenamiento y validación del modelo de Vision Transformers.

```
for epoch in range(num_epochs):
    # Carga del dataset del entrenamiento
    model.train()
    running_loss = 0.0
    for inputs, labels in train_dataloader:
        # Carga a la GPU:
        inputs = inputs.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        labels_one_hot = F.one_hot(labels, num_classes=2).float()
        loss = criterion(outputs, labels_one_hot)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    train_loss = running_loss / len(train_dataloader)

    # Validación por época
    model.eval()
    running_loss = 0.0
    correct_predictions = 0
    with torch.no_grad():
        for inputs, labels in val_dataloader:
            # Carga a la GPU
            inputs = inputs.to(device)
            labels = labels.to(device)

            outputs = model(inputs)
            labels_one_hot = F.one_hot(labels, num_classes=2).float()
            loss = criterion(outputs, labels_one_hot)
            running_loss += loss.item()
            preds = torch.argmax(outputs, dim=1)
            correct_predictions += (preds == labels).sum().item()
    val_loss = running_loss / len(val_dataloader)
    val_acc = correct_predictions / len(val_dataset)
```

Fuente: Elaboración propia

Luego al inicio del código de la implementación del modelo se agregó la validación cruzada de 5, utilizando la librería “KFold” en el modelo de Vision Transformers.

Figura 125

Código de construcción de la validación cruzada con k=5 para el modelo Vision Transformers con las variables definidas

```
# Dividir el conjunto de datos en k subconjuntos utilizando KFold
accuracy_list = []
y_real = []
y_predecido = []


kf = KFold(n_splits=5)
for fold, (train_index, val_index) in enumerate(kf.split(os.listdir(os.path.join(directorio_data, 'Aedes')))):
```

Fuente: Elaboración propia

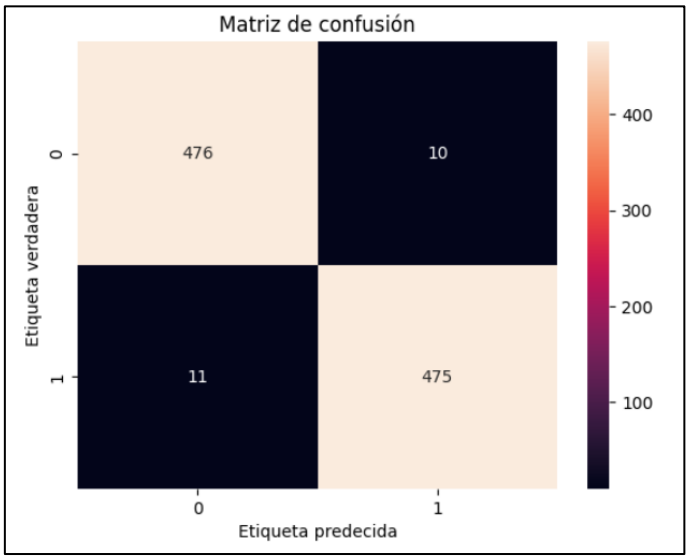
Posteriormente en el bucle “for” de la función “KFold” se agregó el entrenamiento del modelo Vision Transformers en la que se guardó en cada iteración las matrices de confusión y el “Accuracy”. Para luego realizar su correspondiente promedio.

4.3.2.2. Resultados del modelo. Al término del entrenamiento del modelo Vision Transformers con una validación cruzada de K=5, se obtuvo 5 indicadores de “accurracy” por cada iteración en la cual se aplicó el promedio para obtener el accuraccy final del modelo que fue de 97.98%.

Así mismo de cada iteración de validación cruzada también se obtuvo 5 matrices de confusión en la cual se sumaron estas matrices y se obtuvieron su promedio para obtener la matriz de confusión del modelo.

Figura 126

Matriz de confusión de Vision Transformers

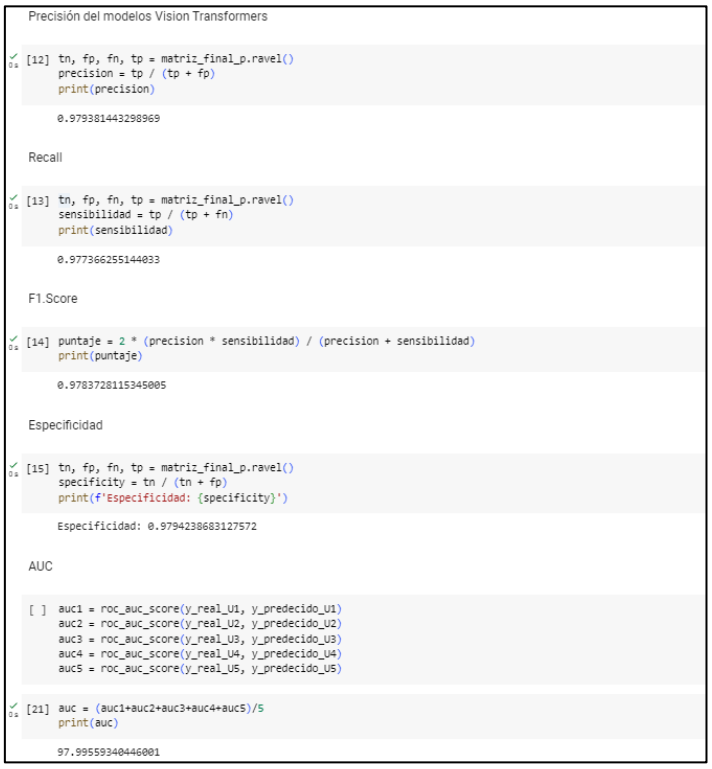


Fuente: Elaboración propia

A partir de esta matriz de se obtuvieron los valores de los otros 4 indicadores de evaluación del modelo. De precisión se obtuvo un 97.93%, de Recall se obtuvo un 97.73%, F1-Score se obtuvo 97.83%, especificidad de 97.94% y AUC de 97.99%.

Figura 127

Resultados de los indicadores de rendimiento del modelo Vision Transformers



Precisión del modelos Vision Transformers

```
[12] tn, fp, fn, tp = matriz_final_p.ravel()
     precision = tp / (tp + fp)
     print(precision)

     0.979381443298969
```

Recall

```
[13] tn, fp, fn, tp = matriz_final_p.ravel()
     sensibilidad = tp / (tp + fn)
     print(sensibilidad)

     0.977366255144033
```

F1.Score

```
[14] puntaje = 2 * (precision * sensibilidad) / (precision + sensibilidad)
     print(puntaje)

     0.9783728115345005
```

Especificidad

```
[15] tn, fp, fn, tp = matriz_final_p.ravel()
     specificity = tn / (tn + fp)
     print(f'Especificidad: {specificity}')

     Especificidad: 0.9794238683127572
```

AUC

```
[ ]  auc1 = roc_auc_score(y_real_U1, y_predecido_U1)
     auc2 = roc_auc_score(y_real_U2, y_predecido_U2)
     auc3 = roc_auc_score(y_real_U3, y_predecido_U3)
     auc4 = roc_auc_score(y_real_U4, y_predecido_U4)
     auc5 = roc_auc_score(y_real_U5, y_predecido_U5)
```

```
[21] auc = (auc1+auc2+auc3+auc4+auc5)/5
     print(auc)

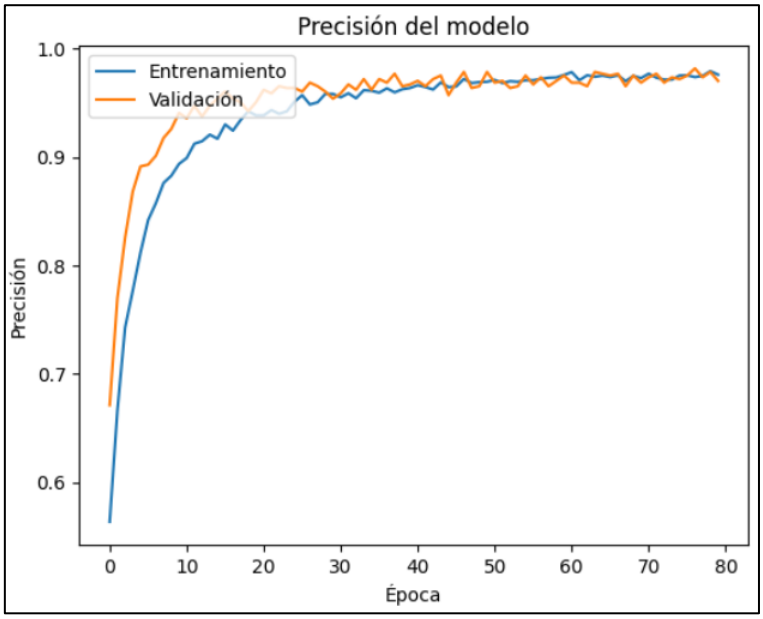
     97.99559340446001
```

Fuente: Elaboración propia

Para la medición del Accuracy del entrenamiento y prueba en cada época. Así como también la pérdida correspondiente en cada época, se presenta en las siguientes gráficas.

Figura 128

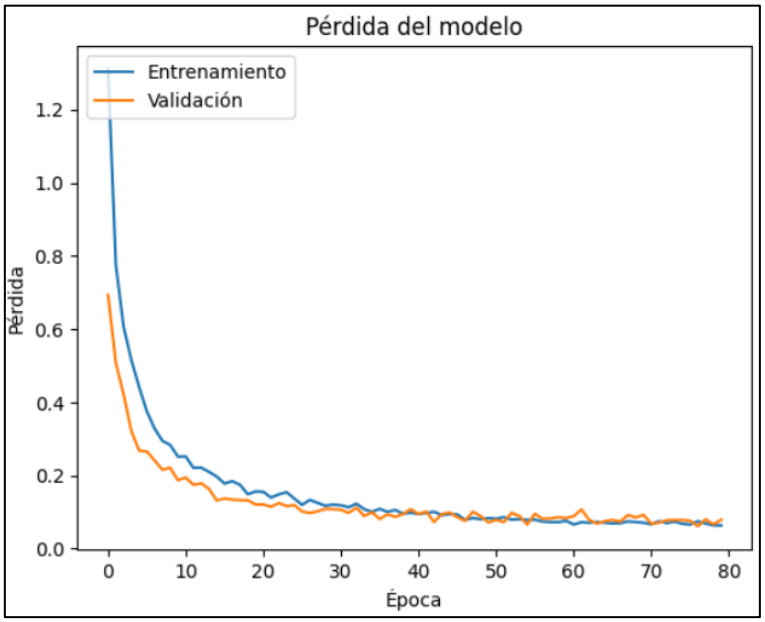
Gráfica de Accuracy por época del modelo Vision Transformers



Fuente: Elaboración propia

Figura 129

Gráfica de la pérdida por época del modelo Vision Transformers



Fuente: Elaboración propia

4.4. Análisis de los resultados de los modelos implementados

Los resultados obtenidos en cada modelo de inteligencia artificial revela la capacidad de clasificación en base a las imágenes obtenidas en esta investigación. A continuación, se muestra una tabla de resumen como un comparativo de los 4 modelos implementados.

Tabla 14

Resumen de resultados de indicadores

Modelos	Accuracy	Precisión	Recall	F1-Score	Especificidad	AUC
VGG16	97.59%	96.70%	98.50%	97.60%	96.70%	97.50%
VGG19	97.74%	97.13%	97.73%	97.43%	97.11%	97.27%
ResNet50	95.27%	94.89%	95.67%	95.28%	94.85%	95.27%
Vision Transformers	97.98%	97.93%	97.73%	97.83%	97.94%	97.99%

Fuente: Elaboración propia

4.4.1. Resultados del modelo VGG16

Los indicadores de rendimiento del modelo VGG16, revela que el modelo ha clasificado un 97.59% total de valores correctamente clasificados, tanto de las clases de Aedes como de Culex. Así mismo tiene una precisión del 96.70% y un AUC de 97.50%.

4.4.2. Resultado del modelo VGG19

Los indicadores de rendimiento del modelo VGG19, revela que el modelo ha clasificado un 97.74% total de valores correctamente clasificados, tanto de las clases de Aedes como de Culex. Así mismo tiene una precisión del 97.13% y un AUC de 97.27%.

4.4.3. Resultado del modelo ResNet50

Los indicadores de rendimiento del modelo ResNet50, revela que el modelo ha clasificado un 95.27% total de valores correctamente clasificados, tanto de las clases de Aedes como de Culex. Así mismo tiene una precisión del 94.89% y un AUC de 95.27%.

4.4.4. Resultado del modelo Vision Transformers

Los indicadores de rendimiento del modelo Vision Transformers, revela que el modelo ha clasificado un 97.98% total de valores correctamente clasificados, tanto de las clases de Aedes como de Culex. Así mismo tiene una precisión del 97.93% y un AUC de 97.99%.

Con estos resultados se revela que el modelo que tiene mayor rendimiento con respecto a los indicadores más importantes como es el Accuracy, la precisión y el AUC es el de visión Transformers y es óptimo para la clasificación de este tipo de imágenes. Así mismo comparándolo con el segundo mejor modelo que es el VGG19, el modelo de Visión Transformers tienen mejor rendimiento de Accuracy por época con respecto al segundo mejor modelo que es el VGG19.

Figura 130

Comparativo de los dos mejores modelos



Fuente: Elaboración propia

Capítulo V: Conclusiones y Recomendaciones

En este último capítulo se detalla las conclusiones de este trabajo de investigación con respecto a cada etapa realizada en la metodología. De la misma manera se detalla también las recomendaciones generales de este trabajo.

6.1. Conclusiones

La presente investigación inició con la obtención del dataset de imágenes de estos tipos de mosquitos Aedes y Culex. Este dataset se formó a partir de la obtención de 5 base de datos de imágenes que fueron filtradas solo para las especies de Aedes y Culex, que se ubicaban en repositorios de laboratorios de entomología de universidades y otros institutos. Este dataset formado por 2432 imágenes de tipo Aedes y 2081 de tipo Culex participaron en el experimento de entrenamiento de los modelos de inteligencia artificial, en la cual en base a los resultados obtenidos de los 4 modelos implementados, se revela que estas imágenes son adecuadas para ser utilizadas en este tipo de tareas debido a que el Accuracy de los 4 modelos son mayores al 95% y uno de ellos alcanza un 97.98%. Con estos valores se concluye que las imágenes aportan y son adecuadas para los procesos de clasificación de aprendizaje profundo. Esto debido a las imágenes correctamente etiquetadas y a la propia calidad de imagen de cada registro de las 5 base de datos que mostraron la morfología completa de cada mosquito.

Por consiguiente a este dataset set formado por las imágenes de Aedes y Culex se procedió a aplicar 4 técnicas de pre procesamiento. Primero, se aplicó el redimensionamiento de las imágenes con el objetivo de que tengan el adecuado tamaño para el ingreso a los modelos de inteligencia artificial y también para que el proceso de entrenamiento no sea demasiando extenso. Segundo, se aplicó el balanceo de datos para nivelar el número de la clase minoritaria y que ambas clases tengan las mismas cantidades de registro, esto con el objetivo de tener un mejor rendimiento en el entrenamiento de los modelos. Tercero, se

aplicó el aumento de datos para incrementar la cantidad de imágenes por cada clase, esto debido a que en los antecedentes de esta investigación esta técnica a generado buenos resultados en el rendimiento de los modelos y también por que los modelos de aprendizaje profundo requieren grandes cantidades de datos para tener un mejore rendimiento. Por último se aplicó la técnica de normalización de datos para convertir las imágenes de valores de 0 a 255 en valores de 0 y 1 y que los modelos pueden entrenar de mejor manera estos valores. De esta manera correlacionando con los resultados positivos de los 4 modelos implementados se puede concluir que estas técnicas de pre procesamiento han aportado a un mejor rendimiento de clasificación.

Para esta investigación se eligieron 4 modelos de aprendizaje profundo, tres de estos modelos fueron elegidos en base a que se utilizaron en los antecedes de esta investigación que obtuvieron buenos resultados en esos antecedente, estos modelos fueron el VGG16, VGG19 y el ResNet50. El cuarto modelo que es Vision Transformers fue elegido por ser uno de los últimos modelos de inteligencia artificial en tener un buen rendimiento en tareas de clasificación. Este cuarto modelo también se agregó al experimento para medir su nivel de competitividad de clasificación de este tipo de imágenes de insectos con respecto a los tres modelos robustos que tienen mayor antigüedad. En base a los resultados de los 4 modelos se puede concluir que todos los modelos implementados son adecuados para esta tarea de clasificación de este tipo de imágenes debido a que superan un Accuracy mayor o igual al 95% luego de una validación cruzada de k = 5. En la cual el modelo que ha tenido un mejor Accuracy es el modelo de Vision Transformers con un valor de 97.98%.

Los resultados obtenidos del mejor modelo a nivel de Accuracy con un 97.98%, precisión con un valor de 97.93%, recall con un valor de 97.73%, f1-score con un valor de 97.83%, especificidad con un valor de 97.94% y AUC de 97.99% revelan que el modelo

implementado en esta metodología cumple con la tarea de clasificación de este tipo de imágenes de insectos de tipo culícidos de la familia Aedes y Culex.

Cada etapa realizada en el presente trabajo de investigación está alineado a los objetivos planteados en el planteamiento del problema. Comenzando con la obtención la base de datos, la aplicación de pre procesamiento, la construcción de los modelos de aprendizaje profundo, la evaluación de los indicadores de rendimiento de cada modelo. De esta manera con las conclusiones de cada etapa se puede concluir de manera general que la implementación de un modelo de inteligencia artificial para la clasificación de imágenes de vectores de arbovirosis del Dengue, Zika y Chikungunya funciona de manera óptima con resultados que llegan al 97.98% de rendimiento en este tipo de tareas.

6.2 Recomendaciones

Con las conclusiones detalladas en cada etapa de esta investigación se recomienda a nivel técnico, construir una base de datos propia que permita aumentar un mayor número de imágenes y que aporte un mejor rendimiento al modelo. Así mismo a futuro implementar nuevos modelo de inteligencia artificial en la clasificación de este tipo de imágenes debido al avance constante de la tecnología y la generación de nuevos modelos de aprendizaje profundo.

Adicionalmente, como recomendación para abordar este tipo de problemas es importante profundizar los procesos de la vigilancia entomológica en el Perú para conocer sus deficiencias e informarse acerca de las especies de insectos que transportan las distintas enfermedades como el Dengue, Zika y Chikungunya.

Referencias

Abad, J. (2021). ¿La salud en el Perú está privatizada? | Conexión ESAN. Recuperado de https://www.esan.edu.pe/conexion-esan/la-salud-en-el-peru-esta-privatizada

Abad, J. (2020). El Perú y las epidemias: ¿Aprendimos la lección? | Conexión ESAN. Recuperado de https://www.esan.edu.pe/conexion-esan/el-peru-y-las-epidemias-aprendimos-la-leccion

Aboul, E., Ashraf, D. (2020). Machine Learning and Big Data Analytics Paradigms: Analysis, Applications and Challenges. Recuperado de https://www.google.com.pe/books/edition/Machine_Learning_and_Big_Data_Analytics/qfsOEAAAQBAJ?hl=es&gbpv=0

Adamssen. J. (2020). Inteligencia artificial: Aprendizaje automático, aprendizaje profundo y procesos de automatización. Recuperado de https://books.google.com.pe/books?id=KN74DwAAQBAJ&dq=historia+de+la+inteligencia+artificial&source=gbs_navlinks_s

Adhane, G., Mahdi, M., Masip, D. (2021). *A deep convolutional neural network for classification of Aedes albopictus mosquitoes*. Recuperado de https://ieeexplore.ieee.org/document/9429188

Appice, A., Gel, Y., Iliev, I., Lyubchich, V., Malerba, D. (2020). A Multi-Stage Machine Learning Approach to Predict Dengue Incidence: A Case Study in Mexico. IEEE Xplorer. https://ieeexplore.ieee.org/document/9035432

Aguirre, M., Amalia, G. (2020). Factores de mal pronóstico asociados a dengue grave en un hospital pediátrico del sureste de México durante el periodo de enero 2019-diciembre 2019.[Grado de Maestría, Universidad Nacional Autónoma de México].https://tesiunam.dgb.unam.mx/F/647RTXHDFKRVXP8LY8HA5HQDTBY57V

M4GS3HQRS7N281VLJ2XH-16184?func=full-set-set&set_number=120473&set_entry=000002&format=999

Aparecida, M., Saxena, S. (2020). *Dengue Fever in a One Health Perspective.* BoD – Books on Demand. Recuperado de https://books.google.com.pe/books?id=FJYtEAAAQBAJ&dq=el+virus+del+dengue&source=gbs_navlinks_s

Alfáu, A. (2012). *Plagas Domésticas: Historia – Patologías – Plaguicidas – Control.* Palibrio. Recuperado de https://books.google.com.pe/books?id=WQ9NAAAAQBAJ&dq=c%C3%BAlex&source=gbs_navlinks_s

Otamendi, U. (2021). Deep Learning object detection architectures in ADAS .[Grado de Titulación, Universidad Politécnica de Madrid]. https://tesiunam.dgb.unam.mx/F/647RTXHDFKRVXP8LY8HA5HQDTBY57VM4GS3HQRS7N281VLJ2XH-19388?func=full-set-set&set_number=120480&set_entry=000002&format=999

Bazi, Y., Bashmal, L., Rahhal, M., Al Dayil ,R., Al Ajlan, N. (2021). Vision Transformers for Remote Sensing Image Classification. MDPI. https://www.mdpi.com/2072-4292/13/3/516

Basurco, A., Villanueva, Sergio. (2021). Atlas de información al propietario. Parásitos: Diagnóstico, control y prevención. Grupo Asís Biomedia S.L. Recuperado de https://books.google.com.pe/books?id=V6dHEAAAQBAJ&dq=cul%C3%ADcidos&source=gbs_navlinks_s

Berlingerio, M., Bonchi, F., Gartner, T., Hurley, N., Geoirgiana, Ifrim. (2019). *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2018, Dublin, Ireland, September 10–14, 2018, Proceedings, Part II.* Springer. Recuperado de https://books.google.com.pe/books?id=Sd2EDwAAQBAJ&dq=auc+machine+learning&source=gbs_navlinks_s

Biswal, A. (2022). Top 10 Deep Learning Algorithms You Should Know in 2022. Recuperado de https://www.simplilearn.com/tutorials/deep-learning-tutorial/deep-learning-algorithm

Bhardwaj, A., Di, W., Wei, J. (2018). *Deep Learning Essentials: Your hands-on guide to the fundamentals of deep learning and neural network modeling*. Packt Publishing Ltd. Recuperado de https://books.google.com.pe/books?id=ISBKDwAAQBAJ&dq=deep+learning+models&source=gbs_navlinks_s

Bobadilla , J. (2021). Machine Learning y Deep Learning: Usando Python, Scikit y Keras. Ediciones de la U. https://books.google.com.pe/books?id=iAAyEAAAQBAJ&dq=Machine+learning&source=gbs_navlinks_s

Bohr. A, Memarzadeh. K. (2021) . Inteligencia Artificial En El Ámbito de la Salud. Recuperado de https://books.google.com.pe/books?id=KChAEAAAQBAJ&hl=es&source=gbs_navlinks_s

Brilian, A., Rusdhianto, A. (2020). CNN Architectures Performance Evaluation for Image Classification of Mosquito in Indonesia. Recuperado de https://ieeexplore.ieee.org/document/9163732

Brownlee, J. (2020). Imbalanced Classification with Python. Recuperado de https://www.google.com.pe/books/edition/Imbalanced_Classification_with_Python/jaXJDwAAQBAJ?hl=es&gbpv=0

Centro Nacional de Epidemiología, Prevención y Control de Enfermedades. Perú (2021). Boletín Epidemiológico del Perú 2021. Lima: CDC. Recuperado de https://www.dge.gob.pe/epipublic/uploads/boletin/boletin_202126_03_160533.pdf

Centro Nacional de Epidemiología, Prevención y Control de Enfermedades. Perú (2022). Sala de situación de salud Sem 41 – Dengue. Lima: CDC. Recuperado de https://www.dge.gob.pe/epipublic/uploads/dengue-diario/dengue-diario_202241_11_123529.pdf

Centro Nacional de Epidemiología, Prevención y Control de Enfermedades. Perú (2022). Sala de situación de salud Sem 52 – Dengue. Lima: CDC. Recuperado de https://www.dge.gob.pe/epipublic/uploads/asis-sala/asis-sala_202252_09_234037.pdf

Centro Nacional de Epidemiología, Prevención y Control de Enfermedades. Perú (2023). Sala de situación de salud – Dengue. Lima: CDC. Recuperado de https://www.dge.gob.pe/epipublic/uploads/asis-sala/asis-sala_20239_16_101206.pdf

Centro Nacional de Epidemiología de Prevención y Control de Enfermedades del Perú. 2021. *Situación epidemiológica del dengue y otras arbovirosis.* Recuperado de https://www.dge.gob.pe/portalnuevo/wp-content/uploads/2021/05/Vigilancia-Epidemiologica-de-dengue-y-otras-arbovirosis.pdf

Cuevas, E. (2021). Introducción al Machine Learning con MATLAB. Marcombo . https://books.google.com.pe/books?id=Ek1OEAAAQBAJ&dq=Machine+learning&source=gbs_navlinks_s

Cueto, M. (2020). El regreso de las epidemias: Salud y sociedad en el Perú del siglo XX. Instituto de Estudios Peruanos. https://books.google.com.pe/books?id=PF3dDwAAQBAJ&dq=Epidemias&source=gbs_navlinks_s

Centros para el Control y la Prevención de Enfermedades, Centro Nacional de Enfermedades Infecciosas Zoonóticas y Emergentes. 2019. *Transmisión.* Recuperado de https://www.cdc.gov/dengue/es/transmission/index.html

Centro Nacional de Epidemiología de Prevención y Control de Enfermedades del Perú. 2021. Número de casos de Chikungunya, Perú 2016 – 2021*. Recuperado de https://www.dge.gob.pe/portal/docs/vigilancia/sala/2021/SE07/chikun.pdf

Centro Nacional de Epidemiología de Prevención y Control de Enfermedades del Perú. 2021. *Incremento de incidencia de casos y ocurrencia de brotes por dengue, y otras arbovirosis en el país.* Recuperado de https://www.dge.gob.pe/epipublic/uploads/alertas/alertas_20219_30_084317.pdf

CDC. (2020). Informe de Evaluación de Resultados del Plan Estratégico Institucional 2019-2023 del Ministerio de salud. Recuperado de https://www.minsa.gob.pe/Recursos/OTRANS/13InformacionAdicional/evaluacion/acciones/pei_2020-2019_23.pdf

CDC. (2021). Código de Conducta del Ministerio de Salud. Recuperado de https://cdn.www.gob.pe/uploads/document/file/2078764/Codigo%20de%20conducta%20MINSA.pdf.pdf

CDC. (2022). Código de Conducta del Ministerio de Salud. Recuperado de https://www.dge.gob.pe/portalnuevo/vigilancia-epidemiologica/prevencion-y-control-de-d

Delisle, P., Anctil-Robitaille, B., Desrosiers, C., Lombaert, H. (2020). Realistic Image Normalization for Multi-Domain Segmentation. Recuperado de https://profs.etsmtl.ca/hlombaert/MedIA-Delisle-Normalization-2020.pdf

Dias,T., Alves,V., Alves,H.,Pinheiro,L., Pontes,R., Araujo,G., Lima,A., Prego,T. (2018). Autonomous Detection of Mosquito-Breeding Habitats Using an Unmanned Aerial Vehicle. https://ieeexplore.ieee.org/document/8588576?signout=success

Domingo, M., Saavedra ,D., Prasad, M. (2020). X-Ray Baggage Inspection With Computer Vision: A Survey. IEEE Xplorer. https://ieeexplore.ieee.org/document/9162101

Diario Gestión (2020). Minsa: el 51% de hospitales a nivel nacional con equipamiento obsoleto. Gestión. https://gestion.pe/economia/minsa-el-51-de-hospitales-a-nivel-nacional-con-equipamiento-obsoleto-noticia/

Diario Gestión (2022). Brotes de influenza y dengue en Perú: ¿en qué regiones del país se reportan más contagios?. Recuperado de https://gestion.pe/peru/brotes-de-influenza-y-dengue-en-peru-en-que-regiones-del-pais-se-reportan-mas-contagios-cifras-sintomas-enfermedades-virales-minsa-invierno-rmmn-noticia/

Domínguez, T. (2021). *Visión artificial: Aplicaciones prácticas con OpenCV – Python*. Marcombo. Recuperado de https://books.google.com.pe/books?id=FE1OEAAAQBAJ&dq=vision+artificial+es&source=gbs_navlinks_s

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Neil Houlsby. (2021). *An image is worth 16x16 words: transformers for image recognition at scale*. Recuperado de https://arxiv.org/pdf/2010.11929v2.pdf

Espinoza, J., Gómez, H. (2006). *Arbovirus causales de fiebre hemorrágico en pacientes del Instituto Mexicano del Seguro Social*. Recuperado de https://www.medigraphic.com/pdfs/imss/im-2006/im064i.pdf

Flores, R., Fernandez, J. (2008). *Las Redes Neuronales Artificiales*. Netbiblo. Recuperado de https://books.google.com.pe/books?id=X0uLwi1Ap4QC&dq=redes+neuronales+artificiales+es&source=gbs_navlinks_s

García, G. (2018). *Procesamiento de imágenes*. Recuperado de https://view.officeapps.live.com/op/view.aspx?src=http%3A%2F%2Fdis.um.es%2F~ginesgm%2Ffiles%2Fdoc%2Fpi%2Ftema4.ppt&wdOrigin=BROWSELINK

García, J. (2021).Comparativa de técnicas de balanceo de datos. Recuperado de https://digibuo.uniovi.es/dspace/bitstream/handle/10651/60629/TFM_Joaqu%C3%ADnGarc%C3%ADaAbad.pdf?sequence=4

García, Z., Yanai, K., Nakano, M., Arista, A., Cleofas, L., Perez H. (2019). *Mosquito Larvae Image Classification based on DenseNet and Guided Grad-CAM*. Recuperado de https://link.springer.com/chapter/10.1007/978-3-030-31321-0_21

Gállego, J. (2007). *Manual de Parasitología. Morfología y biología de los parásitos de interés sanitario*. Edicions Universitat Barcelona. Recuperado de https://books.google.com.pe/books?id=XH4yn_OANn4C&dq=aedes+aegypti+morfolog%C3%ADa&source=gbs_navlinks_s

Geroge, S., Titus, J. (2013). A Comparison Study On Different Interpolation Methods Based On Satellite. Recuperado de https://www.ijert.org/research/a-comparison-study-on-different-interpolation-methods-based-on-satellite-images-IJERTV2IS60011.pdf

Forbes US (2021). Visión artificial, la tecnología que promete una ola de startups multimillonarias. Recuperado de https://forbes.co/2021/03/09/editors-picks/vision-artificial-la-tecnologia-que-promete-una-ola-de-startups-multimillonarias

Hussain, T., Habib, Y., Syazril, A., Tanveer, N., Ahsan, A. (2020). *Expanded Dengue Syndrome.* Springer Nature. Recuperado de https://books.google.com.pe/books?id=I2gKEAAAQBAJ&dq=dengue&source=gbs_navlinks_s

IBM. 2018. ¿Qué es Deep Learning?. Recuperado de https://www.ibm.com/pe-es/cloud/deep-learning

Inga, R. (2022, 5 marzo). Las metas de infraestructura de calidad del sector salud son poco ambiciosas. La Cámara. https://lacamara.pe/las-metas-de-infraestructura-de-calidad-del-sector-salud-son-poco-ambiciosas/

INS. (25 de noviembre del 2020). Dengue: un mosquito transmisor puede poner hasta mil huevos durante su etapa de vida. Instituto Nacional de Salud. Recuperado de https://www.gob.pe/institucion/minsa/noticias/317155-dengue-un-mosquito-transmisor-puede-poner-hasta-mil-huevos-durante-su-etapa-de-vida

Jarmul K., Lawson R. 2017. Web Scraping with Python. Packt Publishing. Recuperado de https://books.google.com.pe/books?id=jHc5DwAAQBAJ&dq=web+scraping&hl=es&source=gbs_navlinks_s

Jiménez, M. (2012). *Los arbovirus emergentes y el cambio global*. Recuperado de https://www.madrimasd.org/blogs/virusemergentes/2012/02/25/287/

Joloudari, J., Marefat, A., Nematollahi, M., Sunday, S., Hussain, S. (2023). Effective Class-Imbalance Learning Based on SMOTE and Convolutional Neural Networks. Recuperado de https://www.mdpi.com/2076-3417/13/6/4006

Kamath, U., Graham, K., Emara W. (2022). Transformers for Machine Learning. Recuperado de https://books.google.com.pe/books/about/Transformers_for_Machine_Learning.html?id=9FC-zgEACAAJ&redir_esc=y

Keystone. J., Kozarsky. P., Connor. B., Nothdurft. H., Mendelson. M., Leder. K. (2020). Medicina del Viajero. Recuperado de https://books.google.com.pe/books?id=SVjwDwAAQBAJ&dq=mosquitos+Aedes&source=gbs_navlinks_s

Kumar, A.,Galindo, J.,Bhushan, M., Garg, L., Hu, Y. (2023). Machine Intelligence, Big Data Analytics, and IoT in Image Processing. Recuperado de https://www.google.com.pe/books/edition/Machine_Intelligence_Big_Data_Analytics/FmuvEAAAQBAJ?hl=es&gbpv=0

Lakshmanan, V., Gorner, M., Gillard, R. (2021). *Practical Machine Learning for Computer Vision.* Recuperado de https://books.google.com.pe/books?id=8ng5EAAAQBAJ&dq=image+preprocessing+deep+learning&source=gbs_navlinks_s

Lorenz, C., Ferraudo, A., Suesdek, L. (2015). *Artificial Neural Network applied as a methodology of mosquito species identification.* Recuperado de https://www.sciencedirect.com/science/article/pii/S0001706X1530108X

Métodos de vigilancia entomológica y control de los principales vectores en las Américas. (2021). En Pan American Health Organization eBooks. Pan American Health Organization. https://doi.org/10.37774/9789275323953

Mehenika, A., Mohammad, S., Tawsin, A., Karl, A. (2020). *Mosquito Classification using Convolutional Neural Network with Data Augmentation*. Recuperado de https://link.springer.com/chapter/10.1007/978-3-030-68154-8_74

Ministerio de Salud. (2020). Sistema Nacional de Salud articulará entidades públicas y privadas para prestar servicios equitativos de salud en el país. Recuperado de https://www.gob.pe/institucion/minsa/noticias/152700-sistema-nacional-de-salud-articulara-entidades-publicas-y-privadas-para-prestar-servicios-equitativos-de-salud-en-el-pais

Mirjalili,V y Raschka,S. (2020). Python Machine Learning. Marcombo. https://books.google.com.pe/books?id=5EtOEAAAQBAJ&dq=Machine+learning&source=gbs_navlinks_s

Muhammad, U., Saima,S., Muhammad, A., Saleem,U., Gyu Sang, C., Mehmood, A. (2020). A Novel Stacked CNN for Malarial Parasite Detection in Thin Blood Smear Images. IEEE Xplorer. https://ieeexplore.ieee.org/document/9093853

Moncada. L.(2021). Introducción al estudio de insectos de interés en salud pública. Recuperado de https://books.google.com.pe/books?id=HMg4EAAAQBAJ&dq=los+insectos&source=gbs_navlinks_s

Moreno. M. (2021). Inteligencia artificial (Umbrales éticos, Derecho y Administraciones Públicas). Recuperado de https://books.google.com.pe/books?id=9X5YEAAAQBAJ&dq=componentes+de+la+inteligencia+artificial&source=gbs_navlinks_s

Mayor, F., Fariñas, F., Vega, S. (2019). *One health: Cambio climático, contaminación ambiental y el impacto sobre la salud humana y animal.* Amazing Books. Recuperado de https://books.google.com.pe/books?id=If3FDwAAQBAJ&dq=tipos+de++aedes&source=gbs_navlinks_s

Marin, A., Novoa, A. (2014). *Conocer lo Social: Estrategias, técnicas de construcción y análisis de datos.* Antonio Lucas Marín. Recuperado de

https://books.google.com.pe/books?id=5JsWBAAAQBAJ&dq=cuasi+experimental+variable+independiente&source=gbs_navlinks_s

MathWorks.com. (2021). *Deep Learning: Tres cosas que es necesario saber.* Recuperado de https://la.mathworks.com/discovery/deep-learning.html

Marmanis, D.; Datcu, M.; Esch, T.; Stilla, U. Deep learning earth observation classification using imagenet pretrained networks. IEEE Geosci. Remote Sens. Lett. 2016, 13, 105–109. Recuperado de https://ieeexplore.ieee.org/document/7342907

Maggiori, E.; Tarabalka, Y.; Charpiat, G.; Alliez, P. Convolutional neural networks for large-scale remote-sensing image classification. IEEE Trans. Geosci. Remote Sens. 2017, 5. Recuperado de https://ieeexplore.ieee.org/document/7592858

Martinez, J. (2021). *TensorFlow 2.0 Computer Vision Cookbook: Implement machine learning solutions to overcome various computer vision challenges.* Packt Publishing Ltd. Recuperado de https://books.google.com.pe/books?id=9KgWEAAAQBAJ&dq=computer+vision&source=gbs_navlinks_s

Muños, J., Rodríguez, N (2017). *Pequeños pero matones: vectores minúsculos que suponen una amenaza mayúscula.* Recuperado de https://www.isglobal.org/es/healthisglobal/-/custom-blog-portlet/pro-1/91127/0

Nadda, W., Boonchieng, W., Boonchieng , E. (2022). Influenza, dengue and common cold detection using LSTM with fully connected neural network and keywords selection. Springer Link. https://link.springer.com/article/10.1186/s13040-022-00288-9/peer-review

National Geographic. (2020, 2 diciembre). Breve historia de la Inteligencia Artificial https://www.nationalgeographic.com.es/ciencia/breve-historia-visual-inteligencia-artificial_14419

Nogueira, K.; Miranda, W.O.; Santos, J.A.D. Improving spatial feature representation from aerial scenes by using convolutional networks. In Proceedings of the 2015 28th SIBGRAPI Conference on Graphics, Patterns and Images, Salvador, Brazil, 26–29. August 2015; pp. 289–296.

Ñaupas, H., Mejía, E., Novoa, E., Villagómez. (2014). *Metodología de la Investigación*. Ediciones de la U. Recuperado de https://books.google.com.pe/books?id=VzOjDwAAQBAJ&dq=Metodolog%C3%ADa+de+la+investigaci%C3%B3n++Cuantitativa+-+Cualitativa++y+Redacci%C3%B3n+de+la+Tesis&source=gbs_navlinks_s

Ocean, B., Necula, M. (2021). Web scraping techniques for price statistics – the Romanian experience. Statistical Journal. https://content.iospress.com/articles/statistical-journal-of-the-iaos/sji190529

OMS. 2019. *Dengue y Dengue Grave*. Recuperado de https://www.who.int/health-topics/dengue-and-severe-dengue#tab=tab_1

OMS. 2020. *Chikungunya*. Recuperado de https://www.who.int/es/news-room/fact-sheets/detail/chikungunya

OMS. (10 de enero del 2022). *Dengue y dengue grave. Organización Mundial de la Salud*. Recuperado de https://www.who.int/es/news-room/fact-sheets/detail/dengue-and-severe-dengue

Panigrahi, S., Nanda, A., Swarnkar, T. (2018). Deep Learning Approach for Image Classification. Recuperado de https://ieeexplore.ieee.org/document/8588970

Pérez, I., Gegúndez, E. (2021). Deep Learning. Servicio de Publicaciones de la Universidad de Huelva. Recuperado de

https://books.google.com.pe/books?id=kzsvEAAAQBAJ&dq=deep+learning&source=gbs_navlinks_s

Piattini, M. 2018. Calidad de datos. RA-MA. Recuperado de https://books.google.com.pe/books?id=yI6fDwAAQBAJ&dq=que+es+la+calidad+de+datos&source=gbs_navlinks_s

Raschka, S., Mirjalili, V. (2017). Python Machine Learning. Recuperado de https://www.google.com.pe/books/edition/Python_Machine_Learning/_plGDwAAQBAJ?hl=es&gbpv=0

Ram, A. (2021). *Spatial Distribution of Dengue Vectors*. Booksclinic Publishing. Recuperado de https://books.google.com.pe/books?id=BSI3EAAAQBAJ&dq=dengue&source=gbs_navlinks_s

RPP. (2022). ¿Cuándo ir a una posta y cuándo al hospital? Diferencias entre el primer, segundo y tercer nivel de atención. RPP. https://rpp.pe/campanas/valor-compartido/cuando-ir-a-una-posta-y-cuando-al-hospital-diferencias-entre-el-primer-segundo-y-tercer-nivel-de-atencion-noticia-1384837?ref=rpp

Rouhiainen. L. (2018). Inteligencia artificial: 101 cosas que debes saber hoy sobre nuestro futuro. Recuperado de https://books.google.com.pe/books?id=_T9xDwAAQBAJ&dq=inteligencia+artificial+en+la+actualidad&source=gbs_navlinks_s

Rosman, C., Cardellach, F. (2020). *Farreras Rozman Medicina Interna*. Elsevier Health Sciences. Recuperado de https://books.google.com.pe/books?id=_nfnDwAAQBAJ&dq=tratamientos+dengue&source=gbs_navlinks_s

Rovid, A., Roth, J., Galyon, J. (2011). Enfermedades Emergentes y Exóticas de los Animales. CFSPH Iowa State University. Recuperado de https://books.google.com.pe/books?id=s1R6wsyeT4IC&dq=vector+de+transmisi%C3%B3n&source=gbs_navlinks_s

Rojas, F. (2021). Régimen laboral del médico en el sector privado. Recuperado de https://lpderecho.pe/regimen-laboral-medico-sector-privado/

Rosado, P., Figueras, E., Planas, M., Reverter, F. (2016). *La visión artificial, un nuevo aliado para el análisis de imágenes artísticas*. Universidad de Barcelona. Recuperado de https://revistas.ucm.es/index.php/ARIS/article/view/48802

Ruckert, C., Weger-Lucarelli1, J ., M. Garcia-Luna, S ., Young, M ., Byas , A., Murrieta, R ., Fauver, J ., Ebel, G. (2017). *Impact of simultaneous exposure to arboviruses on infection and transmission by Aedes aegypti mosquitoes*. Recuperado de https://www.nature.com/articles/ncomms15412.pdf

Sanchez, M.(2014). El Sistema Nacional de Salud en el Perú. Recuperado de https://rpmesp.ins.gob.pe/rpmesp/article/view/129/1897

Sumoso, E. (2022). Desarrollo de una aplicación de visión artificial para escanear objetos 3d con cámaras de fotos.[Pregrado, Universidad Católica del Perú]. https://tesis.pucp.edu.pe/repositorio/handle/20.500.12404/21800

Silva, J. (2019). Sistemas de información en salud de dengue: estudio de caso sobre el abordaje de la epidemia ocurrida el año 2017 en Perú.[Grado de Maestria, Univerisdad Peruana Cayetano Heredia]. https://repositorio.upch.edu.pe/handle/20.500.12866/7245

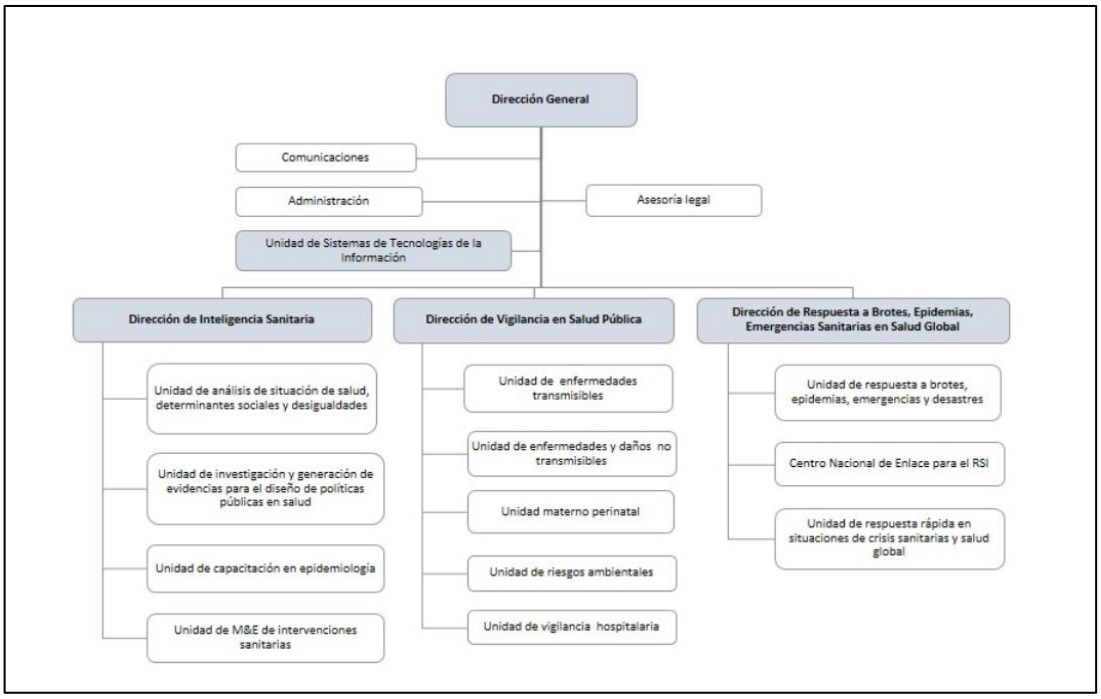
Valbuena. R. (2021). Inteligencia Artificial: Investigación Científica Avanzada Centrada en Datos. Recuperado de https://books.google.com.pe/books?id=SoMTEAAAQBAJ&dq=historia+de+la+inteligencia+artificial&source=gbs_navlinks_s

Vegara. S. (2021). Inteligencia artificial. Recuperado de https://tesis.pucp.edu.pe/repositorio/handle/20.500.12404/21800

Yavari, F., Dewi, K. (2021). Identifcation of signifcant climatic risk factors and machine learning models in dengue outbreak prediction. Science Direct. https://www.researchgate.net/publication/351247439_Identification_of_significant_climatic_

ANEXOS

Organigrama del sector epidemiológico



Mapa de Procesos del sector epidemiológico

